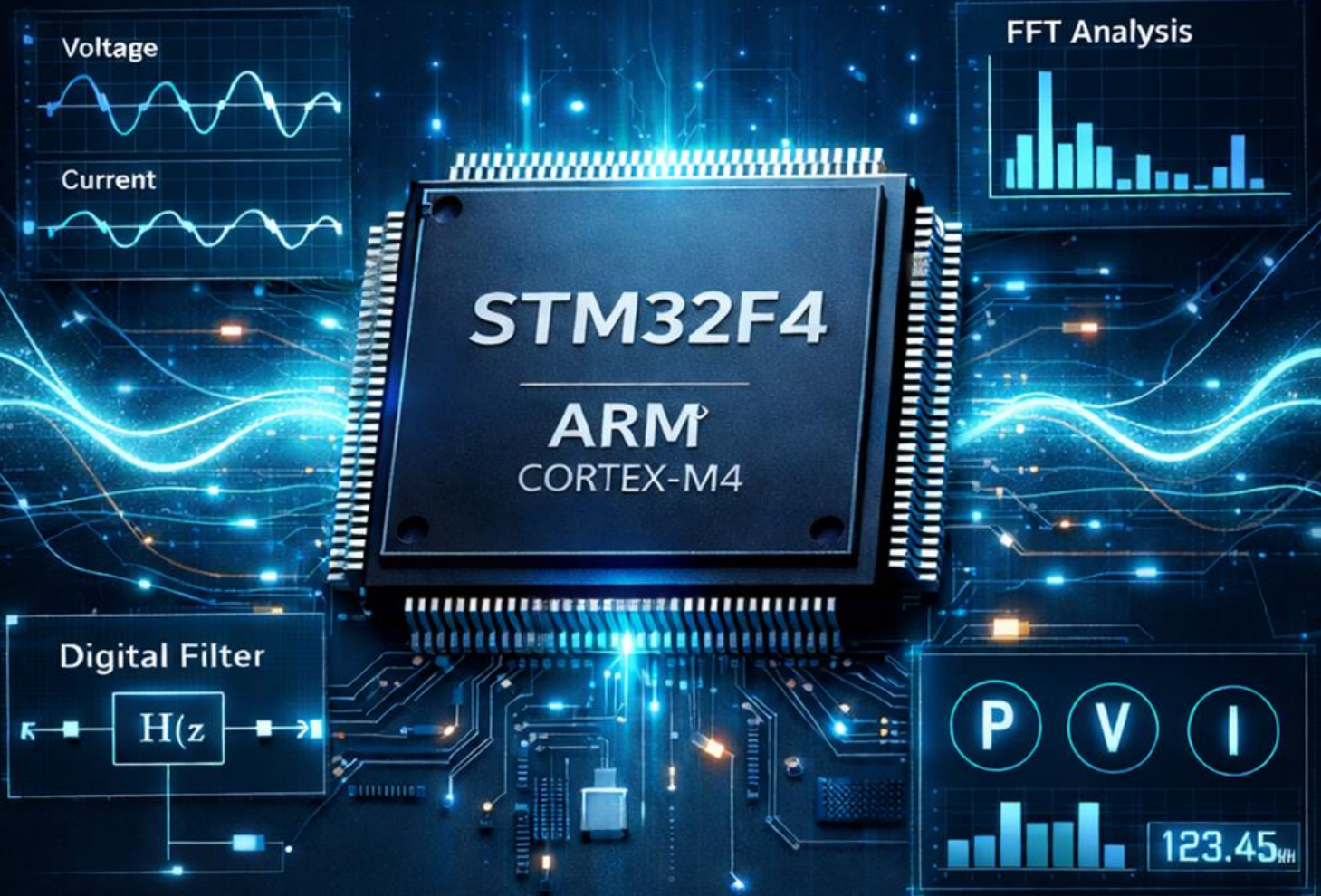


Energy_Data_Acquisition_With Custom_DSP



Project Report

January 2026

TABLE OF CONTENTS

1. HEADER FILES	2
<u>1.1 STM32F446xx.h Header File</u>	<u>3</u>
<u>1.2 I²C Driver.h Header File</u>	<u>41</u>
<u>1.3 USART Driver.h Header File</u>	<u>51</u>
<u>1.4 ADC DMA Driver.h Header File</u>	<u>68</u>
<u>1.5 Timer Driver.h Header File</u>	<u>74</u>
2. SOURCE CODE	76
<u>2.1 ADC DMA Driver.c Source File</u>	<u>77</u>
<u>2.2 I2C Driver.c Source File</u>	<u>82</u>
<u>2.3 UART Driver.c Source File</u>	<u>91</u>
<u>2.4 Timer Driver.c Source File</u>	<u>113</u>
<u>2.5 Energy Meter.c Source File</u>	<u>115</u>

STM32F446XX.H header file explanation

This is a hardware abstraction layer (HAL) header file for STM32F446xx microcontroller. It provides direct hardware access definitions for bare-metal or low-level embedded programming.

```
/*
 * =====
 *                               1. TYPE DEFINITIONS
 * =====
 */

typedef unsigned char      uint8_t;    // Define 8-bit unsigned integer
typedef unsigned short    uint16_t;   // Define 16-bit unsigned integer
typedef unsigned int       uint32_t;   // Define 32-bit unsigned integer
typedef unsigned long long uint64_t;   // Define 64-bit unsigned integer
typedef signed char        int8_t;     // Define 8-bit signed integer
typedef signed short       int16_t;    // Define 16-bit signed integer
typedef signed int         int32_t;    // Define 32-bit signed integer
typedef signed long long   int64_t;    // Define 64-bit signed integer
typedef float              float32_t;  // Define 32-bit floating point number
```

uint8_t - 8-bit Unsigned Integer

- **Size:** Exactly 1 byte (8 bits)
- **Range:** 0 to 255 (0x00 to 0xFF)
- **Memory layout:** Single byte in memory
- **Use cases:**
 - Storing ASCII characters
 - Boolean flags (0/1)
 - Small counters or indices
 - Peripheral register bytes
 - I/O port data

uint16_t - 16-bit Unsigned Integer

- **Size:** Exactly 2 bytes (16 bits)
- **Range:** 0 to 65,535 (0x0000 to 0xFFFF)
- **Memory layout:** 2 consecutive bytes (endianness dependent)
- **Use cases:**
 - ADC (Analog-to-Digital) values (typically 12-16 bit)
 - Timer counter values
 - Sensor readings

- Communication protocol data units
- Memory addresses in small microcontrollers

uint32_t - 32-bit Unsigned Integer

- **Size:** Exactly 4 bytes (32 bits)
- **Range:** 0 to 4,294,967,295 (0x00000000 to 0xFFFFFFFF)
- **Memory layout:** 4 consecutive bytes
- **Use cases:**
 - **STM32 Register Access:** All peripheral registers are 32-bit
 - Memory addresses (32-bit address space)
 - Large counters (millisecond timers)
 - CRC calculations
 - Bitmask operations on 32-bit registers
 - System time values

uint64_t - 64-bit Unsigned Integer

- **Size:** Exactly 8 bytes (64 bits)
- **Range:** 0 to 18,446,744,073,709,551,615 (0xFFFFFFFFFFFFFFFF)
- **Memory layout:** 8 consecutive bytes
- **Use cases:**
 - High-precision timing (nanosecond counters)
 - Large file sizes
 - Cryptographic operations
 - Double-precision calculations in fixed-point
 - Accumulating large sensor data sets

Signed Integer Types (int8_t, int16_t, int32_t, int64_t)

- **Two's complement representation:** Standard for negative numbers
- **Range symmetry:** $-2^{(n-1)}$ to $2^{(n-1)}-1$
- **Example:** int8_t ranges from -128 to 127
- **Use cases:**
 - Sensor values with negative readings (temperature, acceleration)
 - Error codes
 - Difference calculations

- Signal processing (audio, vibration)

float32_t - 32-bit Floating Point

- **IEEE 754 single-precision format**
- **Structure:**
 - 1 sign bit
 - 8 exponent bits
 - 23 mantissa bits
- **Range:** Approximately $\pm 3.4 \times 10^{38}$
- **Precision:** About 7 decimal digits
- **Use cases:**
 - Sensor data processing (temperature, pressure)
 - Control algorithms (PID controllers)
 - Signal filtering
 - Mathematical computations
 - Unit conversions

```

/*
 * =====
 *                               2. HARDWARE BASE ADDRESSES
 * =====
 */
#define RCC_BASE      0x40023800U    // Base address for Reset and Clock Control (RCC)
#define GPIOA_BASE    0x40020000U    // Base address for GPIO Port A
#define GPIOB_BASE    0x40020400U    // Base address for GPIO Port B
#define GPIOC_BASE    0x40020800U    // Base address for GPIO Port C
#define ADC1_BASE     0x40012000U    // Base address for ADC1 peripheral
#define DMA2_BASE     0x40026400U    // Base address for DMA2 controller
#define TIM2_BASE     0x40000000U    // Base address for Timer 2
#define I2C1_BASE     0x40005400U    // Base address for I2C1 peripheral
#define USART2_BASE   0x40004400U    // Base address for USART2 peripheral

```

What are Base Addresses?

In embedded systems, peripherals (hardware modules) are accessed by reading/writing to specific memory locations. Each peripheral has a **base address** - the starting point in memory where its registers begin.

RCC (Reset and Clock Control) - 0x40023800

Controls system clocks and peripheral enable/reset functions. You must enable a peripheral's clock through RCC before using it.

GPIO Ports (General Purpose Input/Output)

- **GPIOA - 0x40020000:** Port A pins (PA0-PA15)
- **GPIOB - 0x40020400:** Port B pins (PB0-PB15)

- **GPIOC - 0x40020800:** Port C pins (PC0-PC15)

ADC1 (Analog-to-Digital Converter) - 0x40012000

Converts analog voltages to digital values for sensor reading

DMA2 (Direct Memory Access) - 0x40026400

Transfers data between peripherals and memory without CPU involvement, improving efficiency (often used with ADC for continuous sampling)

TIM2 (Timer 2) - 0x40000000

General-purpose timer for PWM generation, delays, counting events, or time measurement

I2C1 (Inter-Integrated Circuit) - 0x40005400

Serial communication bus for connecting sensors and other ICs (oled displays, etc.)

USART2 (Universal Synchronous/Asynchronous Receiver/Transmitter) - 0x40004400

Serial communication interface, commonly used for debugging via USB-to-Serial adapters

```
/*
 * FPU Coprocessor Access Control Register
 * Using volatile for register access
 */
#define FPU_CPACR      (*((volatile uint32_t*)0xE000ED8U)) // Pointer to FPU Access Control Register
```

FPU Coprocessor Access Control Register Explanation

This code defines a macro to access the **FPU (Floating Point Unit) Coprocessor Access Control Register** on ARM Cortex-M4 processors.

```
/*
 * =====
 * 3. PERIPHERAL REGISTER STRUCTURES
 * =====
 * structs used for memory mapping must be volatile to prevent compiler reordering.
 */
// Structure definition for RCC registers
typedef struct {
    volatile uint32_t CR;           // Clock Control Register
    volatile uint32_t PLLCFGR;     // PLL Configuration Register
    volatile uint32_t CFGR;        // Clock Configuration Register
    volatile uint32_t CIR;         // Clock Interrupt Register
    volatile uint32_t AHB1RSTR;    // AHB1 Peripheral Reset Register
    volatile uint32_t AHB2RSTR;    // AHB2 Peripheral Reset Register
    volatile uint32_t AHB3RSTR;    // AHB3 Peripheral Reset Register
    volatile uint32_t RES0;        // Reserved
    volatile uint32_t APB1RSTR;    // APB1 Peripheral Reset Register
    volatile uint32_t APB2RSTR;    // APB2 Peripheral Reset Register
    volatile uint32_t RES1[2];     // Reserved
    volatile uint32_t AHB1ENR;     // AHB1 Peripheral Clock Enable Register
    volatile uint32_t AHB2ENR;     // AHB2 Peripheral Clock Enable Register
    volatile uint32_t AHB3ENR;     // AHB3 Peripheral Clock Enable Register
    volatile uint32_t RES2;        // Reserved
    volatile uint32_t APB1ENR;     // APB1 Peripheral Clock Enable Register
    volatile uint32_t APB2ENR;     // APB2 Peripheral Clock Enable Register
    volatile uint32_t BDCR;        // Backup Domain Control Register
    volatile uint32_t CSR;         // Clock Control & Status Register
    volatile uint32_t SSCGR;       // Spread Spectrum Clock Generation Register
    volatile uint32_t PLLI2SCFGR;  // PLLI2S Configuration Register
    volatile uint32_t PLLSAICFGR;  // PLLSAI Configuration Register
    volatile uint32_t DCKCFGR;     // Dedicated Clock Configuration Register
    volatile uint32_t CKGATENR;    // Clock Gating Control Register
    volatile uint32_t DCKCFGR2;    // Dedicated Clock Configuration Register 2
} RCC_TypeDef;
```

1. CR — Clock Control Register

Logic: Controls and monitors main clock sources (HSI, HSE, PLL).

Real-world usage:

- Turn ON HSI/HSE/PLL
 - Wait until clock becomes stable (RDY bits)
 - First register touched during clock initialization.
-

2. PLLCFGR — PLL Configuration Register

Logic: Defines how PLL multiplies/divides the input clock.

Real-world usage:

- Generate high-speed CPU clock (84/168 MHz)
 - Generate exact 48 MHz for USB
 - Performance tuning register.
-

3. CFGR — Clock Configuration Register

Logic: Selects SYSCLK source and sets bus prescalers.

Real-world usage:

- Choose SYSCLK = HSI / HSE / PLL
 - Divide clocks for AHB, APB1, APB2
 - Prevents peripherals from overclocking.
-

4. CIR — Clock Interrupt Register

Logic: Handles clock-related interrupts and flags.

Real-world usage:

- Rarely used
 - Used only in advanced fault detection systems
 - Mostly ignored in normal projects.
-

5. AHB1RSTR, AHB2RSTR, AHB3RSTR — AHB Reset Registers

Logic: Reset peripherals without resetting MCU.

Real-world usage:

- Reset GPIO, DMA, Ethernet, USB, etc.
 - Recover from peripheral lockups
 - Software “hard reset” for peripherals.
-

6. APB1RSTR, APB2RSTR — APB Reset Registers

Logic: Same as AHB reset but for APB peripherals.

Real-world usage:

- Reset TIM, I2C, SPI, UART, ADC
→ Used in driver re-initialization.
-

7. AHB1ENR, AHB2ENR, AHB3ENR — AHB Clock Enable Registers

Logic: Supplies clock to peripherals.

Real-world usage:

- Enable GPIO, DMA, CRC, USB OTG
 - Disable unused blocks to save power
→ **No clock = peripheral does not work** (very common bug).
-

8. APB1ENR, APB2ENR — APB Clock Enable Registers

Logic: Enables clocks for slower peripherals.

Real-world usage:

- Enable TIM, USART, I2C, SPI, ADC
→ Mandatory before accessing any peripheral registers.
-

9. BDCR — Backup Domain Control Register

Logic: Controls RTC and backup domain clocks.

Real-world usage:

- Enable LSE oscillator
 - Configure RTC clock source
→ Used in **energy meters, clocks, data loggers**.
-

10. CSR — Clock Control & Status Register

Logic: Tracks reset cause and low-speed clock status.

Real-world usage:

- Detect watchdog reset vs power-on reset
 - Debug unexpected MCU resets
→ Very useful for fault analysis.
-

11. SSCGR — Spread Spectrum Clock Generation

Logic: Slightly modulates clock frequency to reduce EMI.

Real-world usage:

- EMI-sensitive industrial products
→ Almost never used in hobby or normal firmware.
-

12. PLLI2SCFGR — PLLI2S Configuration

Logic: Separate PLL for audio clocks.

Real-world usage:

- I2S, audio DAC/ADC applications
→ Skip unless audio involved.
-

13. PLLSAICFGR — PLLSAI Configuration

Logic: Dedicated PLL for LCD, SAI, camera interfaces.

Real-world usage:

- LCD-TFT displays
 - Camera modules
→ Only for multimedia MCUs.
-

14. DCKCFGR — Dedicated Clock Configuration Register

Logic: Selects clock sources for specific peripherals.

Real-world usage:

- Choose USART, I2C, TIM clock source
→ Fixes baud-rate and timing accuracy issues.
-

15. CKGATENR — Clock Gating Enable Register

Logic: Enables/disables internal clock gating.

Real-world usage:

- Power optimization
→ Rarely touched directly.
-

16. DCKCFGR2 — Dedicated Clock Configuration Register 2

Logic: Extended peripheral clock selections.

Real-world usage:

- Advanced clock routing in complex systems
→ Mostly for high-end configurations.

6.3.28 RCC register map

Table 21 gives the register map and reset values.

Table 21. RCC register map and reset values

Addr. offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x00	RCC_CR	Res	Res	PLL SAIRDY	PLL SAION	PLL I2SRDY	PLL I2SON	PLL RDY	PLL ON	Res	Res	Res	Res	CSSON	HSEBYP	HSE RDY	HSEON	HSICAL[7:0]				HSITRIM[4:0]				Res	HSIRDY	HSION					
	Reset value			0	0	0	0	0	0			Res	Res		0	0	0	0	x	x	x	x	x	x	x	x	1	0	0	0	0		1
0x04	RCC_PLL CFGR	PLL R [2:0]		PLL Q[3:0]			Res		PLL SRC	Res	Res	Res	Res	Res	Res	PLL P [1:0]	Res	PLL N[8:0]				PLL M[5:0]				Res							
	Reset value		0	1	0	0	1	0	0		0					0	0		0	1	1	0	0	0	0	0	0	0	0	1	0	0	0
0x08	RCC_CFGR	MCO 2 [1:0]	MCO2 PRE [2:0]		MCO1 PRE [2:0]		Res		MCO 1 [1:0]	RTCPRE[4:0]				PPRE2 [2:0]		PPRE1 [2:0]		Res	HPRE[3:0]				SWS [1:0]		SW [1:0]		Res						
	Reset value	0	0	0	0	0	0	0		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x0C	RCC_CIR	Res	Res	Res	Res	Res	Res	Res	Res	CSSC	PLLSAIRDYC	PLLI2SRDYC	PLLIRDYC	HSE RDYC	HSIRDYC	LSERDYC	LSIRDYC	Res	PLLSAIRDYE	PLLI2SRDYE	PLLIRDYE	HSE RDYE	HSIRDYE	LSERDYE	LSIRDYE	CSSF	PLLSAIRDYF	PLLI2SRDYF	PLLIRDYF	HSE RDYF	HSIRDYF	LSERDYF	LSIRDYF
	Reset value								0		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x10	RCC_AHB1 RSTR	Res	Res	OTGHSRST	Res	Res	Res	Res	Res	Res	DMA2RST	DMA1RST	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res
	Reset value			0							0	0									0						0	0	0	0	0	0	0
0x14	RCC_AHB2 RSTR	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res
	Reset value																										0						
0x18	RCC_AHB3 RSTR	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res
	Reset value																																
0x1C	Reserved	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res
	Reset value																																
0x20	RCC_APB1 RSTR	Res	Res	DACRST	PWRST	CECRST	CANRST	CAN1RST	FMP2C1RST	IC3RST	IC2RST	IC1RST	UART5RST	UART4RST	UART3RST	UART2RST	SPDIFRST	SPI3RST	SPI2RST	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res
	Reset value			0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0													

Table 21. RCC register map and reset values (continued)

Addr. offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x50	RCC_AHB1 LPENR	Res	OTGHSULP	PEN	OTGHSLPEN	Res	Res	Res	Res	Res	DMA2LPEN	DMA1LPEN	Res	Res	BKPSRAMLPEN	SRAM2LPEN	SRAM1LPEN	FLITFLPEN	Res	Res	CRCLPEN	Res	Res	Res	Res	Res	GPIOHLPEN	GPIOGLPEN	GPIOFLPEN	GPIOELPEN	GPIOOLPEN	GPIOCLPEN	GPIOALPEN
	Reset value		1	1							1	1			1	1	1	1			1					1	1	1	1	1	1	1	
0x54	RCC_AHB2 LPENR	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	OTGFSLPEN	Res	Res	Res	Res	Res	DCMLPEN
	Reset value																									1							1
0x58	RCC_AHB3 LPENR	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	QSPILPEN
	Reset value																															1	1
0x5C	Reserved	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res
	Reset value																																
0x60	RCC_APB1 LPENR	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res
	Reset value				1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
0x64	RCC_APB2 LPENR	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res
	Reset value										1	1				1	1	1			1	1	1	1	1	1	1	1	1	1	1	1	1
0x68	Reserved	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res
	Reset value																																
0x6C	Reserved	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res
	Reset value																																
0x70	RCC_BDCR	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res
	Reset value																																
0x74	RCC_CSR	LPWRRSTF	WDGRSTF	WDGRSTF	SFTRSTF	PORRSTF	PADRSTF	BORRSTF	RMVF	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res
	Reset value	0	0	0	0	1	1	1	0																								0

Table 21. RCC register map and reset values (continued)

Addr. offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x50	RCC_AHB1 LPENR	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res
	Reset value	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
0x54	RCC_AHB2 LPENR	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res
	Reset value	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
0x58	RCC_AHB3 LPENR	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res
	Reset value	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
0x5C	Reserved	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res
	Reset value	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
0x60	RCC_APB1 LPENR	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res
	Reset value	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
0x64	RCC_APB2 LPENR	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res
	Reset value	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
0x68	Reserved	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res
	Reset value	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
0x6C	Reserved	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res
	Reset value	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
0x70	RCC_BDCR	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x74	RCC_CSR	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Table 21. RCC register map and reset values (continued)

Addr. offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x78	Reserved	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
	Reset value																																
0x7C	Reserved	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
	Reset value																																
0x80	RCC_SS CGR	SSCGEN	SPREADSEL			INCSTEP																MODPER											
	Reset value	0	0			0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x84	RCC_PLLI2 SCFGR	Res.	PLLI2SR [2:0]		PLLI2SQ [3:0]											PLLI 2SP [1:0]			PLLI2SN[8:0]								PLLI2SM[5:0]						
	Reset value		0	1	0	0	1	0	0							0	0		0	1	1	0	0	0	0	0	0	0	0	1	0	0	0
0x88	RCC_PLL SAI CFGR	Res.	PLLSAIR [2:0]		PLLSAIQ [3:0]											PLLSAIP[1:0]			PLLSAIN[8:0]								PLLSAIM[5:0]						
	Reset value		0	0	0	0	1	0	0							0	0		0	1	1	0	0	0	0	0	0	0	0	1	0	0	0
0x8C	RCC_DCK CFGR	Res.	Res.	Res.	I2S2SRC[1:0]		I2S1SRC[1:0]		TIMPRE		SAI2SCR[1:0]		SAI1SCR[1:0]								PLLSAIDIVQ [4:0]						Res.	Res.	Res.	PLLI2SDIVQ [4:0]			
	Reset value				0	0	0	0	0	0	0	0	0	0								0	0	0	0	0				0	0	0	0
0x90	RCC_CK GATENR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	RCC_CKEN	FLITF_CKEN	SRAM_CKEN	SPARE_CKEN	CM4DBG_CKEN	AHB2APB2_CKEN	AHB2APB1_CKEN
	Reset value																										0	0	0	0	0	0	0
0x94	RCC_DCK CFGR2	Res.	Res.	SPDIFRXSEL	SDIOSEL	CK48MSEL	CECSEL				FMP I2C1 SEL [1:0]																						
	Reset value			0	0	0	0				0	0																					

```
// Structure definition for GPIO registers
typedef struct {
    volatile uint32_t MODER;           // GPIO Mode Register
    volatile uint32_t OTYPER;          // GPIO Output Type Register
    volatile uint32_t OSPEEDR;         // GPIO Output Speed Register
    volatile uint32_t PUPDR;           // GPIO Pull-up/Pull-down Register
    volatile uint32_t IDR;              // GPIO Input Data Register
    volatile uint32_t ODR;              // GPIO Output Data Register
    volatile uint32_t BSRR;             // GPIO Bit Set/Reset Register
    volatile uint32_t LCKR;             // GPIO Configuration Lock Register
    volatile uint32_t AFR1;             // GPIO Alternate Function Low Register
    volatile uint32_t AFRH;            // GPIO Alternate Function High Register
} GPIO_TypeDef;
```

1. MODER — GPIO Mode Register

Logic: Selects *what a pin is*.

Each pin uses **2 bits** to define its mode.

Modes:

- 00 → Input
- 01 → Output
- 10 → Alternate Function
- 11 → Analog

Real-world usage:

- Button → Input
- LED → Output
- UART/SPI/I2C → Alternate Function
- ADC → Analog

→ First register you touch when configuring a pin.

2. OTYPER — Output Type Register

Logic: Defines *how* a pin drives output.

Types:

- 0 → Push-Pull
- 1 → Open-Drain

Real-world usage:

- LED, relay → Push-Pull
- I2C lines → Open-Drain

→ Matters only when pin is in **output or AF mode**.

3. OSPEEDR — Output Speed Register

Logic: Controls signal rise/fall time (slew rate).

Speeds:

- Low
- Medium
- High
- Very High

Real-world usage:

- Low speed → Buttons, LEDs (less EMI, less power)
- High speed → SPI, fast toggling signals

→ Not CPU speed — **electrical switching speed**.

4. PUPDR — Pull-Up / Pull-Down Register

Logic: Prevents floating inputs.

Options:

- 00 → No pull
- 01 → Pull-Up
- 10 → Pull-Down

Real-world usage:

- Button without resistor → Enable pull-up/down
- Unused pin → Define safe state

→ Very important for stable input reading.

5. IDR — Input Data Register

Logic: Reads the *actual logic level* on pins.

Real-world usage:

- Read button state
- Read sensor digital output

→ **Read-only** register.

6. ODR — Output Data Register

Logic: Stores output values for pins.

Real-world usage:

- Turn LED ON/OFF
- Drive GPIO output manually

⚠ Writing here can cause **read-modify-write issues**.

7. BSRR — Bit Set/Reset Register

Logic: Atomic way to set or reset pins.

Structure:

- Lower 16 bits → Set pin
- Upper 16 bits → Reset pin

Real-world usage:

- Fast, safe GPIO toggling
- Interrupt-safe GPIO control

→ **Preferred over ODR** in professional code.

8. LCKR — Configuration Lock Register

Logic: Locks GPIO configuration until reset.

Real-world usage:

- Safety-critical pins
- Prevent accidental reconfiguration

→ Rare but important in industrial systems.

9. AFRL — Alternate Function Low Register (Pins 0–7)

Logic: Selects which peripheral controls the pin.

Real-world usage:

- Map UART, SPI, I2C, TIM, etc.
- Example: PA9 → USART1_TX

→ Used only when MODER = Alternate Function.

10. AFRH — Alternate Function High Register (Pins 8–15)

Logic: Same as AFRL but for higher pins.

Real-world usage:

- Configure high-numbered pins for peripherals

→ AFRL + AFRH together cover all 16 pins.


```
// Structure definition for DMA Stream registers
typedef struct {
    volatile uint32_t CR;           // DMA Stream x Configuration Register
    volatile uint32_t NDTR;        // DMA Stream x Number of Data Register
    volatile uint32_t PAR;         // DMA Stream x Peripheral Address Register
    volatile uint32_t M0AR;        // DMA Stream x Memory 0 Address Register
    volatile uint32_t M1AR;        // DMA Stream x Memory 1 Address Register
    volatile uint32_t FCR;         // DMA Stream x FIFO Control Register
} DMA_Stream_TypeDef;
```

1. CR — DMA Stream Configuration Register

9.5.5 DMA stream x configuration register (DMA_SxCR)

This register is used to configure the concerned stream.

Address offset: $0x010 + 0x18 * x$, ($x = 0$ to 7)

Reset value: $0x0000\ 0000$

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	CHSEL[2:0]			MBURST [1:0]		PBURST[1:0]		Res.	CT	DBM	PL[1:0]	
				rw	rw	rw	rw	rw	rw	rw		rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PINC OS	MSIZE[1:0]		PSIZE[1:0]		MINC	PINC	CIRC	DIR[1:0]		PF CTRL	TCIE	HTIE	TEIE	DMEIE	EN
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Logic: This register defines *how* the DMA transfer behaves.

Controls:

- Transfer direction (Peripheral→Memory, Memory→Peripheral, Memory→Memory)
- Circular mode
- Increment modes (memory / peripheral)
- Data width (8/16/32-bit)
- Priority level
- Interrupt enable (TC, HT, TE)
- Channel selection

Real-world usage:

- ADC → RAM continuous sampling
- UART RX without CPU polling
- SPI TX/RX at high speed

→ This is the **brain of the DMA stream**.

2. NDTR — Number of Data Register

9.5.6 DMA stream x number of data register (DMA_SxNDTR)

Address offset: $0x014 + 0x18 * x$, ($x = 0$ to 7)

Reset value: $0x0000\ 0000$

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
NDT[15:0]															
rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW

Logic: Holds how many data items to transfer.

Important behavior:

- Loaded before enabling DMA
- Decrements automatically during transfer
- Reaches zero → transfer complete

Real-world usage:

- ADC buffer size (e.g., 256 samples)
- UART RX packet length
- SPI frame count

→ DMA stops (or reloads in circular mode) when this hits zero.

3. PAR — Peripheral Address Register

9.5.7 DMA stream x peripheral address register (DMA_SxPAR)

Address offset: $0x018 + 0x18 * x$, ($x = 0$ to 7)

Reset value: $0x0000\ 0000$

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
PAR[31:16]															
rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PAR[15:0]															
rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW

Logic: Stores the fixed address of the peripheral data register.

Real-world usage:

- `&ADC1->DR`
- `&USART2->DR`
- `&SPI1->DR`

→ Usually **constant**, never changes during transfer.

4. M0AR — Memory 0 Address Register

9.5.8 DMA stream x memory 0 address register (DMA_SxM0AR)

Address offset: $0x01C + 0x18 * x$, ($x = 0$ to 7)

Reset value: $0x0000\ 0000$

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
M0A[31:16]															
rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
M0A[15:0]															
rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW

Logic: Points to the first memory buffer.

Real-world usage:

- ADC sample buffer
- UART RX buffer
- Audio sample buffer

→ Used in **normal and circular modes**.

5. M1AR — Memory 1 Address Register

9.5.9 DMA stream x memory 1 address register (DMA_SxM1AR)

Address offset: $0x020 + 0x18 * x$, ($x = 0$ to 7)

Reset value: $0x0000\ 0000$

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
M1A[31:16]															
rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
M1A[15:0]															
rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW

Logic: Second memory buffer for **double-buffer mode**.

Real-world usage:

- Ping-pong buffers in audio
- Continuous ADC sampling without data loss

→ While DMA fills one buffer, CPU processes the other.

6. FCR — FIFO Control Register

9.5.10 DMA stream x FIFO control register (DMA_SxFCR)

Address offset: $0x024 + 0x18 * x$, ($x = 0$ to 7)

Reset value: $0x0000\ 0021$

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	FEIE	Res.	FS[2:0]			DMDIS	FTH[1:0]	
								rw		r	r	r	rw	rw	rw

Logic: Controls DMA internal FIFO behavior.

Controls:

- FIFO enable/disable
- FIFO threshold level
- Direct mode error flags

Real-world usage:

- High-throughput transfers
- Prevent bus congestion
- Improve performance in burst transfers

→ Often left default unless performance tuning is needed.

```
// Structure definition for DMA Controller registers
typedef struct {
    volatile uint32_t LISR;           // DMA Low Interrupt Status Register
    volatile uint32_t HISR;           // DMA High Interrupt Status Register
    volatile uint32_t LIFCR;          // DMA Low Interrupt Flag Clear Register
    volatile uint32_t HIFCR;          // DMA High Interrupt Flag Clear Register
} DMA_TypeDef;
```

1. LISR — Low Interrupt Status Register

9.5.1 DMA low interrupt status register (DMA_LISR)

Address offset: $0x000$

Reset value: $0x0000\ 0000$

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	TCIF3	HTIF3	TEIF3	DMEIF3	Res.	FEIF3	TCIF2	HTIF2	TEIF2	DMEIF2	Res.	FEIF2
				r	r	r	r		r	r	r	r	r		r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	TCIF1	HTIF1	TEIF1	DMEIF1	Res.	FEIF1	TCIF0	HTIF0	TEIF0	DMEIF0	Res.	FEIF0
				r	r	r	r		r	r	r	r	r		r

Logic: Shows interrupt status for **Streams 0–3**.

Flags per stream:

- Transfer complete (TCIF)
- Half transfer (HTIF)
- Transfer error (TEIF)
- Direct mode error (DMEIF)
- FIFO error (FEIF)

Real-world usage:

- Check if DMA transfer finished
- Detect runtime transfer errors
- Used inside DMA IRQ handler

→ Read-only status register.

2. HISR — High Interrupt Status Register**9.5.2 DMA high interrupt status register (DMA_HISR)**

Address offset: 0x004

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	TCIF7	HTIF7	TEIF7	DMEIF7	Res.	FEIF7	TCIF6	HTIF6	TEIF6	DMEIF6	Res.	FEIF6
				r	r	r	r		r	r	r	r	r		r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	TCIF5	HTIF5	TEIF5	DMEIF5	Res.	FEIF5	TCIF4	HTIF4	TEIF4	DMEIF4	Res.	FEIF4
				r	r	r	r		r	r	r	r	r		r

Logic: Same as LISR but for **Streams 4–7**.

Real-world usage:

- Same purpose as LISR
- Needed when using higher-numbered DMA streams

→ Which register you read depends on **stream number**.

3. LIFCR — Low Interrupt Flag Clear Register

9.5.3 DMA low interrupt flag clear register (DMA_LIFCR)

Address offset: 0x008

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	CTCIF3	CHTIF3	CTEIF3	CDMEIF3	Res.	CFEIF3	CTCIF2	CHTIF2	CTEIF2	CDMEIF2	Res.	CFEIF2
				w	w	w	w		w	w	w	w	w		w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	CTCIF1	CHTIF1	CTEIF1	CDMEIF1	Res.	CFEIF1	CTCIF0	CHTIF0	CTEIF0	CDMEIF0	Res.	CFEIF0
				w	w	w	w		w	w	w	w	w		w

Logic: Clears interrupt flags for **Streams 0–3**.

Important rule:

- Writing 1 clears the corresponding flag
- Writing 0 does nothing

Real-world usage:

- Clear TC/HT/TE flags after handling interrupt
- Prevent interrupt from retriggering forever

→ Always clear flags here, never in LISR.

4. HIFCR — High Interrupt Flag Clear Register

9.5.4 DMA high interrupt flag clear register (DMA_HIFCR)

Address offset: 0x00C

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	CTCIF7	CHTIF7	CTEIF7	CDMEIF7	Res.	CFEIF7	CTCIF6	CHTIF6	CTEIF6	CDMEIF6	Res.	CFEIF6
				w	w	w	w		w	w	w	w	w		w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	CTCIF5	CHTIF5	CTEIF5	CDMEIF5	Res.	CFEIF5	CTCIF4	CHTIF4	CTEIF4	CDMEIF4	Res.	CFEIF4
				w	w	w	w		w	w	w	w	w		w

Logic: Clears interrupt flags for **Streams 4–7**.

Real-world usage:

- Same as LIFCR, but for higher streams

```
// Structure definition for ADC registers
typedef struct {
    volatile uint32_t SR;           // ADC Status Register
    volatile uint32_t CR1;          // ADC Control Register 1
    volatile uint32_t CR2;          // ADC Control Register 2
    volatile uint32_t SMPR1;        // ADC Sample Time Register 1
    volatile uint32_t SMPR2;        // ADC Sample Time Register 2
    volatile uint32_t JOFR1;        // ADC Injected Channel Data Offset Register 1
    volatile uint32_t JOFR2;        // ADC Injected Channel Data Offset Register 2
    volatile uint32_t JOFR3;        // ADC Injected Channel Data Offset Register 3
    volatile uint32_t JOFR4;        // ADC Injected Channel Data Offset Register 4
    volatile uint32_t HTR;           // ADC Watchdog Higher Threshold Register
    volatile uint32_t LTR;           // ADC Watchdog Lower Threshold Register
    volatile uint32_t SQR1;          // ADC Regular Sequence Register 1
    volatile uint32_t SQR2;          // ADC Regular Sequence Register 2
    volatile uint32_t SQR3;          // ADC Regular Sequence Register 3
    volatile uint32_t JSQR;          // ADC Injected Sequence Register
    volatile uint32_t JDR1;          // ADC Injected Data Register 1
    volatile uint32_t JDR2;          // ADC Injected Data Register 2
    volatile uint32_t JDR3;          // ADC Injected Data Register 3
    volatile uint32_t JDR4;          // ADC Injected Data Register 4
    volatile uint32_t DR;            // ADC Regular Data Register
} ADC_TypeDef;
```

ADC = analog → digital converter.

Logically, ADC works in this order:

1. Configure **how** ADC works
2. Configure **which channels & sequence**
3. Start conversion
4. Read result
5. Handle events/errors

Each register exists to support **one of these steps**.

1. SR — Status Register

Logic: Tells you *what just happened*.

Flags include:

- End of conversion (EOC)
- End of sequence (EOS)
- Overrun (OVR)
- Analog watchdog flag

Real-world usage:

- Polling ADC conversion complete
- Debug data loss (overrun)
- Interrupt handling

→ Read-only flags, cleared by software sequence.

2. CR1 — Control Register 1

Logic: Controls **ADC behavior and monitoring**.

Controls:

- Resolution (12/10/8/6 bit)
- Scan mode
- Interrupt enable
- Analog watchdog enable
- Discontinuous mode

Real-world usage:

- Multi-channel ADC
 - Lower resolution for faster sampling
 - Safety monitoring (over/under voltage)
-

3. CR2 — Control Register 2

Logic: Controls **conversion execution**.

Controls:

- ADC ON/OFF
- Start conversion (SWSTART)
- Continuous mode
- DMA enable
- External trigger selection

Real-world usage:

- Continuous ADC + DMA
- Timer-triggered sampling
- Low-power controlled sampling

→ Most frequently modified control register.

4. SMPR1 — Sample Time Register 1 (Channels 10–18)

5. SMPR2 — Sample Time Register 2 (Channels 0–9)

Logic: Controls **how long ADC samples input voltage**.

Real-world usage:

- High-impedance sensors → longer sample time
- Fast signals → shorter sample time

→ Wrong sample time = wrong voltage reading (very common bug).

6. JOFR1–JOFR4 — Injected Channel Offset Registers

Logic: Subtract offset from injected conversion result.

Real-world usage:

- Sensor calibration
- Current sensing
- Motor control applications

→ Used only with **injected channels**.

7. HTR — Analog Watchdog High Threshold

8. LTR — Analog Watchdog Low Threshold

Logic: Hardware threshold comparator.

Real-world usage:

- Over-voltage protection
- Under-voltage detection
- Alarm systems

→ ADC can raise interrupt **without CPU polling**.

9. SQR1 — Regular Sequence Register 1

10. SQR2 — Regular Sequence Register 2

11. SQR3 — Regular Sequence Register 3

Logic: Defines **which channels are converted and in what order**.

Details:

- Sequence length (1–16 conversions)
- Channel order

Real-world usage:

- Read multiple sensors in one ADC run
- ADC scan mode with DMA

→ ADC **does not guess channels**—you must define sequence.

12. JSQR — Injected Sequence Register

Logic: Separate high-priority conversion sequence.

Real-world usage:

- Motor control
- Emergency current/voltage sensing

→ Can interrupt regular ADC sequence.

13. JDR1–JDR4 — Injected Data Registers

Logic: Stores injected conversion results.

Real-world usage:

- Fast critical measurements

→ Separate from normal ADC data.

14. DR — Regular Data Register

Logic: Holds latest ADC conversion result.

Real-world usage:

- Read ADC value (polling)
- DMA source register

→ **Most used ADC register.**

Table 91. ADC register map and reset values for each ADC

Offset	Register	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x00	ADC_SR	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	OVR	STRT	JSTRT	JEOC	EOC	AWD
	Reset value																											0	0	0	0	0	0
0x04	ADC_CR1	Res	Res	Res	Res	Res	OVRIE	RES(1:0)	AWDEN	JAWDEN									DISC NUM [2:0]	JDISCEN	DISCEN	JAUTO	AWD SGL	SCAN	JEOCIE	AWDIE	EOCIE	AWDCH[4:0]					
	Reset value						0	0	0	0	0								0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Table 91. ADC register map and reset values for each ADC (continued)

Offset	Register	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0				
0x08	ADC_CR2	SWSTART	EXTEN[1:0]		EXTSEL [3:0]					JSWSTART	JEXTEN[1:0]		JEXTSEL [3:0]										ALIGN	EOCS	DDS	DMA						CONT	ADON				
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0					0	0	0	0						0	0				
0x0C	ADC_SMPR1	Sample time bits SMPx_x																																			
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0				
0x10	ADC_SMPR2	Sample time bits SMPx_x																																			
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0				
0x14	ADC_JOFR1																					JOFFSET1[11:0]															
	Reset value																					0	0	0	0	0	0	0	0	0	0	0	0				
0x18	ADC_JOFR2																					JOFFSET2[11:0]															
	Reset value																					0	0	0	0	0	0	0	0	0	0	0	0				
0x1C	ADC_JOFR3																					JOFFSET3[11:0]															
	Reset value																					0	0	0	0	0	0	0	0	0	0	0	0				
0x20	ADC_JOFR4																					JOFFSET4[11:0]															
	Reset value																					0	0	0	0	0	0	0	0	0	0	0	0				
0x24	ADC_HTR																					HT[11:0]															
	Reset value																					1	1	1	1	1	1	1	1	1	1	1	1				
0x28	ADC_LTR																					LT[11:0]															
	Reset value																					0	0	0	0	0	0	0	0	0	0	0	0				
0x2C	ADC_SQR1					L[3:0]				Regular channel sequence SQx_x bits																											
	Reset value									0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0				
0x30	ADC_SQR2	Regular channel sequence SQx_x bits																																			
	Reset value			0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0				
0x34	ADC_SQR3	Regular channel sequence SQx_x bits																																			
	Reset value			0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0				
0x38	ADC_JSQR									JL[1:0]				Injected channel sequence JSQx_x bits																							
	Reset value											0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0				
0x3C	ADC_JDR1																					JDATA[15:0]															
	Reset value																					0	0	0	0	0	0	0	0	0	0	0	0				
0x40	ADC_JDR2																					JDATA[15:0]															
	Reset value																					0	0	0	0	0	0	0	0	0	0	0	0				
0x44	ADC_JDR3																					JDATA[15:0]															
	Reset value																					0	0	0	0	0	0	0	0	0	0	0	0				
0x48	ADC_JDR4																					JDATA[15:0]															
	Reset value																					0	0	0	0	0	0	0	0	0	0	0	0				
0x4C	ADC_DR	Regular DATA[15:0]																																			
	Reset value																					0	0	0	0	0	0	0	0	0	0	0	0				

```
// Structure definition for Timer registers
typedef struct {
    volatile uint32_t CR1;        // TIM control register 1
    volatile uint32_t CR2;        // TIM control register 2
    volatile uint32_t SMCR;       // TIM slave mode control register
    volatile uint32_t DIER;       // TIM DMA/interrupt enable register
    volatile uint32_t SR;         // TIM status register
    volatile uint32_t EGR;        // TIM event generation register
    volatile uint32_t CCMR1;      // TIM capture/compare mode register 1
    volatile uint32_t CCMR2;      // TIM capture/compare mode register 2
    volatile uint32_t CCER;       // TIM capture/compare enable register
    volatile uint32_t CNT;        // TIM counter
    volatile uint32_t PSC;        // TIM prescaler
    volatile uint32_t ARR;        // TIM auto-reload register
} TIM_TypeDef;
```

1. CR1 — Control Register 1

Logic: Controls *basic timer behavior*.

Controls:

- Counter enable (CEN)
- Up/down counting
- Auto-reload preload enable
- One-pulse mode

Real-world usage:

- Start/stop timer
- Choose upcounter for delays
- Enable preload for stable PWM

→ First register touched after configuring PSC & ARR.

2. CR2 — Control Register 2

Logic: Controls *advanced timer features*.

Controls:

- Master mode (TRGO output)
- Capture/compare control
- Idle state control (advanced timers)

Real-world usage:

- Trigger ADC from timer
- Synchronize multiple timers

→ Mostly used in **timer-triggered peripherals**.

3. SMCR — Slave Mode Control Register

Logic: Allows timer to be controlled by *another signal*.

Modes:

- Reset mode
- Gated mode
- Trigger mode
- Encoder mode

Real-world usage:

- Encoder interface (motors)
- External frequency measurement
- Timer synchronization

→ Enables **hardware timing without CPU**.

4. DIER — DMA / Interrupt Enable Register

Logic: Selects which timer events generate interrupts or DMA.

Real-world usage:

- Enable update interrupt (overflow)
- Enable capture/compare interrupt
- Enable timer-DMA transfer

→ Without DIER, **no interrupt happens**.

5. SR — Status Register

Logic: Shows *which timer event occurred*.

Flags:

- Update event (overflow)
- Capture/compare event
- Trigger event

Real-world usage:

- Check interrupt source
- Clear flags inside ISR

→ Always read & clear inside IRQ.

6. EGR — Event Generation Register

Logic: Forces timer events by software.

Real-world usage:

- Reload prescaler immediately
- Reset counter manually
- Synchronize config changes

→ Very useful during initialization.

7. CCMR1 — Capture/Compare Mode Register 1

8. CCMR2 — Capture/Compare Mode Register 2

Logic: Defines *what each channel does*.

Modes:

- Output Compare (toggle, PWM)
- Input Capture
- PWM mode 1 / 2

Real-world usage:

- PWM generation
- Pulse width measurement
- Signal capture

→ Channel behavior is defined here.

9. CCER — Capture/Compare Enable Register

Logic: Enables channel output and polarity.

Real-world usage:

- Enable PWM output pin
- Select rising/falling edge capture

→ PWM won't appear without CCER.

10. CNT — Counter Register

Logic: Holds current counter value.

Real-world usage:

- Read elapsed time
- Measure pulse width
- Reset counter manually

→ Hardware increments this automatically.

11. PSC — Prescaler Register

Logic: Divides input clock before counting.

Real-world usage:

- Convert MHz clock to kHz/Hz timing
- Create long delays

→ **Timer frequency = $TIM_CLK / (PSC + 1)$**

12. ARR — Auto-Reload Register

Logic: Defines when counter overflows.

Real-world usage:

- Delay length
- PWM period
- Interrupt rate

→ Counter resets when CNT reaches ARR.

Table 114. TIM2 to TIM5 register map and reset values

Offset	Register	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
0x00	TIMx_CR1	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	CKD [1:0]	ARPE	CMS [1:0]	DIR	OPM	URS	UDIS	CEN			
	Reset value																							0	0	0	0	0	0	0	0	0		
0x04	TIMx_CR2	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res		
	Reset value																									0	0	0	0	0	0	0	0	
0x08	TIMx_SMCR	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	ETP	ECE	ETPS [1:0]	ETF [3:0]	Res	Res	Res	MSM	TS [2:0]	Res	SMS [2:0]	Res	Res	Res		
	Reset value																		0	0	0	0	0	0	0	0	0	0	0	0	0	0		
0x0C	TIMx_DIER	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	TDE	COMDE	CC4DE	CC3DE	CC2DE	CC1DE	UDE	Res	TIE	Res	CC4IE	CC3IE	CC2IE	CC1IE	UIE
	Reset value																			0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0x10	TIMx_SR	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	
	Reset value																					0	0	0	0	0	0	0	0	0	0	0	0	
0x14	TIMx_EGR	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	
	Reset value																										0	0	0	0	0	0	0	0
0x18	TIMx_CCMR1 Output Compare mode	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	OC2CE	OC2M [2:0]	OC2PE	OC2FE	CC2S [1:0]	OC1CE	OC1M [2:0]	OC1PE	OC1FE	IC1PSC [1:0]	CC1S [1:0]	Res	Res		
	Reset value																			0	0	0	0	0	0	0	0	0	0	0	0	0	0	
	TIMx_CCMR1 Input Capture mode	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	IC2F [3:0]	IC2PSC [1:0]	CC2S [1:0]	IC1F [3:0]	IC1PSC [1:0]	CC1S [1:0]	Res	Res	Res	Res	Res	Res	Res		
	Reset value																			0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0x1C	TIMx_CCMR2 Output Compare mode	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	OC2CE	OC4M [2:0]	OC4PE	OC4FE	CC4S [1:0]	OC3CE	OC3M [2:0]	OC3PE	OC3FE	CC3S [1:0]	Res	Res	Res		
	Reset value																			0	0	0	0	0	0	0	0	0	0	0	0	0	0	
	TIMx_CCMR2 Input Capture mode	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	IC4F [3:0]	IC4PSC [1:0]	CC4S [1:0]	IC3F [3:0]	IC3PSC [1:0]	CC3S [1:0]	Res	Res	Res	Res	Res	Res	Res		
	Reset value																			0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0x20	TIMx_CCER	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	CC4NP	CC4P	CC4E	CC3NP	CC3P	CC3E	CC2NP	CC2P	CC2E	CC1NP	CC1P	CC1E	Res	Res	
	Reset value																			0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0x24	TIMx_CNT	CNT[31:16] (TIM2 and TIM5 only, reserved on the other timers)																CNT[15:0]																
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

Table 114. TIM2 to TIM5 register map and reset values (continued)

Offset	Register	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x28	TIMx_PSC	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	PSC[15:0]															
	Reset value																	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x2C	TIMx_ARR	ARR[31:16] (TIM2 and TIM5 only, reserved on the other timers)																ARR[15:0]															
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0


```
// Structure definition for I2C registers
typedef struct {
    volatile uint32_t CR1;        // I2C Control register 1
    volatile uint32_t CR2;        // I2C Control register 2
    volatile uint32_t OAR1;       // I2C Own address register 1
    volatile uint32_t OAR2;       // I2C Own address register 2
    volatile uint32_t DR;         // I2C Data register
    volatile uint32_t SR1;        // I2C Status register 1
    volatile uint32_t SR2;        // I2C Status register 2
    volatile uint32_t CCR;        // I2C Clock control register
    volatile uint32_t TRISE;      // I2C TRISE register
} I2C_TypeDef;
```

1. CR1 — Control Register 1

Logic: Master ON/OFF and core behavior.

Controls:

- Peripheral enable (PE)
- START / STOP generation
- ACK enable
- Software reset
- NOSTRETCH, ENGCALL

Real-world usage:

- Enable I²C peripheral
- Generate START and STOP conditions
- Control ACK during receive

→ Most frequently touched control register.

2. CR2 — Control Register 2

Logic: Timing base and interrupt/DMA control.

Controls:

- APB clock frequency (FREQ field)
- Event / buffer / error interrupts
- DMA enable

Real-world usage:

- Must be set correctly for **proper SCL timing**
- Enable I²C interrupts or DMA transfers

→ Wrong CR2 = broken bus timing.

3. OAR1 — Own Address Register 1

Logic: MCU's primary I²C address.

Real-world usage:

- Slave mode address configuration
- 7-bit or 10-bit addressing

→ Not needed in master-only designs.

4. OAR2 — Own Address Register 2

Logic: Secondary I²C address.

Real-world usage:

- Rare multi-address slave devices

→ Almost never used in normal projects.

5. DR — Data Register

Logic: Transmit/receive data byte.

Real-world usage:

- Write → transmit data
- Read → receive data

→ Actual data always flows through DR.

6. SR1 — Status Register 1

Logic: Reports events and errors.

Flags include:

- SB (Start bit sent)
- ADDR (Address sent/matched)
- TXE / RXNE
- BTF (Byte transfer finished)
- Error flags

Real-world usage:

- Drive I²C state machine
- Detect when to send/read next byte

→ Most read register during I²C operation.

7. SR2 — Status Register 2

Logic: Provides bus state information.

Contains:

- Busy flag
- Master/slave mode
- Direction (Tx/Rx)

Real-world usage:

- Must be read after SR1.ADDR to clear ADDR flag
- Check bus busy state

→ SR1 + SR2 reads are often paired.

8. CCR — Clock Control Register

Logic: Defines SCL clock speed.

Real-world usage:

- Configure 100 kHz (Standard mode)
- Configure 400 kHz (Fast mode)

→ Along with TRISE, this defines I²C speed.

9. TRISE — TRISE Register

Logic: Maximum SCL rise time compensation.

Real-world usage:

- Required by I²C spec
- Depends on APB clock frequency

→ Wrong TRISE = unreliable I²C at higher speeds.

Table 147. I²C register map and reset values

Offset	Register	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x00	I2C_CR1	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	SWRST	Res	ALERT	PEC	POS	ACK	STOP	START	NOSTRETCH	ENG	ENPEC	ENARP	SMBTYPE	SMBUS	PF	
	Reset value																	0		0	0	0	0	0	0	0	0	0	0	0	0	0	
0x04	I2C_CR2	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	LAST	DMAEN	ITBUFEN	ITEVTEN	ITERRN	Res	Res	FREQ[5:0]						
	Reset value																			0	0	0	0	0			0	0	0	0	0	0	
0x08	I2C_OAR1	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	ADD[9:8]	ADD[7:1]					ADD0			
	Reset value																	0						0	0	0	0	0	0	0	0	0	
0x0C	I2C_OAR2	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	ADD2[7:1]					ENDUAL			
	Reset value																								0	0	0	0	0	0	0	0	
0x10	I2C_DR	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	DR[7:0]								
	Reset value																																
0x14	I2C_SR1	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	SMBALERT	TIMEOUT	PECERR	OVR	AF	ARLO	BERR	TxE	RxNE	STOPF	ADD10	BTF	ADDR	SR	
	Reset value																	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0x18	I2C_SR2	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	PEC[7:0]					DUALF	SMBHOST	SMBDEFAULT	GENCALL	TRA	BUSY	MSI			
	Reset value																		0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0x1C	I2C_CCR	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	F/S	DUTY	CCR[11:0]												
	Reset value																		0	0		0	0	0	0	0	0	0	0	0	0	0	
0x20	I2C_TRISE	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	TRISE[5:0]							
	Reset value																									0	0	0	0	1			

```
// Structure definition for USART registers
typedef struct {
    volatile uint32_t SR;           // USART Status register
    volatile uint32_t DR;           // USART Data register
    volatile uint32_t BRR;          // USART Baud rate register
    volatile uint32_t CR1;          // USART Control register 1
    volatile uint32_t CR2;          // USART Control register 2
    volatile uint32_t CR3;          // USART Control register 3
    volatile uint32_t GTPR;         // USART Guard time and prescaler register
} USART_TypeDef;
```

1. SR — Status Register

Logic: Tells you *what is happening right now*.

Important flags:

- TXE – Transmit data register empty
- RXNE – Receive data register not empty
- TC – Transmission complete
- ORE / FE / NE – Error flags

Real-world usage:

- Polling TX/RX
- Interrupt-driven UART
- Detect framing or overrun errors

→ Most frequently read register during communication.

2. DR — Data Register

Logic: Holds the byte being transmitted or received.

Real-world usage:

- Write to send a byte
- Read to receive a byte

→ All UART data **must pass through DR**.

3. BRR — Baud Rate Register

Logic: Programs baud rate generator.

Real-world usage:

- Set 9600 / 115200 / custom baud rates
- Depends on APB clock frequency

→ Wrong BRR = garbage data.

4. CR1 — Control Register 1

Logic: Core USART configuration.

Controls:

- USART enable (UE)
- Transmitter enable (TE)
- Receiver enable (RE)
- Word length (8/9 bits)

- Parity control
- RX/TX interrupts

Real-world usage:

- Enable UART
- Choose 8N1, parity, interrupts

→ Mandatory for any USART operation.

5. CR2 — Control Register 2

Logic: Frame format and advanced signaling.

Controls:

- Stop bits (1 / 1.5 / 2)
- Clock enable (synchronous mode)
- LIN support

Real-world usage:

- Configure stop bits
- Rarely used unless synchronous/LIN mode

→ Mostly default in async UART.

6. CR3 — Control Register 3

Logic: Advanced features and hardware assistance.

Controls:

- DMA enable (TX/RX)
- Flow control (RTS/CTS)
- Error interrupt enable

Real-world usage:

- UART + DMA for high-speed logging
- Hardware flow control with modems

→ Used in performance-critical designs.

7. GTPR — Guard Time and Prescaler Register

Logic: Smart-card and IrDA timing support.

Real-world usage:

- ISO-7816 smart cards
- IrDA mode

→ **Almost never used** in normal UART projects.

Table 164. USART register map and reset values

[illegible]

```

/*
 * =====
 *
 * 4. PERIPHERAL INSTANCE POINTERS
 *
 * =====
 */

#define RCC ((RCC_TypeDef*)RCC_BASE) // Pointer to RCC register struct
#define GPIOA ((GPIO_TypeDef*)GPIOA_BASE) // Pointer to GPIOA register struct
#define GPIOB ((GPIO_TypeDef*)GPIOB_BASE) // Pointer to GPIOB register struct
#define GPIOC ((GPIO_TypeDef*)GPIOC_BASE) // Pointer to GPIOC register struct
#define DMA2 ((DMA_TypeDef*)DMA2_BASE) // Pointer to DMA2 register struct
#define DMA2_Stream0 ((DMA_Stream_TypeDef*)(DMA2_BASE + 0x10U)) // Pointer to DMA2 Stream 0 (Offset 0x10)
#define ADC1 ((ADC_TypeDef*)ADC1_BASE) // Pointer to ADC1 register struct
#define TIM2 ((TIM_TypeDef*)TIM2_BASE) // Pointer to TIM2 register struct
#define I2C1 ((I2C_TypeDef*)I2C1_BASE) // Pointer to I2C1 register struct
#define USART1 ((USART_TypeDef*)0x40011000U) // Pointer to USART1 register struct (APB2)
#define USART2 ((USART_TypeDef*)USART2_BASE) // Pointer to USART2 register struct (APB1)
#define USART3 ((USART_TypeDef*)0x40004800U) // Pointer to USART3 register struct (APB1)
#define UART4 ((USART_TypeDef*)0x40004C00U) // Pointer to UART4 register struct (APB1)
#define UART5 ((USART_TypeDef*)0x40005000U) // Pointer to UART5 register struct (APB1)
#define USART6 ((USART_TypeDef*)0x40011400U) // Pointer to USART6 register struct (APB2)

```

This block defines **peripheral instance pointers**—it is the **bridge between C code and real**

hardware registers.

Logically, each `#define` converts a **fixed memory base address (from the reference manual)** into a **typed pointer** so that you can access hardware registers using **struct fields instead of raw addresses**.

```
/*
 * =====
 *                               5. COMMON BIT DEFINITIONS
 * =====
 */
#define ENABLE      1U    // Logical high state / Enable
#define DISABLE     0U    // Logical low state / Disable
#define SET         1U    // Set bit
#define RESET       0U    // Reset bit

// Peripheral Clock Enable Macros
#define ENABLE_GPIOA() (RCC->AHB1ENR |= (1U << 0)) // Enable clock for GPIOA (Bit 0)
#define ENABLE_GPIOB() (RCC->AHB1ENR |= (1U << 1)) // Enable clock for GPIOB (Bit 1)
#define ENABLE_GPIOC() (RCC->AHB1ENR |= (1U << 2)) // Enable clock for GPIOC (Bit 2)
#define ENABLE_DMA2()  (RCC->AHB1ENR |= (1U << 22)) // Enable clock for DMA2 (Bit 22)
#define ENABLE_ADC1()  (RCC->APB2ENR |= (1U << 8))  // Enable clock for ADC1 (Bit 8)
#define ENABLE_TIM2()  (RCC->APB1ENR |= (1U << 0))  // Enable clock for TIM2 (Bit 0)
#define ENABLE_I2C1()  (RCC->APB1ENR |= (1U << 21)) // Enable clock for I2C1 (Bit 21)
#define ENABLE_UART2() (RCC->APB1ENR |= (1U << 17)) // Enable clock for USART2 (Bit 17)
```

Macro	RCC Register	Bus	
ENABLE_GPIOA()	AHB1ENR	AHB1	
ENABLE_GPIOB()	AHB1ENR	AHB1	
ENABLE_GPIOC()	AHB1ENR	AHB1	
ENABLE_DMA2()	AHB1ENR	AHB1	
ENABLE_ADC1()	APB2ENR	APB2	
ENABLE_TIM2()	APB1ENR	APB1	
ENABLE_I2C1()	APB1ENR	APB1	
ENABLE_UART2()	APB1ENR	APB1	

I²C Driver.h Header File Explanation

```

/*
 * =====
 *                               I2C BIT DEFINITIONS
 * =====
 */

// I2C Control Register 1 (CR1)
#define I2C_CR1_PE      (1U << 0)  // Peripheral Enable bit (Bit 0)
#define I2C_CR1_START   (1U << 8)  // Start Generation bit (Bit 8)
#define I2C_CR1_STOP    (1U << 9)  // Stop Generation bit (Bit 9)
#define I2C_CR1_ACK     (1U << 10) // Acknowledge Enable bit (Bit 10)
#define I2C_CR1_SWRST   (1U << 15) // Software Reset bit (Bit 15)

// I2C Status Register 1 (SR1)
#define I2C_SR1_SB      (1U << 0)  // Start Bit flag (Master mode) (Bit 0)
#define I2C_SR1_ADDR    (1U << 1)  // Address sent/matched flag (Bit 1)
#define I2C_SR1_BTF     (1U << 2)  // Byte transfer finished flag (Bit 2)
#define I2C_SR1_ADD10   (1U << 3)  // 10-bit header sent flag (Bit 3)
#define I2C_SR1_STOPF   (1U << 4)  // Stop detection flag (Bit 4)
#define I2C_SR1_RXNE    (1U << 6)  // Data register not empty flag (Bit 6)
#define I2C_SR1_TXE     (1U << 7)  // Data register empty flag (Bit 7)
#define I2C_SR1_BERR    (1U << 8)  // Bus error flag (Bit 8)
#define I2C_SR1_ARLO    (1U << 9)  // Arbitration lost flag (Bit 9)
#define I2C_SR1_AF      (1U << 10) // Acknowledge failure flag (Bit 10)
#define I2C_SR1_OVR     (1U << 11) // Overrun/Underrun flag (Bit 11)
#define I2C_SR1_PECERR   (1U << 12) // PEC Error flag (Bit 12)
#define I2C_SR1_TIMEOUT  (1U << 14) // Timeout or low error flag (Bit 14)
#define I2C_SR1_SMBALERT (1U << 15) // SMBus alert flag (Bit 15)

// I2C Status Register 2 (SR2)
#define I2C_SR2_BUSY    (1U << 1)  // Bus Busy flag (Bit 1)

/*
 * Note: The following existing I2C_FLAG_ macros were in bitmask format (1 << pos)
 * internally in the old header but named like bit POSITIONS in some places.
 * We are standardizing on masks.
 */
#define I2C_FLAG_TXE     I2C_SR1_TXE  // Alias for Tx buffer empty flag
#define I2C_FLAG_RXNE    I2C_SR1_RXNE // Alias for Rx buffer not empty flag
#define I2C_FLAG_SB      I2C_SR1_SB   // Alias for Start Bit flag
#define I2C_FLAG_BTF     I2C_SR1_BTF   // Alias for Byte Transfer Finished flag
#define I2C_FLAG_ADDR    I2C_SR1_ADDR  // Alias for Address matched flag
#define I2C_FLAG_STOPF   I2C_SR1_STOPF // Alias for Stop detected flag

```

24.6.1 I²C control register 1 (I2C_CR1)

Address offset: 0x00

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SW RST	Res.	ALERT	PEC	POS	ACK	STOP	START	NO STRET CH	ENG	ENPEC	ENARP	SMB TYPE	Res.	SM BUS	PE
rw		rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw		rw	rw

I²C Control Register 1 (CR1)

PE – Peripheral Enable

This bit powers the internal I²C logic: clock synchronization, SDA/SCL control, and the protocol state machine. When this bit is cleared, the peripheral is electrically inactive even though registers are accessible. Engineers toggle this bit during initialization or recovery to bring the I²C block into a known state.

START – Start Generation

This bit instructs the hardware to generate a START condition on the bus. Physically, the I²C controller forces SDA low while SCL remains high, claiming ownership of the bus. This bit exists so the CPU does not need to manually manipulate GPIO pins to begin communication.

STOP – Stop Generation

This bit causes the hardware to generate a STOP condition, releasing the bus. It signals the end of a transaction and allows other masters to take control. If this bit is not asserted correctly, the bus remains occupied and communication with other devices can stall.

ACK – Acknowledge Enable

This bit controls whether the device actively acknowledges received bytes. At the electrical level, it determines whether the device pulls SDA low during the acknowledge clock pulse. Engineers rely on this bit to properly terminate read operations and to comply with the I²C protocol's flow-control rules.

SWRST – Software Reset

This bit resets the internal I²C state machine without resetting the entire MCU. It clears protocol errors, releases stuck bus states, and reinitializes internal logic. This exists because real I²C buses can lock up due to noise, misbehaving slaves, or interrupted transactions.

24.6.6 I²C status register 1 (I2C_SR1)

Address offset: 0x14

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SMB ALERT	TIMEO UT	Res.	PEC ERR	OVR	AF	ARLO	BERR	TxE	RxNE	Res.	STOPF	ADD10	BTF	ADDR	SB
rc_w0	rc_w0		rc_w0	rc_w0	rc_w0	rc_w0	rc_w0	r	r		r	r	r	r	r

I²C Status Register 1 (SR1)

SB – Start Bit Flag

This flag confirms that the START condition has been successfully generated and recognized by the hardware. It tells the CPU that the bus has transitioned from idle to active master mode and that the address phase can proceed.

ADDR – Address Sent / Matched

This flag indicates that the address phase completed successfully. On the bus, it means a slave acknowledged the address or the device recognized its own address. Internally, this flag locks certain bus operations until it is cleared to prevent protocol violations.

BTF – Byte Transfer Finished

This flag indicates that a data byte has been fully transferred and both internal buffers are free. It represents a clean synchronization point where the hardware guarantees that no partial byte remains in transit.

ADD10 – 10-bit Address Header Sent

This flag exists only for 10-bit addressing mode. It confirms transmission of the extended address header. Most systems never encounter this flag because 7-bit addressing dominates modern designs.

STOPF – Stop Detection

This flag indicates that a STOP condition was detected on the bus while operating as a slave. It allows the device to detect when a master ends communication and to reset internal reception logic accordingly.

RXNE – Receive Buffer Not Empty

This flag tells the CPU that valid data has been received and is waiting in the data register. It prevents data loss by signaling that the hardware buffer must be serviced before another byte arrives.

TXE – Transmit Buffer Empty

This flag indicates that the transmit buffer is ready for the next byte. It exists to prevent overwriting data that has not yet been sent on the bus.

BERR – Bus Error

This flag signals illegal bus conditions such as malformed START or STOP sequences. It usually indicates electrical noise, incorrect timing configuration, or severe protocol violations on the bus.

ARLO – Arbitration Lost

This flag indicates that another master won control of the bus. The hardware detected that SDA did not match the expected level during arbitration. This exists to support safe multi-master systems without bus corruption.

AF – Acknowledge Failure

This flag means that the addressed device did not acknowledge a byte. In practice, this occurs when a slave is absent, busy, or when the master intentionally terminates a read operation.

OVR – Overrun / Underrun

This flag indicates that data was lost because the CPU or DMA did not service the data register in time. It reflects a performance mismatch between bus speed and software response.

PECERR – PEC Error

This flag reports a Packet Error Code mismatch in SMBus mode. It is used mainly in industrial or battery-management systems that require extra data integrity checks.

TIMEOUT – Timeout or Tlow Error

This flag indicates that clock stretching exceeded allowed limits. It protects the system from slaves that hold SCL low indefinitely and freeze the bus.

SMBALERT – SMBus Alert

This flag indicates an SMBus-specific alert condition. It is used in systems that support standardized alert signaling, such as smart batteries or power controllers.

24.6.7 I²C status register 2 (I2C_SR2)

Address offset: 0x18

Reset value: 0x0000

Note: Reading I2C_SR2 after reading I2C_SR1 clears the ADDR flag, even if the ADDR flag was set after reading I2C_SR1. Consequently, I2C_SR2 must be read only when ADDR is found set in I2C_SR1 or when the STOPF bit is cleared.

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PEC[7:0]								DUALF	SMB HOST	SMB DEFAULT	GEN CALL	Res.	TRA	BUSY	MSL
r	r	r	r	r	r	r	r	r	r	r	r		r	r	r

I²C Status Register 2 (SR2)

BUSY – Bus Busy

This flag reflects the physical state of the bus. It indicates whether SDA or SCL is currently engaged in an active transaction. Engineers use this to avoid collisions and to detect stuck bus conditions.

```
typedef struct
{
    uint32_t I2C_SCLSpeed;           // I2C Clock speed (e.g., 100000)
    uint8_t  I2C_DeviceAddress;      // I2C Device address (7-bit)
    uint8_t  I2C_AckControl;          // Enable/Disable Acknowledgement
    uint8_t  I2C_FMDutyCycle;         // Fast Mode Duty Cycle selection
}I2C_Config_t;
```

```

@I2C_SCLSpeed
/
#define I2C_SCL_SPEED_SM      100000U      // Standard Mode Speed (100kHz)
#define I2C_SCL_SPEED_FM4K    400000U      // Fast Mode Speed (400kHz)
#define I2C_SCL_SPEED_FM2K    200000U      // Fast Mode Speed (200kHz)

@I2C_AckControl
/
#define I2C_ACK_ENABLE        1U           // Acknowledgement Enabled
#define I2C_ACK_DISABLE       0U           // Acknowledgement Disabled

@I2C_FMDutyCycle
/
#define I2C_FM_DUTY_2         0U           // Duty cycle 2
#define I2C_FM_DUTY_16_9      1U           // Duty cycle 16/9

Application states
/
#define I2C_READY              0U          // I2C Peripheral Ready state
#define I2C_BUSY_IN_RX         1U          // I2C Peripheral Busy in Reception
#define I2C_BUSY_IN_TX         2U          // I2C Peripheral Busy in Transmission

```

I2C_SCLSpeed

This field represents the **desired clock frequency of the I²C bus**.

It expresses *how fast data should move* across SDA/SCL, not how the MCU internally generates it. Engineers choose this value based on **sensor capability, bus length, and noise margin**. Lower speeds improve reliability on long or noisy lines, while higher speeds reduce latency in data-heavy systems.

I2C_DeviceAddress

This field stores the **7-bit address that identifies the device on the I²C bus**.

It defines *who the MCU is* when acting as a slave, or *who it talks to* when acting as a master (depending on driver design). This abstraction avoids mixing protocol-level identity with timing or control logic, which is critical in multi-device systems.

I2C_AckControl

This field describes whether the device **actively acknowledges received bytes**.

Acknowledgment is not optional in I²C—it is a **flow-control signal** that tells the sender whether communication should continue. Exposing this as a configuration option allows the driver to handle receive termination correctly and prevents subtle protocol violations.

I2C_FMDutyCycle

This field applies only when **Fast Mode** is selected.

It controls the **high-time vs low-time ratio of the SCL signal**, which affects signal integrity and

timing compliance. Engineers adjust this to meet I²C electrical specifications when running at higher speeds, especially on buses with higher capacitance.

I²C SCL Speed definitions

These values represent **target bus frequencies**, not raw hardware settings.

They describe the *contract* between the MCU and all devices on the bus: how fast bits are allowed to move.

Standard Mode – 100 kHz

This speed prioritizes **signal integrity and compatibility**. It tolerates longer wires, higher capacitance, and electrically noisy environments. Engineers choose this when reliability matters more than throughput or when dealing with older or low-cost sensors.

Fast Mode – 400 kHz

This speed increases throughput by shortening clock periods. It requires cleaner electrical conditions and proper timing configuration. Used when systems need faster sensor reads or reduced communication latency, such as display updates or high-rate data acquisition.

Fast Mode – 200 kHz

This is a **practical compromise**. It offers better performance than standard mode while remaining tolerant of imperfect wiring. Engineers often use this in real hardware where 400 kHz is unstable but 100 kHz is too slow.

Acknowledge control definitions

Acknowledgment is the **flow-control mechanism** of I²C.

These definitions control whether the device actively participates in that handshake.

ACK Enabled

The device confirms receipt of each byte by pulling SDA low during the acknowledge clock. This signals that communication should continue. Without ACK, the sender interprets silence as a stop condition.

ACK Disabled

The device intentionally refrains from acknowledging a byte. This is not an error; it is a **protocol-level signal** used to terminate reads or indicate that no more data is desired.

Fast Mode duty cycle definitions

In Fast Mode, the I²C specification allows different **high/low time ratios** for the SCL clock. These definitions control the electrical shape of the clock.

Duty Cycle 2

This produces a symmetric or near-symmetric clock. It is simpler, more forgiving, and works well on most boards. Engineers prefer this unless there is a strong reason to optimize timing margins.

Duty Cycle 16/9

This produces a longer high time relative to low time. It helps meet timing requirements at higher speeds or with heavier bus capacitance. Used when pushing Fast Mode limits or when required by specific devices.

Application state definitions

These values represent the **logical state of the I²C peripheral from the software's point of view**. They do not correspond directly to hardware bits; instead, they describe **driver-level intent and status**.

READY

The peripheral is idle, configured, and not participating in any transaction. This state indicates that the bus and driver are available for a new operation.

BUSY IN RX

The peripheral is actively receiving data. This state prevents new transactions from starting and ensures that receive buffers and acknowledgment behavior are handled correctly.

BUSY IN TX

The peripheral is actively transmitting data. This state ensures that transmit timing, buffering, and stop conditions are managed without interference.

```
*****
*                               APIs supported by this driver
*                               *****/
/*
* Peripheral Clock setup
*/
void I2C_PeriClockControl(I2C_RegDef_t *pI2Cx, uint8_t EnorDi); // Enable/Disable I2C Clock

/*
* Init and De-init
*/
void I2C1_Init(void); // Keeping specific init for this project
void I2C_Init(I2C_Handle_t *pI2CHandle); // Generic Init
void I2C_DeInit(I2C_RegDef_t *pI2Cx); // De-Init

/*
* Data Send and Receive
*/
void I2C1_WriteMulti(uint8_t addr, uint8_t reg, uint8_t* d, uint16_t c); // Write generic data
void I2C1_Write(uint8_t addr, uint8_t reg, uint8_t data); // Write single byte

// Generic APIs (Cleaned up prototypes)
void I2C_MasterSendData(I2C_Handle_t *pI2CHandle, uint8_t *pTxbuffer, uint32_t Len, uint8_t SlaveAddr, uint8_t Sr); // Send data as Master
void I2C_MasterReceiveData(I2C_Handle_t *pI2CHandle, uint8_t *pRxBuffer, uint8_t Len, uint8_t SlaveAddr, uint8_t Sr); // Receive data as Master

uint8_t I2C_GetFlagStatus(I2C_RegDef_t *pI2Cx, uint32_t FlagName); // Check flag status
void I2C_ApplicationEventCallback(I2C_Handle_t *pI2CHandle, uint8_t AppEv); // Callback for application events
```

Peripheral clock control API

I2C_PeriClockControl

This API controls whether the I²C peripheral is **electrically powered by the clock tree**. It exists because, in STM32, peripherals physically do nothing unless their clock is enabled.

Engineers think of this API as the **power switch** for the I²C block. No configuration or data transfer has meaning unless this step is handled correctly.

Initialization and de-initialization APIs

I2C1_Init

This is a **project-specific initialization entry point**.

It reflects a conscious design choice: sometimes a project needs a fixed, known I²C configuration that should not be altered dynamically.

Engineers use this when:

- Hardware design is fixed
 - Timing and addressing are constant
 - Simplicity and predictability are preferred
-

I2C_Init

This is the **generic initializer**.

It converts a configuration description (speed, address, ACK behavior, duty cycle) into a correctly configured I²C peripheral.

This API exists to:

- Support multiple I²C instances
 - Allow reuse across projects
 - Decouple configuration intent from hardware details
-

I2C_DeInit

This API returns the peripheral to a **clean, reset-like state**.

It is used to:

- Recover from bus lockups
- Reinitialize after errors
- Reduce power consumption

Engineers view this as a **controlled shutdown**, not just cleanup.

Data send and receive APIs (application-focused)

I2C1_WriteMulti

This API represents **register-oriented communication**, which is the most common I²C usage pattern in the real world.

It exists because most I²C devices:

- Expose internal registers
- Expect a register address followed by data
- Support multi-byte writes

This API hides protocol repetition and improves readability at the application level.

I2C1_Write

This API is a **specialized convenience shortcut** for writing a single byte to a device register.

Engineers include this because:

- Single-register writes are extremely common
 - It reduces boilerplate
 - It improves intent clarity in application code
-

Generic master communication APIs

I2C_MasterSendData

This API represents **raw master transmission capability**.

It is protocol-agnostic and does not assume register-based devices.

Engineers use this when:

- Communicating with non-register devices
- Implementing custom protocols
- Building higher-level abstractions on top

It is the **core transmit engine** of the driver.

I2C_MasterReceiveData

This API is the receive counterpart to master send.

It represents controlled acquisition of data from a slave device while respecting I²C flow-control rules.

This API exists because receive operations are **not symmetric** with transmit operations in I²C and require careful handling of acknowledgment and termination.

Flag and state inspection API

I2C_GetFlagStatus

This API exposes **hardware state visibility** to the driver or application layer.

It exists so software can:

- Observe progress
- Detect errors
- Make decisions based on peripheral state

Engineers treat this as a **window into the I²C state machine**, not as control logic.

Application event callback

I2C_ApplicationEventCallback

This API is the **bridge between driver and application logic**.

It allows the driver to report meaningful events without knowing how the application will respond.

Typical events include:

- Transmission complete
- Reception complete
- Error conditions

This design enables:

- Interrupt-driven systems
- Event-based architectures

Usart driver.h header file Explanation

```

* =====
*                                     USART BIT DEFINITIONS
* =====
*/

// USART Control Register 1 (CR1)
#define USART_CR1_RE                (1U << 2)    // Receiver Enable (Bit 2)
#define USART_CR1_TE                (1U << 3)    // Transmitter Enable (Bit 3)
#define USART_CR1_PS                (1U << 9)    // Parity Selection (Bit 9)
#define USART_CR1_PCE                (1U << 10)   // Parity Control Enable (Bit 10)
#define USART_CR1_M                (1U << 12)    // Word Length (Bit 12)
#define USART_CR1_UE                (1U << 13)    // USART Enable (Bit 13)
#define USART_CR1_OVER8              (1U << 15)   // Oversampling Mode (Bit 15)

// USART Control Register 2 (CR2)
#define USART_CR2_STOP_1             (0x0U << 12) // 1 Stop bit (Bits 12-13 -> 00)
#define USART_CR2_STOP_0_5          (0x1U << 12) // 0.5 Stop bit (Bits 12-13 -> 01)
#define USART_CR2_STOP_2            (0x2U << 12) // 2 Stop bits (Bits 12-13 -> 10)
#define USART_CR2_STOP_1_5          (0x3U << 12) // 1.5 Stop bits (Bits 12-13 -> 11)

// USART Control Register 3 (CR3)
#define USART_CR3_RTSE               (1U << 8)    // RTS Enable (Bit 8)
#define USART_CR3_CTSE               (1U << 9)    // CTS Enable (Bit 9)

// USART Status Register (SR)
#define USART_SR_RXNE                (1U << 5)    // Read Data Register Not Empty (Bit 5)
#define USART_SR_TC                  (1U << 6)    // Transmission Complete (Bit 6)
#define USART_SR_TXE                 (1U << 7)    // Transmit Data Register Empty (Bit 7)

// Flag Macros
#define USART_FLAG_TXE               USART_SR_TXE // Transmit Data Register Empty Flag
#define USART_FLAG_RXNE              USART_SR_RXNE // Receive Data Register Not Empty Flag
#define USART_FLAG_TC                USART_SR_TC   // Transmission Complete Flag

```

25.6.4 Control register 1 (USART_CR1)

Address offset: 0x0C

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OVER8	Res.	UE	M	WAKE	PCE	PS	PEIE	TXEIE	TCIE	RXNEIE	IDLEIE	TE	RE	RWU	SBK
rw		rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

25.6.5 Control register 2 (USART_CR2)

Address offset: 0x10

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	LINEN	STOP[1:0]		CLKEN	CPOL	CPHA	LBCL	Res.	LBDIE	LBDL	Res.	ADD[3:0]			
	rw	rw	rw	rw	rw	rw	rw		rw	rw		rw	rw	rw	rw

25.6.6 Control register 3 (USART_CR3)

Address offset: 0x14

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res	Res	Res	Res	ONEBIT	CTSIE	CTSE	RTSE	DMAT	DMAR	SCEN	NACK	HDSEL	IRLP	IREN	EIE
				rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

25.6.1 Status register (USART_SR)

Address offset: 0x00

Reset value: 0x00C0 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res	Res	Res	Res	Res	Res	CTS	LBD	TXE	TC	RXNE	IDLE	ORE	NF	FE	PE
						rc_w0	rc_w0	r	rc_w0	rc_w0	r	r	r	r	r

USART Control Register 1 (CR1)

RE – Receiver Enable

This bit activates the receive path of the USART.

Internally, it powers the sampling logic, shift register, and receive buffer. If this bit is cleared, incoming bits on the RX pin are ignored regardless of baud rate or framing.

Engineers use this to:

- Disable reception when not needed
- Prevent buffer overruns
- Save power in low-activity systems

TE – Transmitter Enable

This bit activates the transmit path.

It allows data written by software to be serialized and driven onto the TX pin. Without this bit, the USART cannot physically transmit even if data is written.

This separation between TX and RX enables **half-duplex**, diagnostics, and controlled communication direction.

PS – Parity Selection

This bit selects whether parity is **even or odd** when parity is enabled.

Parity itself is a **single-bit error-detection mechanism**, not correction. It exists to detect line noise, framing errors, or clock mismatch.

Engineers care about this when:

- Communicating with legacy devices
- Working in noisy industrial environments
- Matching strict protocol requirements

PCE – Parity Control Enable

This bit determines whether a parity bit is included in the frame at all.

When enabled, parity becomes part of the transmitted and received frame; when disabled, frames contain only data and stop bits.

This bit directly affects **frame length and interpretation** on both ends of the link.

M – Word Length

This bit selects the number of data bits in a frame.

Changing word length alters how incoming bit streams are grouped into data values. It exists to support different protocols and legacy equipment.

Engineers treat this bit as a **contract**: both sides must agree on frame size or communication becomes meaningless.

UE – USART Enable

This is the master enable for the entire USART peripheral.

When cleared, the USART clock domain and state machine are disabled even if other bits are set.

When set, the USART becomes active and responsive.

This bit exists to allow:

- Clean startup sequencing
 - Safe reconfiguration
 - Power control
-

OVER8 – Oversampling Mode

This bit controls how many samples the hardware takes per bit period.

Oversampling improves noise immunity and timing tolerance at the cost of tighter baud-rate constraints.

Engineers choose this based on:

- Clock accuracy
 - Noise environment
 - Required baud rates
-

USART Control Register 2 (CR2) – Stop bit configuration

Stop bits define **how a frame ends** and give the receiver time to resynchronize.

1 Stop Bit

The most common and efficient choice.
Provides minimal idle time while maintaining reliability.

0.5 Stop Bit

Used in specific synchronous or legacy protocols.
Rare in modern async communication.

2 Stop Bits

Adds extra idle time between frames.
Used when:

- Clock mismatch is likely
 - Long cable lengths exist
 - Devices need more recovery time
-

1.5 Stop Bits

Mainly for compatibility with older UART standards.
Rarely used in modern embedded systems.

USART Control Register 3 (CR3)

RTSE – RTS Enable

This enables **hardware flow control output**.
The USART automatically asserts or deasserts RTS to tell the other device when it is ready to receive data.

Engineers rely on this to prevent buffer overruns without software intervention.

CTSE – CTS Enable

This enables **hardware flow control input**.
Transmission is automatically paused when the other device indicates it is not ready.
This bit offloads timing-critical flow control from software to hardware.

USART Status Register (SR)

These flags reflect the **live state of the USART data pipeline**.

RXNE – Receive Data Register Not Empty

This flag indicates that a complete data frame has been received and is waiting to be read. It exists to prevent data loss and to synchronize software with incoming data.

From an electrical perspective, this means:

- Start bit detected
 - Data bits sampled
 - Stop bit validated
-

TC – Transmission Complete

This flag indicates that:

- All data bits
- Parity (if any)
- Stop bits
have fully left the TX pin.

Engineers use this when it is important to know that the line is **electrically idle**, not just that the buffer is empty.

TXE – Transmit Data Register Empty

This flag indicates that the transmit buffer can accept new data.

It does **not** mean the previous byte finished transmitting—only that it has moved to the shift register.

This distinction is crucial for high-throughput systems.

```

/*
 * Configuration structure for USARTx peripheral
 */
typedef struct
{
    uint8_t USART_Mode;           // Mode: Rx, Tx, or RxTx
    uint32_t USART_Baud;          // Communication Baud Rate
    uint8_t USART_NoOfStopBits;   // Number of Stop Bits
    uint8_t USART_WordLength;     // Word Length (8 or 9 bits)
    uint8_t USART_ParityControl;  // Parity: None, Odd, Even
    uint8_t USART_HWFlowControl;  // Hardware Flow Control: CTS/RTS
}USART_Config_t;

```

```

/*
 * @USART_Mode
 */
#define USART_MODE_ONLY_TX 0U    // Transmitter only mode
#define USART_MODE_ONLY_RX 1U    // Receiver only mode
#define USART_MODE_TXRX 2U      // Transmitter and Receiver mode

/*
 * @USART_Baud
 */
#define USART_STD_BAUD_1200 1200U // 1200 bns
#define USART_STD_BAUD_2400 2400U // 2400 bns
#define USART_STD_BAUD_9600 9600U // 9600 bns
#define USART_STD_BAUD_19200 19200U // 19200 bns
#define USART_STD_BAUD_38400 38400U // 38400 bns
#define USART_STD_BAUD_57600 57600U // 57600 bns
#define USART_STD_BAUD_115200 115200U // 115200 bns
#define USART_STD_BAUD_230400 230400U // 230400 bns
#define USART_STD_BAUD_460800 460800U // 460800 bns
#define USART_STD_BAUD_921600 921600U // 921600 bns
#define USART_STD_BAUD_2M 2000000U // 2 Mbps
#define USART_STD_BAUD_3M 3000000U // 3 Mbps

/*
 * @USART_ParityControl
 */
#define USART_PARITY_EN_ODD 2U    // Odd Parity Enable
#define USART_PARITY_EN_EVEN 1U  // Even Parity Enable
#define USART_PARITY_DISABLE 0U  // Parity Disable

```



```

/*
 * @USART_WordLength
 */
#define USART_WORDLEN_8BITS 0U // 8-bit word length
#define USART_WORDLEN_9BITS 1U // 9-bit word length

/*
 * @USART_NoOfStopBits
 */
#define USART_STOPBITS_1 0U // 1 Stop Bit
#define USART_STOPBITS_0_5 1U // 0.5 Stop Bit
#define USART_STOPBITS_2 2U // 2 Stop Bits
#define USART_STOPBITS_1_5 3U // 1.5 Stop Bits

/*
 * @USART_HWFlowControl
 */
#define USART_HW_FLOW_CTRL_NONE 0U // No Flow Control
#define USART_HW_FLOW_CTRL_CTS 1U // CTS only
#define USART_HW_FLOW_CTRL_RTS 2U // RTS only
#define USART_HW_FLOW_CTRL_CTS_RTS 3U // CTS and RTS

/*
 * Application states
 */
#define USART_BUSY_IN_RX 1U // Rx Busy state
#define USART_BUSY_IN_TX 2U // Tx Busy state
#define USART_READY 0U // Ready state

```

USART_Mode

This field defines **directionality of communication**.

- **Transmit only** means the device is a talker, not a listener
- **Receive only** means the device is a listener, not a talker
- **Transmit + Receive** enables full bidirectional communication

Engineers care about this because:

- It affects power consumption
- It prevents unintended line driving
- It avoids false reception in noisy environments

This is not about convenience—it's about **electrical responsibility** on the TX/RX pins.

USART_Baud

This field defines **how fast bits are transmitted on the wire**.

Baud rate is a **timing agreement** between two devices.

If both sides don't agree closely enough, framing collapses and data becomes garbage.

Engineers select baud rates based on:

- Clock accuracy
- Cable length
- Noise margin
- Throughput requirements

Higher baud $\neq$ better by default.

It's always a trade-off between **speed and reliability**.

USART_NoOfStopBits

Stop bits define **how long the line stays idle between characters**.

They exist to:

- Allow receiver resynchronization
- Tolerate clock mismatch
- Improve reliability on slow or noisy links

Engineers increase stop bits when:

- Using long cables
- Communicating with legacy equipment
- Facing timing uncertainty

This field directly affects **frame spacing**, not data content.

USART_WordLength

This field defines **how many data bits form one character**.

Changing word length changes:

- How bits are grouped
- How parity is applied
- How received data must be interpreted

Engineers treat this as a **protocol contract**.

If word length doesn't match on both ends, communication technically occurs—but meaning is lost.

USART_ParityControl

Parity adds **single-bit error detection** to each frame.

- Disabled → maximum throughput, minimal overhead
- Enabled → improved error detection at the cost of one bit

Engineers enable parity when:

- Operating in noisy environments
- Communicating with industrial or safety devices
- Needing basic corruption detection without CRC

Parity does not correct errors—it only **signals suspicion**.

USART_HWFlowControl

This field determines whether **hardware handshaking** is used.

Flow control exists to prevent **buffer overruns**, not protocol errors.

- **None** → software must keep up
- **CTS** → remote device controls when we transmit
- **RTS** → we control when remote device transmits
- **CTS + RTS** → fully hardware-managed flow

Engineers enable hardware flow control when:

- Data rates are high
- CPU load is unpredictable
- Data loss is unacceptable

This is about **physical pacing**, not data format.

```
/*
 * Handle structure for USARTx peripheral
 */
typedef struct
{
    USART_RegDef_t *pUSARTx; // Pointer to USART peripheral base address
    USART_Config_t  USART_Config; // USART Configuration structure
    uint8_t *pTxBuffer; // Pointer to Tx buffer
    uint8_t *pRxBuffer; // Pointer to Rx buffer
    uint32_t TxLen; // Length of Tx data
    uint32_t RxLen; // Length of Rx data
    uint8_t TxBusyState; // Tx Busy state flag
    uint8_t RxBusyState; // Rx Busy state flag
}USART_Handle_t;
```

Pointer to USART peripheral (pUSARTx)

This field anchors the handle to **one specific hardware block**.

It answers the question: *Which USART does this handle control?*

Engineers rely on this so:

- One driver can manage multiple USART instances
- The same logic works for USART1, USART2, etc.
- The handle cleanly maps software state to physical silicon

This pointer is the **bridge between software state and hardware reality**.

USART_Config (configuration snapshot)

This field stores the **chosen communication policy** for this USART instance.

It represents:

- Baud rate expectations
- Framing rules
- Direction and flow control decisions

Engineers include this inside the handle so:

- Configuration is always associated with the instance
- Reinitialization or recovery can reuse known settings
- Debugging becomes easier (state + config together)

This is **design memory**, not live state.

Transmit buffer pointer (pTxBuffer)

This field points to the **current data being transmitted**.

Conceptually, it answers:

- *What data is the USART trying to send right now?*

Engineers use this to:

- Track progress in interrupt-driven transmission
- Resume sending after each hardware event
- Decouple application buffers from hardware timing

This pointer exists because **transmission is incremental**, not instantaneous.

Receive buffer pointer (pRxBuffer)

This field points to the **destination buffer for incoming data**.

It represents:

- Where received bytes should be stored
- The ownership of incoming data during an active receive operation

Engineers include this so:

- Reception can proceed asynchronously
- Partial data can be accumulated safely

- Application logic does not need to poll continuously

This is essential for **non-blocking and interrupt-based reception**.

Transmit length (TxLen)

This field represents **how much data remains to be transmitted**.

It is not a configuration—it is **progress tracking**.

Engineers use this to:

- Know when transmission is complete
- Decide when to stop interrupts
- Trigger completion events correctly

Without this, the driver would not know **when it is done**.

Receive length (RxLen)

This field represents **how much data is still expected**.

It allows the driver to:

- Know when a receive operation has finished
- Manage acknowledgment or termination behavior
- Signal completion accurately

This field exists because receive operations must be **bounded and intentional**.

Transmit busy state (TxBusyState)

This field represents whether the driver is **currently engaged in transmission**.

It enforces:

- Mutual exclusion
- Correct sequencing
- Protection against overlapping operations

Engineers use this to prevent:

- Corrupted data
- Race conditions
- Invalid API usage

This is **software discipline**, not hardware state.

Receive busy state (RxBusyState)

This field represents whether the driver is **currently engaged in reception**.

It ensures:

- Only one receive operation is active
- Buffer ownership is clear
- Interrupts are interpreted correctly

This is critical in systems where **data may arrive at any time**.

```

/*****
 *
 *                               APIs supported by this driver
 *
 *****/

// Peripheral Clock Control
void USART_PeriClockControl(USART_RegDef_t *pUSARTx, uint8_t EnOrDi);

// Init and De-Init
void USART_Init(USART_Handle_t *pUSARHandle);
void USART_DeInit(USART_Handle_t *pUSARHandle);

// Data Transmission and Reception (Blocking)
void USART_SendData(USART_Handle_t *pUSARHandle, uint8_t *pTxBuffer, uint32_t Len);
void USART_ReceiveData(USART_Handle_t *pUSARHandle, uint8_t *pRxBuffer, uint32_t Len);

// Interrupt Data Transmission and Reception
uint8_t USART_SendDataIT(USART_Handle_t *pUSARHandle, uint8_t *pTxBuffer, uint32_t Len);
uint8_t USART_ReceiveDataIT(USART_Handle_t *pUSARHandle, uint8_t *pRxBuffer, uint32_t Len);

// Interrupt Configuration
void USART_IRQInterruptConfig(uint8_t IRQNumber, uint8_t EnOrDi);
void USART_IRQPriorityConfig(uint8_t IRQNumber, uint32_t IRQPriority);
void USART_IRQHandling(USART_Handle_t *pUSARHandle);

// Control and Status
uint8_t USART_GetFlagStatus(USART_RegDef_t *pUSARTx, uint8_t StatusFlagName);
void USART_ClearFlag(USART_RegDef_t *pUSARTx, uint16_t StatusFlagName);
void USART_PeripheralControl(USART_RegDef_t *pUSARTx, uint8_t EnOrDi);
void USART_SetBaudRate(USART_RegDef_t *pUSARTx, uint32_t BaudRate);
void USART_ApplicationEventCallback(USART_Handle_t *pUSARHandle, uint8_t ApEv);

// --- Legacy Support (Application Specific Wrapper) ---
void UART2_Init(void);           // Initialize UART2 specifically
void UART2_SendChar(char c);    // Send single character via UART2
void UART2_SendString(char *string); // Send string via UART2
void UART2_SendNumber(int number); // Send integer as text via UART2
char UART2_GetChar(void);        // Receive char via UART2

```

Peripheral clock control

USART_PeriClockControl

This API controls whether the USART peripheral is **electrically alive**.

Without a clock, the USART cannot sample RX, shift TX bits, or generate baud timing.

Engineers think of this as:

- Power gating

- Safe startup sequencing
- Mandatory precondition for all other APIs

This function exists to make **hardware dependency explicit**, not implicit.

Initialization and de-initialization

USART_Init

This API translates **communication intent** (mode, baud, framing, parity, flow control) into a working USART configuration.

Engineers expect this API to:

- Fully prepare the peripheral
- Leave it in a predictable, usable state
- Be reusable across different USART instances

This is the **official entry point** into using the USART.

USART_DeInit

This API returns the USART to a **clean, inactive state**.

It exists for:

- Error recovery
- Reinitialization
- Power management
- Graceful shutdown

Professionally written drivers always include a way to **undo initialization**.

Blocking data transmission and reception

USART_SendData

This API represents **synchronous transmission**.

From an engineer's perspective:

- The function does not return until data is fully handed off
- CPU execution is tied to communication progress
- Simpler logic, predictable flow

Used when:

- Data volume is small
- Timing is not critical

- Simplicity matters more than throughput
-

USART_ReceiveData

This API represents **synchronous reception**.

It ensures:

- Data is fully received before returning
- No background state remains unfinished

Engineers use this when:

- Waiting for command input
 - Receiving fixed-size responses
 - Debug or console interfaces
-

Interrupt-based data transmission and reception

USART_SendDataIT

This API represents **asynchronous transmission**.

Conceptually:

- Data transfer happens in the background
- CPU is free to do other work
- Completion is reported later via events

Engineers choose this when:

- Throughput matters
 - Latency must be minimized
 - The system is multitasking or real-time
-

USART_ReceiveDataIT

This API represents **asynchronous reception**.

It exists because reception is timing-sensitive and unpredictable.

The USART must be able to accept data **immediately**, even if the CPU is busy.

This API is foundational for:

- Event-driven systems
 - Interrupt-based drivers
 - DMA-based extensions later
-

Interrupt configuration

USART_IRQInterruptConfig

This API controls whether the USART can **signal the CPU** via interrupts.

Engineers use it to:

- Enable or suppress interrupt-driven behavior
- Integrate USART into system-level interrupt design

This keeps **NVIC policy separate from peripheral logic**.

USART_IRQPriorityConfig

This API defines **how important USART events are** relative to others.

Engineers care deeply about this in systems with:

- Multiple interrupts
- Real-time constraints
- Mixed criticality tasks

This API is about **system behavior**, not USART behavior.

USART_IRQHandling

This API is the **bridge between hardware events and driver logic**.

It exists to:

- Decode why the interrupt occurred
- Advance transmission or reception state
- Detect and report errors

Engineers see this as the **heart of the interrupt-driven driver**.

Control and status APIs

USART_GetFlagStatus

This API exposes **live USART state** in a readable way.

It allows software to:

- Observe progress
- Make decisions
- Detect readiness or completion

This API exists to avoid direct register peeking throughout the application.

USART_ClearFlag

This API acknowledges that some USART states **must be explicitly cleared**.

It enforces:

- Proper state transitions
- Predictable behavior
- Clean interrupt handling

Engineers view this as **state hygiene**.

USART_SetBaudRate

This API allows baud rate adjustment **without full reinitialization**.

It exists for:

- Dynamic configuration
- Protocol negotiation
- Debug interfaces that change speed

This shows foresight in driver design.

USART_ApplicationEventCallback

This API decouples the driver from application logic.

The driver reports *what happened*

The application decides *what to do*

This enables:

- Clean event-driven design
- Reusability
- Testability

Professionals expect this pattern.

Legacy / application-specific support

UART2_Init

This is a **project-level convenience entry point**.

It exists because:

- Some projects always use the same USART
- Boilerplate should be hidden

- Simplicity sometimes beats generality
-

UART2_SendChar / SendString / SendNumber / GetChar

These APIs represent **human-facing communication helpers**.

They exist for:

- Debug logging
- CLI interfaces
- Early bring-up
- Teaching and testing

Engineers know these are **wrappers**, not core driver functionality.

Adc dma driver.h header file Explanations

```
/*
 * =====
 *                               ADC/DMA CONFIGURATION MACROS
 * =====
 */

// GPIO Mode Definitions (MODER)
#define GPIO_MODE_INPUT      0x00U    // Input mode configuration value
#define GPIO_MODE_OUTPUT    0x01U    // Output mode configuration value
#define GPIO_MODE_ALTFN     0x02U    // Alternate function mode configuration value
#define GPIO_MODE_ANALOG    0x03U    // Analog mode configuration value
```

7.4.1 GPIO port mode register (GPIOx_MODER) (x = A..H)

Address offset: 0x00

Reset values:

- 0xA800 0000 for port A
- 0x0000 0280 for port B
- 0x0000 0000 for other ports

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
MODER15[1:0]		MODER14[1:0]		MODER13[1:0]		MODER12[1:0]		MODER11[1:0]		MODER10[1:0]		MODER9[1:0]		MODER8[1:0]	
r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MODER7[1:0]		MODER6[1:0]		MODER5[1:0]		MODER4[1:0]		MODER3[1:0]		MODER2[1:0]		MODER1[1:0]		MODER0[1:0]	
r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w

Bits 2y:2y+1 **MODERy[1:0]**: Port x configuration bits (y = 0..15)

These bits are written by software to configure the I/O direction mode.

00: Input (reset state)

01: General purpose output mode

10: Alternate function mode

11: Analog mode

GPIO_MODE_INPUT

This mode connects the pin to the **digital input buffer**.

Conceptually:

- The pin senses logic levels (0/1)
- Schmitt trigger and digital sampling logic are active
- Analog path is *not* optimized

Engineers choose this when:

- Reading buttons or digital sensors
- Using the pin purely as a logic signal
- ADC is **not** involved

In ADC systems, this mode is usually **avoided**, because the digital input buffer introduces noise and leakage.

GPIO_MODE_OUTPUT

This mode connects the pin to a **digital output driver**.

Conceptually:

- The MCU actively drives the pin high or low
- Strong output transistors are enabled
- The pin is no longer a passive signal input

Engineers use this for:

- LEDs
- Control signals
- Chip selects

In ADC/DMA designs, this mode is **never used** for analog inputs because it **forces voltage onto the pin**, overriding external signals.

GPIO_MODE_ALTFN (Alternate Function)

This mode disconnects normal GPIO logic and hands control to a **peripheral**.

Conceptually:

- The pin becomes a wire for internal hardware (SPI, I²C, USART, TIM, etc.)
- GPIO logic is bypassed
- The peripheral owns timing and signal behavior

Engineers use this when:

- Pins are shared between peripherals
- Hardware timing accuracy is required

For ADC, this mode is typically **not used** (ADC has its own analog path), but it is critical when DMA-triggered peripherals are involved elsewhere in the system.

GPIO_MODE_ANALOG

This mode **disconnects all digital circuitry** from the pin.

Conceptually:

- No digital input buffer
- No digital output driver
- Pin connects directly to the analog domain

This is the **correct and mandatory mode for ADC inputs**.

Engineers choose this because:

- It minimizes noise

- It eliminates leakage from digital logic
- It reduces power consumption
- It allows accurate sampling by the ADC

When DMA continuously transfers ADC data, this mode ensures the signal path is **purely analog** from pin to ADC.

```
// ADC Control Register 1 (CR1)
#define ADC_CR1_SCAN      (1U << 8)  // Scan mode enable bit (Bit 8). Scans all channels.

// ADC Control Register 2 (CR2)
#define ADC_CR2_ADON      (1U << 0)  // A/D Converter ON / OFF bit (Bit 0)
#define ADC_CR2_DMA       (1U << 8)  // Direct Memory Access mode enable bit (Bit 8)
#define ADC_CR2_DDS       (1U << 9)  // DMA disable selection bit (Bit 9). 1=DMA requests continue.
#define ADC_CR2_EXTEN_RISING (1U << 28) // External trigger enable: Rising edge (Bits 28-29 -> 01)
#define ADC_CR2_EXTSEL_TIM2_TRGO (0x6U << 24) // External event select: TIM2_TRGO (Bits 24-27 -> 0110)

// ADC Regular Sequence Register 1 (SQR1)
#define ADC_SQR1_L_2CONV  (1U << 20) // Regular channel sequence length: 2 conversions (Bits 20-23 -> 0001)
```

13.13.2 ADC control register 1 (ADC_CR1)

Address offset: 0x04

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	OVRIE	RES[1:0]		AWDEN	JAWDEN	Res.	Res.	Res.	Res.	Res.	Res.
					r/w	r/w	r/w	r/w	r/w						
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DISCNUM[2:0]			JDISCEN	DISCEN	JAUTO	AWDSGL	SCAN	JEOCIE	AWDIE	EOCIE	AWDCH[4:0]				
r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w

ADC_CR1_SCAN — Scan Mode Enable

This setting tells the ADC to **walk through multiple input channels automatically** instead of repeatedly sampling a single one.

Conceptually:

- The ADC becomes a *sequencer*
- Each conversion step corresponds to a different channel
- Results are produced in a fixed, predictable order

Engineers use this when:

- Measuring voltage + current
- Sampling multiple sensors
- Feeding DSP blocks with synchronized data sets

Without scan mode, multi-channel acquisition becomes CPU-heavy and error-prone.

13.13.3 ADC control register 2 (ADC_CR2)

Address offset: 0x08

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	SWSTART	EXTEN[1:0]		EXTSEL[3:0]				Res.	JSWSTART	JEXTEN[1:0]		JEXTSEL[3:0]			
	rw	rw	rw	rw	rw	rw	rw		rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	ALIGN	EOCS	DDS	DMA	Res.	Res.	Res.	Res.	Res.	Res.	CONT	ADON
				rw	rw	rw	rw							rw	rw

ADC_CR2_ADON — ADC Enable

This setting powers up the ADC's internal analog circuitry.

Conceptually:

- Reference buffers are activated
- Sampling capacitors are energized
- Conversion logic becomes operational

Engineers treat this as the **ADC master switch**.

Configuration has no effect unless the ADC is actually enabled.

ADC_CR2_DMA — DMA Enable

This setting allows ADC results to be **exported directly to memory** without CPU involvement.

Conceptually:

- ADC produces data
- DMA transports data
- CPU is bypassed

Engineers enable this when:

- Sampling continuously
- Processing large data sets
- Implementing FFT, RMS, power, or harmonic analysis

This is the step that turns the ADC from a *sensor reader* into a *data stream source*.

ADC_CR2_DDS — DMA Continuous Requests

This setting determines whether DMA requests **stop after one sequence** or **continue indefinitely**.

Conceptually:

- ADC → DMA becomes a *pipeline*
- Each completed sequence automatically triggers the next transfer

Engineers use this for:

- Continuous waveform sampling
- Real-time monitoring
- Streaming data to DSP buffers

Without this, DMA would stop silently after one batch.

ADC_CR2_EXTEN_RISING — External Trigger Enable (Rising Edge)

This setting tells the ADC to **wait for a hardware event** before starting conversion.

Conceptually:

- ADC no longer runs on CPU commands
- It listens to an external timing signal
- Sampling becomes deterministic and jitter-free

Engineers use this when:

- Timing accuracy matters
- Sampling must align with line frequency
- ADC must sync with PWM, timers, or control loops

This is essential in **energy and power-quality systems**.

ADC_CR2_EXTSEL_TIM2_TRGO — Trigger Source Selection

This setting chooses **which hardware event controls the ADC**.

Conceptually:

- A timer becomes the *sampling clock*
- ADC conversions are locked to timer edges
- Sampling frequency is defined by timer configuration

Engineers choose timer triggers because:

- Timers are stable
- Timers are precise
- Timers are hardware-controlled

This removes CPU jitter entirely from the sampling process.

13.13.9 ADC regular sequence register 1 (ADC_SQR1)

Address offset: 0x2C

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res	Res	Res	Res	Res	Res	Res	Res	L[3:0]				SQ16[4:1]			
								RW	RW	RW	RW	RW	RW	RW	RW
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SQ16_0	SQ15[4:0]					SQ14[4:0]					SQ13[4:0]				
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW

ADC_SQR1_L_2CONV — Sequence Length = 2 Conversions

This setting defines **how many channels form one complete sampling sequence**.

Conceptually:

- One “ADC cycle” contains two samples
- DMA transfers two results per trigger
- Data alignment is guaranteed

Engineers use this for:

- Voltage + current pairs
- Differential measurements
- Multi-parameter sampling per instant

This makes the ADC output **structurally meaningful**, not just raw numbers.

```
// DMA Streams
#define DMA_STREAM_EN (1U << 0) // Stream Enable bit (Bit 0)
```

DMA_STREAM_EN — Stream Enable

This bit represents the **moment DMA stops being a configuration and starts being a machine**.

Conceptually, everything before this bit is:

- Planning
- Wiring
- Arming

Setting this bit means:

- The DMA stream is now *live*
- Hardware is allowed to move data autonomously
- Transfers begin as soon as trigger conditions are met

Engineers think of this bit as the **gate between setup and execution**.

Timer driver.h header file Explanations

```
// Timer Parameters for 8kHz Trigger
// Timer Clock = 16MHz (APB1 clock)
// Target Frequency = 8000Hz (for ADC conversion rate)
// Formula: ARR = (TimerClock / ( (PSC+1) * TargetFreq )) - 1
// With PSC = 0, ARR = (16000000 / (1 * 8000)) - 1 = 1999
#define TIM2_PSC_VALUE      0U          // Prescaler value (0 means divide by 1)
#define TIM2_ARR_VALUE      1999U       // Auto-Reload Register value

// TIM CR2 Bits
#define TIM_CR2_MMS_UPDATE  (0x2U << 4) // Master Mode Selection: Update Event (TRG0). Bits 4-6 -> 010.

// TIM CR1 Bits
#define TIM_CR1_CEN         (1U << 0)   // Counter Enable bit (Bit 0)
```

Timer clock assumption (16 MHz APB1)

This defines the **raw time resolution** available to the timer.

Every timing decision the timer makes is ultimately derived from this clock.

Engineers care because:

- Any clock error directly becomes a sampling-rate error
- ADC spectral accuracy depends on clock stability

This is the **root of time truth** for the system.

Prescaler value (PSC = 0)

This choice means:

- The timer counts **every incoming clock tick**
- No clock division is applied
- Maximum timing resolution is preserved

Engineers do this when:

- Target frequency is moderate
- Fine granularity is needed
- Jitter must be minimized

Prescalers trade resolution for range—here, resolution wins.

Auto-Reload value (ARR = 1999)

This value defines **how many timer ticks make one full period**.

Conceptually:

- The timer counts from 0 → ARR
- When ARR is reached, a cycle ends
- One precise timing event occurs

With this value, one complete cycle equals **8 kHz**, meaning:

- Exactly 8000 equally spaced trigger moments per second
- Uniform sampling intervals
- Predictable DSP behavior

This is the **heartbeat** of the acquisition system.

Master Mode Selection: Update Event (TRGO)

This configuration turns the timer into a **time master**.

Conceptually:

- The timer doesn't just count
- It *announces* when a period completes
- Other peripherals can synchronize to it

The ADC listens to this announcement.

This is what converts:

- A free-running timer
into
 - A system-wide timing reference
-

Counter Enable (CEN)

This is the moment when **time starts flowing**.

Before this:

- The timer is configured but inert
- No counting happens
- No triggers are generated

After this:

- The counter advances automatically
- Update events occur periodically
- ADC triggering becomes continuous

Engineers see this as the **commit point** for timing.

SOURCE CODE

Adc dma driver.c source file Explanations

```
void ADC_DMA_Init(uint32_t *buffer, uint32_t length) {  
    // 1. Enable Peripheral Clocks  
    ENABLE_GPIOA();    // Enable Clock for GPIO Port A (Pins PA0, PA1) by setting RCC AHB1ENR bit  
    ENABLE_ADC1();      // Enable Clock for ADC1 Peripheral by setting RCC APB2ENR bit  
    ENABLE_DMA2();      // Enable Clock for DMA2 Peripheral (ADC1 is on DMA2) by setting RCC AHB1ENR bit  
}
```

The signal chain is:

Analog Pin → GPIO → ADC → DMA → Memory

If any one of these blocks is not clocked, the entire chain silently fails.

Enabling GPIOA clock

This step powers the **physical interface to the outside world**.

Conceptually:

- The GPIO block controls how the pin is electrically connected
- Without its clock, pin mode settings do not take effect
- The ADC cannot reliably see the external signal

Engineers enable the GPIO clock first because:

- Pin mode must be set before sampling
- Analog paths depend on GPIO configuration
- Floating or misconfigured pins ruin measurement accuracy

This step ensures the **signal can physically enter the chip**.

Enabling ADC1 clock

This step powers the **conversion engine**.

Conceptually:

- Reference buffers, sampling capacitors, and comparators are activated
- The ADC becomes capable of turning voltage into numbers
- Conversion logic becomes responsive to triggers

Engineers treat this as:

- Powering the measurement instrument
- Not yet measuring, just waking it up

Without this clock:

- ADC registers may appear writable
- But no conversion can ever occur

Enabling DMA2 clock

This step powers the **data transport engine**.

Conceptually:

- DMA becomes capable of moving data autonomously
- ADC results can flow directly to memory
- CPU is removed from the critical data path

Engineers enable DMA early because:

- ADC is configured to rely on DMA
- Missing DMA clock causes silent data loss
- High-speed sampling depends on hardware movement, not software copies

This step ensures **numbers don't get stuck inside the ADC**.

```
// 2. Configure GPIO Pins (PA0, PA1) as Analog Mode
// MODER Register: Set bits 0-1 (PA0) and 2-3 (PA1) to '11' (Binary 3) which is Analog Mode
GPIOA->MODER |= (GPIO_MODE_ANALOG << 0) | (GPIO_MODE_ANALOG << 2);
```

Configuring PA0 and PA1 in analog mode disconnects all digital circuitry and creates a clean electrical path from the external signal to the ADC, which is essential for accurate, low-noise sampling in ADC + DMA systems.

```
// 3. Configure ADC1 Settings

// CR1 (Control Register 1): Enable SCAN Mode
// Scan mode converts channels in a group one after another
ADC1->CR1 |= ADC_CR1_SCAN;

// CR2 (Control Register 2): DMA and Trigger Configuration
// ADC_CR2_DMA: Enable Direct Memory Access mode, ADC requests DMA transfer after conversion
// ADC_CR2_DDS: DMA disable selection. 1 = DMA requests continue forever (Circular buffer support)
// ADC_CR2_EXTSEL_TIM2_TRGO: Select External Event 6 (TIM2_TRGO) (Bits 24-27 = 0110)
// ADC_CR2_EXTEN_RISING: Enable External Trigger on Rising Edge (Bits 28-29 = 01)
ADC1->CR2 |= (ADC_CR2_DMA | ADC_CR2_DDS | ADC_CR2_EXTSEL_TIM2_TRGO | ADC_CR2_EXTEN_RISING);

// SQR1 (Regular Sequence Register 1): Sequence Length
// Clear L bits (20-23) first to reset length configuration
ADC1->SQR1 &= ~(0xFU << 20);
// Set L = 1 (2 conversions) because L is (Count - 1). So 1 means 2 channels.
ADC1->SQR1 |= ADC_SQR1_L_2CONV;

// SQR3 (Regular Sequence Register 3): Channel Selection
// SQ1 (Bits 0-4) is the 1st conversion in sequence.
// SQ2 (Bits 5-9) is the 2nd conversion in sequence.
// We want Channel 0 first, then Channel 1.
ADC1->SQR3 &= ~((0x1FU << 0) | (0x1FU << 5)); // Clear SQ1 and SQ2 fields
ADC1->SQR3 |= (0U << 0) | (1U << 5); // Set SQ1=0 (Ch0), SQ2=1 (Ch1)

// Enable ADC Peripheral by setting ADON bit in CR2
ADC1->CR2 |= ADC_CR2_ADON;
```

ADC1 Configuration for Multi-Channel, Timer-Triggered Conversion with DMA

In this step, **ADC1** is configured to operate as a multi-channel data acquisition unit with **hardware triggering and DMA support**. The configuration enables the ADC to perform sequential conversions on multiple channels, synchronize sampling with a timer event, and automatically transfer conversion results to memory without CPU intervention.

Enabling Scan Mode (CR1)

Scan mode is enabled to allow the ADC to convert **multiple channels sequentially** within a single conversion sequence. When scan mode is active, the ADC automatically progresses through the configured channel list, converting each channel one after another in a predefined order. This mode is essential for applications that require simultaneous acquisition of multiple analog signals, such as voltage and current measurements in energy monitoring systems.

DMA and External Trigger Configuration (CR2)

DMA mode is enabled so that each completed ADC conversion generates a DMA request. This allows the ADC conversion results to be transferred directly to memory, eliminating CPU involvement and ensuring consistent data throughput. Continuous DMA request mode is also enabled so that DMA transfers do not stop after a single sequence, supporting uninterrupted data streaming.

The ADC is configured to start conversions based on an **external trigger**, specifically the **TIM2 trigger output (TRGO)**. The trigger is set to respond on the **rising edge**, ensuring that ADC sampling occurs at precise, deterministic intervals defined by the timer. This approach removes software timing jitter and guarantees uniform sampling periods, which are critical for accurate digital signal processing and frequency-domain analysis.

Regular Conversion Sequence Length Configuration (SQR1)

The regular conversion sequence length is configured to **two conversions per sequence**. This setting informs the ADC that each trigger event should result in two channel conversions before the sequence completes. Defining the sequence length ensures that the ADC and DMA remain synchronized and that the correct number of samples is transferred to memory for each sampling instant.

Channel Selection and Order Configuration (SQR3)

The conversion order is configured such that **Channel 0** is converted first, followed by **Channel 1**. This establishes a fixed and predictable sampling order within each ADC sequence. Maintaining a consistent channel order is crucial for correctly interpreting sampled data, especially when channel values are processed as paired measurements in downstream calculations or DSP algorithms.

Enabling the ADC Peripheral

Finally, the ADC is enabled by activating its control logic. This step powers the internal ADC circuitry and prepares it to respond to external trigger events. At this stage, the ADC is fully configured but remains idle until it receives a trigger signal from the timer, ensuring controlled and synchronized operation.

```
// 4. Configure DMA2 Stream 0 (ADC1 is mapped to Stream 0, Channel 0)

// Disable Stream 0 before configuration by clearing EN bit in CR
DMA2_Stream0->CR &= ~DMA_STREAM_EN;

// Wait until EN bit is cleared (hardware acknowledgement)
// This ensures no active transfer is interrupted and stream is ready
while((DMA2_Stream0->CR & DMA_STREAM_EN) != 0U);

// Configure Addresses
// PAR: Peripheral Address Register. Set to ADC1 Data Register (DR) address
DMA2_Stream0->PAR = (uint32_t)&ADC1->DR;
// M0AR: Memory 0 Address Register. Set to the user provided buffer address
DMA2_Stream0->M0AR = (uint32_t)buffer;
// NDTR: Number of Data Items to Transfer. Set to buffer length.
DMA2_Stream0->NDTR = length;

// Configure Stream Control Register (CR)
// Channel Selection (CHSEL): Channel 0 is 000 (Bits 25-27)
// Priority Level (PL): Very High is 11 (3) (Bits 16-17)
// Memory Data Size (MSIZE): 32-bit is 10 (2) (Bits 13-14)
// Peripheral Data Size (PSIZE): 32-bit is 10 (2) (Bits 11-12)
// Memory Increment Mode (MINC): Enabled is 1 (Bit 10) - increment memory pointer
// Circular Mode (CIRC): Enabled is 1 (Bit 8) - buffer wraps around
// Data Transfer Direction (DIR): Peripheral to Memory is 00 (Bits 6-7)
// The following expression combines these flags:
DMA2_Stream0->CR = (0U << 25) | (3U << 16) | (2U << 13) | (2U << 11) | (1U << 10) | (1U << 8);

// Enable DMA Stream by setting EN bit in CR
DMA2_Stream0->CR |= DMA_STREAM_EN;
```

DMA2 Stream 0 Configuration for ADC1 Data Transfer

In this step, **DMA2 Stream 0** is configured to automatically transfer ADC1 conversion results from the ADC data register to a memory buffer. This configuration enables continuous, high-speed data movement without CPU intervention, which is essential for real-time data acquisition and digital signal processing applications.

Disabling the DMA Stream Before Configuration

Before configuring the DMA stream, Stream 0 is explicitly disabled. This ensures that no data transfer is active during configuration. DMA hardware does not allow modification of critical configuration fields while a stream is enabled, as doing so could corrupt ongoing transfers. Waiting until the enable bit is fully cleared guarantees that the stream is in a safe and idle state before proceeding.

Peripheral and Memory Address Configuration

The peripheral address is set to the **ADC1 data register**, which is the location where each ADC conversion result becomes available. This establishes the source of the data stream.

The memory address is set to a user-provided buffer in RAM. This buffer acts as the destination for the ADC samples. By defining this address, the DMA engine knows exactly where to store each conversion result as it is produced.

The number of data items to transfer is configured to match the buffer length. This defines how many ADC samples the DMA should move before completing or wrapping the transfer sequence.

DMA Stream Control Configuration

The DMA stream is configured to use **Channel 0**, which corresponds to the ADC1 DMA request mapping. This ensures that DMA transfer requests generated by ADC1 are correctly routed to this stream.

A **very high priority level** is selected for the DMA stream. This minimizes the chance of transfer delays when multiple DMA streams or bus masters are active, ensuring reliable and timely acquisition of ADC data.

Both peripheral and memory data sizes are configured as **32-bit**, matching the ADC data register width and ensuring proper data alignment in memory.

Memory increment mode is enabled so that each successive ADC sample is written to the next memory location in the buffer. This allows the buffer to store a sequence of samples rather than repeatedly overwriting a single location.

Circular mode is enabled to allow the buffer to **wrap around automatically** once the end is reached. This supports continuous data acquisition, where the DMA repeatedly fills the buffer with new samples without requiring reconfiguration.

The data transfer direction is set from **peripheral to memory**, reflecting the flow of data from the ADC to system memory.

Enabling the DMA Stream

After all configuration parameters are set, the DMA stream is enabled. This action activates the DMA engine and allows it to respond to ADC conversion completion events. Once enabled, the DMA autonomously transfers ADC results into the memory buffer whenever the ADC generates a DMA request.

At this stage, the DMA is fully operational and ready to support continuous ADC data acquisition in synchronization with the configured trigger source.

I2C driver.c source file Explanations

```
/*
 * @brief  Initializes I2C1 Peripheral
 * @param  None
 * @retval None
 */
void I2C1_Init(void) {
    // 1. Enable Clocks
    ENABLE_GPIOB();           // Enable Clock for GPIOB (SCL/SDA pins are on PB8/PB9)
    ENABLE_I2C1();            // Enable Clock for I2C1 Peripheral
}
```

Enabling Clocks for I2C1 and GPIOB

In this step, the required clock signals for **GPIO Port B** and the **I2C1 peripheral** are enabled. Clock activation is a mandatory prerequisite for configuring and operating any peripheral in the STM32 microcontroller, as peripherals remain inactive until their corresponding clock domains are powered.

The clock for **GPIOB** is enabled first because the I2C1 communication lines—**SCL (Serial Clock Line)** and **SDA (Serial Data Line)**—are mapped to **PB8 and PB9**. Enabling the GPIOB clock ensures that these pins can be properly configured for alternate function operation and that their electrical characteristics can be controlled by software.

Next, the clock for the **I2C1 peripheral** is enabled. This action powers the internal I2C logic, including the timing generation unit, protocol state machine, and data shift registers. Without this clock, any configuration written to I2C1 registers would have no functional effect, and communication on the I2C bus would not be possible.

This step strictly performs peripheral clock activation and does not configure pin modes, timing parameters, or communication behavior. It establishes the electrical readiness of both the GPIO interface and the I2C peripheral, forming the foundation for subsequent I2C configuration operation.

```
// 2. Configure GPIO Pins: PB8 (SCL), PB9 (SDA) as Alternate Function Open Drain

// MODER: Set PB8, PB9 to Alternate Function (10 in binary = 2U)
// Shift by 16 for Pin 8, 18 for Pin 9
GPIOB->MODER &= ~((3U << 16) | (3U << 18));
GPIOB->MODER |= (2U << 16) | (2U << 18);

// OTYPER: Set PB8, PB9 to Open Drain (1)
// Open Drain is required for I2C bus because lines are pulled up externally
GPIOB->OTYPER |= (1U << 8) | (1U << 9);

// OSPEEDR: Set Speed to High Speed (11 in binary = 3U)
// Faster slew rate for cleaner edges
GPIOB->OSPEEDR &= ~((3U << 16) | (3U << 18));
GPIOB->OSPEEDR |= ((3U << 16) | (3U << 18));

// AFRH: Set Alternate Function 4 (AF4 = 0100 = 4U) for I2C1
// PB8 (Bits 0-3 in AFRH), PB9 (Bits 4-7 in AFRH)
GPIOB->AFRH &= ~((0xFU << 0) | (0xFU << 4));
GPIOB->AFRH |= ((4U << 0) | (4U << 4));
```

GPIO Configuration for I2C1 Communication (PB8 – SCL, PB9 – SDA)

In this step, **GPIO pins PB8 and PB9** are configured to operate as **I2C1 communication lines**, specifically the **Serial Clock Line (SCL)** and **Serial Data Line (SDA)**. Proper GPIO configuration is essential for reliable I2C communication, as these pins must adhere to strict electrical and functional requirements defined by the I2C protocol.

Configuring Pins for Alternate Function Mode

The GPIO mode for PB8 and PB9 is set to **alternate function mode**, allowing control of these pins to be handed over from general-purpose GPIO logic to the I2C peripheral. In this mode, the pin behavior is no longer driven by software GPIO writes but by the internal I2C hardware, which generates clock pulses and data transitions according to the I2C protocol.

This configuration ensures that the pins can correctly generate and respond to I2C timing signals rather than behaving as simple digital inputs or outputs.

Configuring Output Type as Open-Drain

The output type of PB8 and PB9 is configured as **open-drain**. Open-drain operation is a fundamental requirement of the I2C bus architecture. In this configuration, devices on the bus can only pull the line low, while the line is pulled high by external pull-up resistors.

This design allows multiple devices to share the same bus safely without electrical contention. If multiple devices attempt to drive the line simultaneously, the open-drain structure prevents short circuits and ensures correct arbitration and acknowledgment behavior.

Configuring Output Speed

The output speed of PB8 and PB9 is set to **high speed**. This controls the slew rate of the output signals, allowing faster rise and fall times. For I2C communication, especially at higher clock frequencies, sharper signal edges help maintain signal integrity and reduce timing uncertainty caused by slow transitions.

Although I2C relies on pull-up resistors for rising edges, configuring the GPIO speed appropriately helps ensure reliable low-level driving and clean signal behavior.

Selecting the Alternate Function Mapping (AF4)

Finally, the alternate function selection for PB8 and PB9 is configured to **AF4**, which corresponds to **I2C1**. This step explicitly connects the physical GPIO pins to the internal I2C1 peripheral signals.

By assigning the correct alternate function, the microcontroller routes the I2C clock and data signals between the I2C hardware block and the external pins. Without this mapping, the I2C peripheral would be unable to communicate with devices on the bus, even if all other configurations were correct.

```
// 3. I2C Configuration

// Reset I2C: Set SWRST (Software Reset) bit in CR1
I2C1->CR1 |= I2C_CR1_SWRST;
// Clear SWRST bit to release reset and allow configuration
I2C1->CR1 &= ~I2C_CR1_SWRST;

// CR2: Set Peripheral Clock Frequency
// APB1 Clock is 16MHz (default HSI). Set FREQ bits (0-5) to 16.
I2C1->CR2 = 16U;

// CCR (Clock Control Register): Set I2C Speed
// For Standard Mode 100kHz:
//  $T_{high} + T_{low} = 10\mu s$  (Period of 100kHz)
//  $T_{pclk} = 62.5ns$  (1/16MHz)
//  $CCR = F_{pclk} / (2 * F_{i2c}) = 16000000 / (2 * 100000) = 80$ .
I2C1->CCR = 80U;

// TRISE (Rise Time Register):
// Max Rise Time for Standard Mode = 1000ns.
//  $TRISE = (Max\ Rise\ Time / T_{pclk}) + 1$ 
//  $1000ns / 62.5ns = 16$ .
//  $TRISE = 16 + 1 = 17$ .
I2C1->TRISE = 17U;

// Enable I2C Peripheral by setting PE bit in CR1
I2C1->CR1 |= I2C_CR1_PE;
```

I2C1 Peripheral Configuration

In this step, the **I2C1 peripheral** is configured to operate in **standard mode (100 kHz)** using the APB1 clock as the timing reference. This configuration establishes correct internal timing, ensures protocol compliance, and prepares the I2C peripheral for reliable communication on the bus.

Software Reset of the I2C Peripheral

The I2C peripheral is first placed into a **software reset state**. This action clears all internal logic, state machines, and residual configuration from any previous operation. Applying a software reset ensures that the I2C peripheral starts from a known, clean state.

After the reset is asserted, it is immediately released. This allows the peripheral to resume normal operation and accept new configuration values. Performing a reset before configuration is a common practice to avoid unpredictable behavior caused by leftover states or bus errors.

Peripheral Clock Frequency Configuration (CR2)

The peripheral clock frequency is configured to match the **APB1 clock**, which operates at **16 MHz**. This value informs the I2C peripheral of the clock frequency driving its internal logic.

Accurate knowledge of the peripheral clock frequency is essential because the I2C timing parameters—such as SCL high and low periods—are derived from this clock. Incorrect configuration at this stage would result in inaccurate bus timing and potential communication failures.

I2C Clock Speed Configuration (CCR)

The I2C clock control register is configured to generate an **SCL frequency of 100 kHz**, corresponding to standard-mode I2C operation. This configuration defines the duration of the SCL high and low periods based on the peripheral clock frequency.

By selecting standard mode timing, the system ensures maximum compatibility with a wide range of I2C devices while maintaining stable signal behavior and sufficient noise margin on the bus.

Rise Time Configuration (TRISE)

The rise time register is configured to comply with the **maximum allowed signal rise time** for standard-mode I2C communication. This setting compensates for the effects of pull-up resistors and bus capacitance on the SDA and SCL lines.

Correct rise time configuration ensures that signal transitions meet I2C electrical specifications, preventing timing violations that could otherwise cause incorrect data sampling or bus errors.

Enabling the I2C Peripheral

Finally, the I2C peripheral is enabled by activating its internal control logic. At this stage, all timing and control parameters are already configured, allowing the peripheral to begin operation safely.

Once enabled, the I2C1 peripheral is fully prepared to generate clock signals, transmit and receive data, and participate in I2C bus communication according to the configured parameters.

```

void I2C1_WriteMulti(uint8_t addr, uint8_t reg, uint8_t* d, uint16_t c) {
    volatile uint32_t timeout; // Volatile to prevent loop optimization

    // Wait until I2C bus is not busy (BUSY flag in SR2)
    timeout = I2C_TIMEOUT;
    while((I2C1->SR2 & I2C_SR2_BUSY) && (timeout > 0U)){ timeout--; }
    if(timeout == 0U) return; // Error: Bus Busy stuck

    // Generate START condition (START bit in CR1)
    I2C1->CR1 |= I2C_CR1_START;

    // Wait for Start Bit (SB) generated flag in SR1
    timeout = I2C_TIMEOUT;
    while(!(I2C1->SR1 & I2C_SR1_SB) && (timeout > 0U)){ timeout--; }
    if(timeout == 0U) return; // Error: Start bit not set

    // Send 7-bit Address. Shift LSB is 0 for Write operation.
    // I2C Standard: Address is transmitted in bits 7:1
    I2C1->DR = (addr << 1); // Write operation

    // Wait for Address matched (ADDR) flag in SR1
    timeout = I2C_TIMEOUT;
    while(!(I2C1->SR1 & I2C_SR1_ADDR) && (timeout > 0U)){ timeout--; }
    if(timeout == 0U) return; // Error: Address not acknowledged

    // Clear ADDR flag: This is done by reading SR1 (done in loop) followed by reading SR2.
    (void)I2C1->SR2;

    // Wait for Transmit Empty (TXE) flag in SR1
    timeout = I2C_TIMEOUT;
    while(!(I2C1->SR1 & I2C_SR1_TXE) && (timeout > 0U)){ timeout--; }

    // Send Register Address as the first data byte
    I2C1->DR = reg;
}

```

I2C Multi-Byte Write Operation (Register-Based Communication)

This function implements a **multi-byte write transaction** over the I2C bus, following the standard **master-transmitter sequence**. It is designed to write data to a specific register of an I2C slave device while ensuring bus safety, protocol compliance, and fault tolerance.

Bus Availability Check

Before initiating communication, the function checks whether the I2C bus is currently busy. The I2C protocol allows only one master transaction at a time, and attempting to start communication while the bus is occupied can lead to arbitration conflicts or undefined behavior.

A timeout mechanism is used to prevent the system from waiting indefinitely in case the bus becomes stuck due to noise, improper termination, or a malfunctioning slave device. If the bus does not become free within the allowed time, the operation is safely aborted.

START Condition Generation

Once the bus is confirmed to be free, a **START condition** is generated. This marks the beginning of a new I2C transaction and signals all devices on the bus that a master is initiating communication.

The function waits until the hardware confirms that the START condition has been successfully generated. This confirmation ensures that the bus is now under master control and ready for address transmission.

Slave Address Transmission (Write Mode)

After the START condition, the 7-bit slave address is transmitted with the **write direction** selected. According to the I2C standard, the least significant bit of the address frame specifies the transfer direction, and a cleared bit indicates a write operation.

The function waits for acknowledgment from the slave device. This acknowledgment confirms that the addressed device is present on the bus and ready to receive data. A timeout is again used to handle cases where the slave does not respond.

Address Phase Completion

Once the slave acknowledges the address, the address phase must be formally completed. This is achieved by clearing the internal address-matched condition, allowing the I2C peripheral to proceed to the data transmission phase.

This step ensures proper synchronization between the master and slave and prevents the bus from remaining locked in the address state.

Transmit Buffer Readiness Check

Before sending any data bytes, the function waits until the transmit buffer is empty. This guarantees that the I2C hardware is ready to accept new data and prevents overwriting of pending transmissions.

Ensuring transmit buffer readiness is critical for maintaining data integrity, especially in systems operating at higher I2C speeds.

Register Address Transmission

The first data byte transmitted after the address phase is the **register address** within the slave device. Most I2C peripherals use an internal register map, and this step selects the target register where subsequent data will be written.

By sending the register address first, the master establishes the internal write pointer of the slave device, preparing it to receive one or more data bytes.

```

// Loop to send remaining data bytes from buffer
for(uint16_t i=0; i<c; i++) {
    // Wait for TXE (buffer empty)
    timeout = I2C_TIMEOUT;
    while(!(I2C1->SR1 & I2C_SR1_TXE) && (timeout > 0U)){ timeout--; }

    // Send Data Byte
    I2C1->DR = d[i];
}

// Wait for last byte TXE
timeout = I2C_TIMEOUT;
while(!(I2C1->SR1 & I2C_SR1_TXE) && (timeout > 0U)){ timeout--; }

// Wait for Byte Transfer Finished (BTF) flag in SR1
// This ensures last byte is physically transmitted and ACK'd before we send STOP.
timeout = I2C_TIMEOUT;
while(!(I2C1->SR1 & I2C_SR1_BTF) && (timeout > 0U)){ timeout--; }

// Generate STOP condition (STOP bit in CR1)
I2C1->CR1 |= I2C_CR1_STOP;

```

Transmission of Data Bytes and Termination of I2C Write Operation

This step completes the **multi-byte data transmission phase** of the I2C write operation. After the slave device and target register have been successfully addressed, the remaining data bytes are transmitted sequentially, followed by proper termination of the communication.

Sequential Data Byte Transmission

The function enters a loop to transmit the remaining data bytes from the provided data buffer. Before sending each byte, the transmit buffer status is checked to ensure that it is empty and ready to accept new data. This guarantees that each data byte is handed to the I2C hardware only when the previous byte has been properly shifted into the transmission logic.

A timeout mechanism is applied during this waiting phase to prevent indefinite blocking in the event of a stalled bus or unresponsive slave device. This ensures robustness and fault tolerance during multi-byte transfers.

Each data byte is then written sequentially, allowing the slave device to store the incoming data into consecutive internal registers, as defined by its register addressing scheme.

Ensuring Completion of the Final Data Byte

After all data bytes have been written, the function performs an additional check to confirm that the transmit buffer is empty. This ensures that the final data byte has been successfully accepted by the I2C hardware for transmission.

Following this, the **Byte Transfer Finished (BTF)** condition is monitored. This step confirms that the final byte has not only been loaded into the shift register but has also been physically transmitted on the bus and acknowledged by the slave device. Waiting for this condition prevents premature termination of the transaction.

STOP Condition Generation

Once the final byte transmission is fully completed and acknowledged, a **STOP condition** is generated. The STOP condition signals the end of the I2C transaction and releases control of the bus, allowing other devices or masters to initiate communication.

Proper STOP generation is critical for maintaining bus integrity. It ensures that the slave device correctly latches the received data and that the bus returns to an idle state.

```
/*
 * @brief Writes a single byte to I2C Device (Wrapper for Multi)
 * @param addr: 7-bit Slave Address
 * @param reg: Register address
 * @param data: Data byte
 * @retval None
 */
void I2C1_Write(uint8_t addr, uint8_t reg, uint8_t data) {
    uint8_t tmp = data; // Store data in a temp variable
    I2C1_WriteMulti(addr, reg, &tmp, 1); // Reuse WriteMulti for single byte
}

// Stubs for Generic APIs if needed by other modules in future
void I2C_PerClockControl(I2C_RegDef_t *pI2Cx, uint8_t EnorDi) {}
void I2C_Init(I2C_Handle_t *pI2CHandle) {}
void I2C_DeInit(I2C_RegDef_t *pI2Cx) {}
void I2C_MasterSendData(I2C_Handle_t *pI2CHandle, uint8_t *pTxbuffer, uint32_t Len, uint8_t SlaveAddr, uint8_t Sr) {}
void I2C_MasterReceiveData(I2C_Handle_t *pI2CHandle, uint8_t *pRxBuffer, uint8_t Len, uint8_t SlaveAddr, uint8_t Sr) {}
uint8_t I2C_GetFlagStatus(I2C_RegDef_t *pI2Cx, uint32_t FlagName) { return 0; }
void I2C_ApplicationEventCallback(I2C_Handle_t *pI2CHandle, uint8_t AppEv) {}
```

Single-Byte I2C Write Operation Using Wrapper Function

This step implements a **single-byte write operation** to an I2C slave device by reusing the previously defined multi-byte write mechanism. Instead of duplicating protocol logic, a lightweight wrapper function is used to improve code reusability, clarity, and maintainability.

Purpose of the Single-Byte Write Wrapper

Many I2C devices expose internal registers that are frequently written one byte at a time, such as configuration registers, control flags, or command registers. Implementing a dedicated single-byte write function simplifies application-level code while relying on a common, well-tested multi-byte write routine underneath.

This approach avoids code duplication and ensures that all I2C write operations follow the same protocol sequence and error-handling behavior.

Temporary Data Handling

The data byte to be written is first stored in a temporary variable. This is necessary because the underlying multi-byte write function expects a pointer to a data buffer rather than a single value. By placing the data into a temporary variable, a valid memory address can be passed to the multi-byte write routine.

This design allows the single-byte operation to seamlessly reuse the same data transmission path used for multi-byte transfers.

Reusing the Multi-Byte Write Mechanism

The wrapper function invokes the multi-byte write routine with a data length of one byte. As a result, the same sequence of operations is performed:

- Bus availability check
- START condition generation
- Slave address transmission
- Register address selection
- Data byte transmission
- STOP condition generation

By leveraging the multi-byte implementation, the single-byte write operation automatically inherits all protocol safeguards, timing checks, and timeout handling already implemented in the lower-level function.

Design Rationale

This wrapper-based design provides several advantages:

- **Consistency:** All I2C write operations follow the same protocol flow.
- **Maintainability:** Any improvement or bug fix in the multi-byte write function benefits single-byte writes automatically.
- **Readability:** Application code can use a simple and intuitive API for common single-byte register writes.
- **Scalability:** The design easily supports additional wrapper functions for other access patterns if required.

Generic API Stubs for Future Expansion

The remaining functions in this section are **placeholder stubs** for generic I2C APIs. These stubs are included to maintain a consistent driver interface and to allow future expansion without modifying application-level code.

They define the expected structure for:

- Peripheral clock control
- Initialization and de-initialization
- Generic master transmit and receive operations
- Flag status queries
- Application-level event callbacks

Although these functions are currently empty or return default values, they establish a clear framework for extending the driver to support interrupt-based operation, DMA transfers, or multi-instance I2C handling in future revisions.

Uart driver.c source file Explanations

```
void USART_PeriClockControl(USART_RegDef_t *pUSARTx, uint8_t EnOrDi)
{
    if(EnOrDi == ENABLE)
    {
        if(pUSARTx == USART1)
        {
            RCC->APB2ENR |= (1U << 4); // Enable USART1 Clock
        }else if(pUSARTx == USART2)
        {
            RCC->APB1ENR |= (1U << 17); // Enable USART2 Clock
        }else if(pUSARTx == USART3)
        {
            RCC->APB1ENR |= (1U << 18); // Enable USART3 Clock
        }
        else if(pUSARTx == UART4)
        {
            RCC->APB1ENR |= (1U << 19); // Enable UART4 Clock
        }
        else if(pUSARTx == UART5)
        {
            RCC->APB1ENR |= (1U << 20); // Enable UART5 Clock
        }
        else if(pUSARTx == USART6)
        {
            RCC->APB2ENR |= (1U << 5); // Enable USART6 Clock
        }
    }else
    {
        // Disable logic to be added if needed
    }
}
```

USART Peripheral Clock Control

This function is responsible for **enabling the clock supply** to a selected USART/UART peripheral. Clock control is a fundamental requirement in STM32 microcontrollers, as a peripheral remains inactive until its corresponding clock is enabled through the Reset and Clock Control (RCC) unit.

Purpose of Peripheral Clock Control

USART peripherals are connected to different internal buses within the microcontroller. Each bus has a dedicated clock enable register. This function abstracts that complexity and provides a unified interface to activate the clock for any supported USART or UART instance.

By controlling the peripheral clock explicitly, the system ensures that:

- The USART hardware is powered and operational
- Register configurations written later take effect correctly
- Power consumption is managed by enabling clocks only when required

Conditional Peripheral Identification

The function identifies the target USART peripheral by comparing the provided peripheral base address pointer with known USART/UART instances. Each comparison corresponds to a specific hardware peripheral present in the microcontroller.

This approach allows a single function to support multiple USART instances without duplicating clock-enable logic for each peripheral.

Clock Enable for APB2-Based USARTs

USART1 and USART6 are connected to the **APB2 bus**. When either of these peripherals is selected, the function enables the corresponding clock by setting the appropriate bit in the APB2 peripheral clock enable register.

Enabling the APB2 clock activates the internal logic of the selected USART, allowing it to generate baud rates, transmit data, and receive data.

Clock Enable for APB1-Based USARTs and UARTs

USART2, USART3, UART4, and UART5 are connected to the **APB1 bus**. For these peripherals, the function enables the clock by setting the corresponding bit in the APB1 peripheral clock enable register.

Since APB1 peripherals typically operate at a lower clock frequency than APB2 peripherals, this distinction ensures that each USART operates with the correct timing reference.

Enable and Disable Control Logic

The function accepts an enable or disable parameter, allowing flexible control over peripheral power. When the enable option is selected, the clock for the specified USART is activated. The disable logic is reserved for future expansion and can be implemented if power-down or dynamic peripheral control is required.

This design supports scalable power management and aligns with low-power embedded system design practices.

```

void USART_SetBaudRate(USART_RegDef_t *pUSARTx, uint32_t BaudRate)
{
    uint32_t usartdiv; // Calculated USART Divider
    uint32_t M_part, F_part; // Mantissa and Fraction parts

    // Formula: USARTDIV = fCK / (8 * (2 - OVER8) * BaudRate)
    // Assuming OVER8 = 0 (Oversampling by 16) -> fCK / (16 * BaudRate)

    if(pUSARTx->CR1 & USART_CR1_OVER8)
    {
        //OVER8 = 1 , over sampling by 8
        usartdiv = ((25U * PCLK1_FREQ) / (2U * BaudRate));
    }else
    {
        //over sampling by 16 (Standard)
        usartdiv = ((25U * PCLK1_FREQ) / (4U * BaudRate));
    }

    // Calculate Mantissa: Integer part of division by 100
    // (Note: The formula above multiplied by 25/100 to avoid floats)
    M_part = usartdiv / 100U;

    // Shift Mantissa to bits 15:4
    pUSARTx->BRR = M_part << 4;

    // Calculate Fraction: Remainder of division
    F_part = (usartdiv - (M_part * 100U));

    if(pUSARTx->CR1 & USART_CR1_OVER8)
    {
        //OVER8 = 1
        // Scale fractional part to 3 bits
        F_part = (((F_part * 8U) + 50U) / 100U) & ((uint8_t)0x07);
    }else
    {
        //OVER8 = 0
        // Scale fractional part to 4 bits
        F_part = (((F_part * 16U) + 50U) / 100U) & ((uint8_t)0x0F);
    }

    // Add Fraction to BRR
    pUSARTx->BRR |= F_part;
}

```

USART Baud Rate Configuration

This function configures the **baud rate** of the selected USART peripheral by calculating and programming the **baud rate register (BRR)** based on the peripheral clock frequency and the selected oversampling mode. Correct baud rate configuration is essential to ensure reliable serial communication between the microcontroller and external devices.

Baud Rate Divider Calculation Principle

The USART generates the baud rate by dividing the peripheral clock using an internal divider. The value of this divider depends on whether the USART operates with **oversampling by 16** or

oversampling by 8. The oversampling mode determines how many samples are taken per bit period and directly affects baud rate precision and noise tolerance.

The function first determines the active oversampling mode by checking the oversampling configuration bit. Based on this, it selects the appropriate divider formula to compute the baud rate accurately.

Handling Oversampling Modes

When oversampling by 16 is used, the USART operates in its standard configuration, providing higher noise immunity. In this case, the divider calculation assumes sixteen samples per bit period.

When oversampling by 8 is enabled, the USART uses fewer samples per bit, allowing higher baud rates at the cost of reduced noise margin. The divider calculation is adjusted accordingly to maintain accurate timing under this mode.

By handling both oversampling options, the function ensures compatibility with a wide range of baud rates and performance requirements.

Integer Arithmetic Optimization

To avoid the use of floating-point arithmetic, which is computationally expensive on microcontrollers without floating-point units, the divider calculation is scaled using integer math. The computation is performed using scaled values to preserve precision while maintaining execution efficiency.

This approach ensures that the baud rate calculation remains accurate while being suitable for real-time embedded systems.

Mantissa Calculation

The calculated divider value is split into an **integer part**, known as the mantissa. This value represents the primary division factor applied to the peripheral clock. The mantissa is positioned into the upper bits of the baud rate register, as required by the USART hardware design.

Correct placement of the mantissa ensures that the USART generates the correct base timing for serial communication.

Fractional Part Calculation

The remainder of the divider calculation forms the **fractional part**, which fine-tunes the baud rate to improve accuracy. The size of the fractional field depends on the oversampling mode:

- In oversampling by 16 mode, a 4-bit fractional field is used.
- In oversampling by 8 mode, a 3-bit fractional field is used.

The fractional value is scaled, rounded, and masked appropriately to fit within the allowed bit width. This step minimizes baud rate error and improves communication reliability, especially at higher baud rates.

Programming the Baud Rate Register

Finally, the mantissa and fractional parts are combined and written into the baud rate register. This completes the baud rate configuration process and prepares the USART to transmit and receive data at the specified speed.

At this stage, the USART timing is fully defined, and subsequent data transfers will adhere to the configured baud rate.

```
/****** Configuration of CR1*****  
  
// Enable Clock for this peripheral  
USART_PeriClockControl(pUSARTHandle->pUSARTx, ENABLE);  
  
// Configure Transfer Mode (Tx/Rx)  
// Read user config and set RE (Receiver Enable) or TE (Transmitter Enable) bits  
if (pUSARTHandle->USART_Config.USART_Mode == USART_MODE_ONLY_RX)  
{  
    tempreg |= USART_CR1_RE;  
}  
else if (pUSARTHandle->USART_Config.USART_Mode == USART_MODE_ONLY_TX)  
{  
    tempreg |= USART_CR1_TE;  
}  
else if (pUSARTHandle->USART_Config.USART_Mode == USART_MODE_TXRX)  
{  
    tempreg |= (USART_CR1_RE | USART_CR1_TE);  
}  
  
// Configure Word Length (8 or 9 bits)  
tempreg |= pUSARTHandle->USART_Config.USART_WordLength << 12;  
// Careful with generic shifting, but macro is 0 or 1.  
  
// Configure Parity Control  
if (pUSARTHandle->USART_Config.USART_ParityControl == USART_PARITY_EN_EVEN)  
{  
    tempreg |= USART_CR1_PCE; // Enable Parity Control  
    // EVEN is default if PS bit is 0  
}  
else if (pUSARTHandle->USART_Config.USART_ParityControl == USART_PARITY_EN_ODD)  
{  
    tempreg |= USART_CR1_PCE; // Enable Parity Control  
    tempreg |= USART_CR1_PS; // Set Parity Selection to Odd  
}  
  
// Program the calculated value to CR1 register  
pUSARTHandle->pUSARTx->CR1 = tempreg;
```

USART Control Register 1 (CR1) Configuration

This step configures the **USART Control Register 1 (CR1)**, which defines the fundamental operating behavior of the USART peripheral. CR1 controls how data is transmitted and received, the data frame format, and error-detection mechanisms. Proper configuration of this register is essential before enabling USART communication.

Enabling the USART Peripheral Clock

Before modifying any USART configuration registers, the clock for the selected USART peripheral is enabled. This ensures that the internal USART logic is powered and capable of accepting

configuration settings. Without enabling the peripheral clock, writes to the control registers would have no functional effect.

Configuring Transfer Direction (Transmit and Receive)

The transfer mode is configured based on the user-selected operating mode:

- **Receive-only mode** enables the receiver circuitry, allowing the USART to accept incoming data.
- **Transmit-only mode** enables the transmitter circuitry, allowing the USART to send data.
- **Full-duplex mode** enables both transmitter and receiver, allowing simultaneous data transmission and reception.

This selective enabling ensures that only the required hardware blocks are active, optimizing power usage and preventing unintended data flow.

Configuring Word Length

The word length configuration determines the size of each data frame transmitted or received by the USART. Depending on the application requirements, the USART can operate with either **8-bit** or **9-bit** data frames.

This setting directly affects how data is framed on the serial line and must match the configuration of the communicating device to ensure correct data interpretation.

Configuring Parity Control

Parity control is configured to provide optional error detection during data transmission. If parity is enabled, an additional parity bit is appended to each data frame to allow the receiver to detect single-bit transmission errors.

- **Even parity** ensures that the total number of logic-high bits is even.
- **Odd parity** ensures that the total number of logic-high bits is odd.
- **Parity disabled** removes the parity bit entirely for applications that do not require error checking.

The selected parity configuration must match on both transmitting and receiving devices to maintain reliable communication.

Programming the CR1 Register

After all individual configuration fields are prepared, the combined configuration value is written into the CR1 register. This action applies the selected transfer mode, word length, and parity settings simultaneously, ensuring a consistent and stable USART configuration.


```

/***** Configuration of CR2 *****/
tempreg=0;
// Configure Stop Bits (1, 0.5, 2, 1.5)
tempreg |= pUSARTHandle->USART_Config.USART_NoOfStopBits << 12; // STOP bits position
// Program CR2
pUSARTHandle->pUSARTx->CR2 = tempreg;

```

USART Control Register 2 (CR2) Configuration

This step configures the **USART Control Register 2 (CR2)**, which defines the **frame termination characteristics** of USART communication. CR2 primarily controls the number of stop bits used to delimit each data frame, ensuring proper synchronization between the transmitter and receiver.

Resetting the Temporary Configuration Variable

Before configuring CR2, the temporary configuration variable is cleared. This ensures that no residual or unintended configuration bits are carried over from previous register setups. Starting with a clean configuration value guarantees that only the explicitly defined settings are applied to CR2.

Configuring Stop Bit Selection

The stop bit configuration determines how each USART data frame is terminated on the serial line. Depending on the application and communication requirements, different stop bit configurations may be used:

- **1 stop bit** is the most commonly used configuration and offers efficient data transmission.
- **0.5 stop bits** and **1.5 stop bits** are typically used in specialized communication scenarios.
- **2 stop bits** provide increased timing margin for slower or less precise receivers.

The selected stop bit setting is applied based on the user configuration structure and positioned correctly within CR2. This configuration must match the stop bit settings of the communicating device to ensure correct frame decoding.

Programming the CR2 Register

After the stop bit configuration is prepared, the value is written into the CR2 register. This action applies the selected frame termination behavior to the USART peripheral.

Once programmed, the USART hardware will use the configured stop bit length for all subsequent data transmissions and receptions.

```

/***** Configuration of CR3 *****/

tempreg=0;

// Configure Hardware Flow Control (CTS/RTS)
if ( pUSARTHandle->USART_Config.USART_HWFlowControl == USART_HW_FLOW_CTRL_CTS)
{
    tempreg |= USART_CR3_CTSE;
}
else if (pUSARTHandle->USART_Config.USART_HWFlowControl == USART_HW_FLOW_CTRL_RTS)
{
    tempreg |= USART_CR3_RTSE;
}
else if (pUSARTHandle->USART_Config.USART_HWFlowControl == USART_HW_FLOW_CTRL_CTS_RTS)
{
    tempreg |= (USART_CR3_CTSE | USART_CR3_RTSE);
}

// Program CR3
pUSARTHandle->USARTx->CR3 = tempreg;

```

USART Control Register 3 (CR3) Configuration

This step configures the **USART Control Register 3 (CR3)**, which controls advanced communication features related to **hardware flow control**. CR3 allows the USART to manage data flow automatically using dedicated control signals, improving reliability in high-speed or buffer-sensitive communication scenarios.

Resetting the Temporary Configuration Variable

Before configuring CR3, the temporary configuration variable is cleared to ensure a clean configuration state. This prevents any unintended bits from previous configurations from affecting the hardware flow control settings.

Configuring Hardware Flow Control

Hardware flow control uses additional signals to regulate the flow of data between the transmitter and receiver, preventing data loss when one side cannot process incoming data quickly enough. The configuration supports the following modes:

- **CTS (Clear To Send):** The USART checks the CTS signal before transmitting data. Transmission proceeds only when the receiving device indicates that it is ready.
- **RTS (Request To Send):** The USART asserts the RTS signal to inform the remote device that it is ready to receive data.
- **CTS and RTS combined:** Both signals are used to provide full bidirectional flow control, offering the highest level of protection against buffer overrun.

The selected flow control mode is determined from the user configuration structure and applied accordingly.

Programming the CR3 Register

After the flow control configuration is prepared, the value is written into the CR3 register. This activates the selected hardware flow control behavior in the USART peripheral.

Once programmed, the USART automatically manages data transmission and reception based on the state of the CTS and RTS signals, without requiring software intervention.

```
/****** Configuration of BRR *****/  
// Calculate and Set Baud Rate  
USART_SetBaudRate(pUSARTHandle->pUSARTx, pUSARTHandle->USART_Config.USART_Baud);  
  
// Enable USART Peripheral (UE Bit)  
pUSARTHandle->pUSARTx->CR1 |= USART_CR1_UE;
```

Baud Rate Register (BRR) Configuration and USART Enable

This step completes the USART initialization process by configuring the **Baud Rate Register (BRR)** and enabling the USART peripheral. At this stage, all communication parameters have been defined, and the USART is prepared for active data transmission and reception.

Baud Rate Configuration

The baud rate is configured by calculating the appropriate divider value based on the peripheral clock frequency and the user-defined baud rate. This calculation ensures that the USART generates precise bit timing that matches the communication speed expected by the connected device.

By programming the BRR after all control registers have been configured, the USART timing parameters are applied in a stable and predictable manner. Accurate baud rate configuration is essential to avoid framing errors, data corruption, or loss of synchronization between communicating devices.

Enabling the USART Peripheral

After the baud rate has been set, the USART peripheral is enabled by activating its internal control logic. This action transitions the USART from a configuration state to an operational state.

Once enabled, the USART begins monitoring its input and output lines and is ready to transmit and receive data according to the previously configured settings. No further configuration changes should be made while the USART is active, as doing so may disrupt ongoing communication.

```

void USART_SendData(USART_Handle_t *pUSARTHandle, uint8_t *pTxBuffer, uint32_t Len)
{
    uint16_t *pdata;
    // Loop over all bytes to send
    for(uint32_t i = 0 ; i < Len; i++)
    {
        // Wait until TXE (Transmit Empty) flag is set
        // This indicates DR register is ready for new data
        while(! (pUSARTHandle->pUSARTx->SR & USART_FLAG_TXE));

        // Start transmission
        // Check for 9-bit mode
        if(pUSARTHandle->USART_Config.USART_WordLength == USART_WORDLEN_9BITS)
        {
            // If 9-bit, load 2 bytes from buffer as 16-bit
            pdata = (uint16_t*) pTxBuffer;
            // Mask to 9 bits
            pUSARTHandle->pUSARTx->DR = (*pdata & (uint16_t)0x01FF);

            // If parity disabled, we consume 2 bytes of buffer for 9 bits of payload
            if(pUSARTHandle->USART_Config.USART_ParityControl == USART_PARITY_DISABLE)
            {
                pTxBuffer++;
                pTxBuffer++;
            }
            else
            {
                // Parity enabled, 9th bit is parity, so consume 1 byte
                pTxBuffer++;
            }
        }
        else
        {
            // 8-bit mode
            pUSARTHandle->pUSARTx->DR = (*pTxBuffer & (uint8_t)0xFF);
            pTxBuffer++; // Increment buffer pointer
        }
    }

    // Wait till TC (Transmission Complete) flag is set
    // This ensures the last frame is physically out of the shift register
    while( ! (pUSARTHandle->pUSARTx->SR & USART_FLAG_TC));
}

```

Blocking USART Data Transmission

This step implements a **blocking-mode USART data transmission routine**, responsible for sending a sequence of data bytes from memory to the USART transmit hardware. The function ensures correct synchronization with the USART status flags and supports both **8-bit and 9-bit word length configurations**, with optional parity handling.

Transmit Loop Initialization

The function iterates over the transmit buffer for the specified data length. Each iteration corresponds to the transmission of one USART data frame. This design ensures that data is sent sequentially and in the correct order as stored in memory.

Transmit Buffer Readiness Check (TXE)

Before transmitting each data unit, the function waits for the **Transmit Data Register Empty (TXE)** flag. This flag indicates that the USART data register is ready to accept new data and that the previous data has already been transferred to the internal shift register.

Waiting for TXE guarantees that no data is overwritten and that each frame is transmitted correctly.

Handling 9-Bit Word Length Mode

When the USART is configured for **9-bit word length**, the data payload spans more than a single byte. In this case, two bytes from the transmit buffer are combined and treated as a single 9-bit data unit.

Only the lower 9 bits are transmitted, as required by the USART hardware. The handling of the transmit buffer pointer depends on the parity configuration:

- **Parity disabled:** All 9 bits represent payload data, so two bytes are consumed from the buffer.
- **Parity enabled:** The 9th bit is reserved for parity, so only one byte of payload data is consumed.

This conditional handling ensures correct data alignment and prevents buffer misinterpretation.

Handling 8-Bit Word Length Mode

When the USART operates in **8-bit mode**, each iteration transmits a single byte from the buffer. The data is masked to 8 bits and written to the USART data register.

After transmission, the buffer pointer is incremented to move to the next byte.

Completion of Transmission

After all data frames have been written to the USART data register, the function waits for the **Transmission Complete (TC)** flag. This flag indicates that the final data frame has fully exited the shift register and has been physically transmitted on the serial line.

Waiting for TC ensures that the transmission process is completely finished before the function returns, preventing premature termination that could truncate the last data frame.

```

void USART_ReceiveData(USART_Handle_t *pUSARTHandle, uint8_t *pRxBuffer, uint32_t Len)
{
    // Loop to receive bytes
    for(uint32_t i = 0 ; i < Len; i++)
    {
        // Wait until RXNE (Read Not Empty) flag is set
        // This indicates data is ready to be read from DR
        while(! (pUSARTHandle->pUSARTx->SR & USART_FLAG_RXNE));

        // Check for 9-bit mode
        if(pUSARTHandle->USART_Config.USART_WordLength == USART_WORDLEN_9BITS)
        {
            if(pUSARTHandle->USART_Config.USART_ParityControl == USART_PARITY_DISABLE)
            {
                // Read 9 bits
                *((uint16_t*) pRxBuffer) = (pUSARTHandle->pUSARTx->DR & (uint16_t)0x01FF);
                pRxBuffer++;
                pRxBuffer++;
            }
            else
            {
                // Read 8 bits (Parity excluded)
                *pRxBuffer = (pUSARTHandle->pUSARTx->DR & (uint8_t)0xFF);
                pRxBuffer++;
            }
        }
        else
        {
            // 8-bit mode
            if(pUSARTHandle->USART_Config.USART_ParityControl == USART_PARITY_DISABLE)
            {
                // Read 8 bits
                *pRxBuffer = (uint8_t) (pUSARTHandle->pUSARTx->DR & (uint8_t)0xFF);
            }
            else
            {
                // Read 7 bits (Parity bit is the 8th)
                *pRxBuffer = (uint8_t) (pUSARTHandle->pUSARTx->DR & (uint8_t)0x7F);
            }
            pRxBuffer++;
        }
    }
}

```

Blocking USART Data Reception

This step implements a **blocking-mode USART data reception routine**, responsible for receiving a fixed number of data frames from the USART peripheral and storing them into a user-provided memory buffer. The function supports **8-bit and 9-bit word lengths**, with correct handling of parity configurations.

Receive Loop Initialization

The function iterates for the specified receive length, with each loop iteration corresponding to the reception of one USART data frame. This ensures deterministic reception behavior and allows precise control over how many data units are read from the serial interface.

Receive Data Availability Check (RXNE)

Before reading any data, the function waits for the **Receive Data Register Not Empty (RXNE)** flag. This flag indicates that a complete data frame has been received and is ready to be read from the data register.

Waiting for RXNE ensures that:

- Data is not read prematurely
- No invalid or incomplete data is processed
- The receive operation remains synchronized with the USART hardware

This blocking behavior guarantees correctness at the cost of CPU waiting time, which is acceptable for low-level or configuration-driven communication.

Handling 9-Bit Word Length Mode

When the USART is configured for **9-bit word length**, the received data frame may contain either:

- **9 bits of payload data** (parity disabled), or
- **8 bits of payload data plus 1 parity bit** (parity enabled)

If parity is disabled, all 9 bits are considered valid data and are read from the data register. Two bytes of memory are consumed to store the received value correctly.

If parity is enabled, the ninth bit is reserved for parity checking and is excluded from the received data. In this case, only the lower 8 bits are stored in the receive buffer.

This conditional handling ensures correct interpretation of received data and prevents parity bits from being mistakenly treated as payload.

Handling 8-Bit Word Length Mode

When operating in **8-bit word length mode**, the data register contains either:

- **8 bits of payload data** (parity disabled), or
- **7 bits of payload data plus 1 parity bit** (parity enabled)

If parity is disabled, all 8 bits are read and stored directly.

If parity is enabled, the most significant bit is used for parity checking and is excluded from the received data. In this case, only the lower 7 bits are stored in the receive buffer.

This behavior ensures compatibility with standard USART framing rules and maintains data integrity across different configurations.

Buffer Management

After each successful data reception, the receive buffer pointer is incremented appropriately. This allows incoming data frames to be stored sequentially in memory, preserving their order and alignment.

Correct buffer management is critical for higher-level protocols that rely on ordered data streams.

```
// Stubs for Interrupt Driven Features (Placeholder)
// These functions are defined but not implemented for this blocking-only driver
uint8_t USART_SendDataIT(USART_Handle_t *pUSARTHandle, uint8_t *pTxBuffer, uint32_t Len) { return 0; }
uint8_t USART_ReceiveDataIT(USART_Handle_t *pUSARTHandle, uint8_t *pRxBuffer, uint32_t Len) { return 0; }
void USART_IRQInterruptConfig(uint8_t IRQNumber, uint8_t EnOrDi) {}
void USART_IRQPriorityConfig(uint8_t IRQNumber, uint32_t IRQPriority) {}
void USART_IRQHandling(USART_Handle_t *pUSARTHandle) {}
void USART_ApplicationEventCallback(USART_Handle_t *pUSARTHandle, uint8_t ApEv) {}
void USART_PeripheralControl(USART_RegDef_t *pUSARTx, uint8_t EnOrDi) {}
uint8_t USART_GetFlagStatus(USART_RegDef_t *pUSARTx, uint8_t StatusFlagName) { return 0; }
void USART_ClearFlag(USART_RegDef_t *pUSARTx, uint16_t StatusFlagName) {}
```

Interrupt-Driven USART API Placeholders

This step defines a set of **interrupt-driven USART function stubs**, which act as placeholders for future enhancements. Although the current driver operates exclusively in **blocking mode**, these functions establish a clear interface for extending the driver to support **interrupt-based communication** without altering application-level code.

Purpose of Interrupt-Based API Stubs

In embedded driver design, it is common to separate the **driver interface** from the **driver implementation**. By declaring interrupt-based APIs in advance, the driver maintains a consistent and scalable structure.

These stubs allow:

- Seamless future migration from blocking mode to interrupt-driven mode
- Compatibility with higher-level application code that expects these APIs
- Cleaner architecture and long-term maintainability

At present, these functions do not perform any operation and either return default values or contain empty bodies.

Interrupt-Based Data Transmission and Reception

The interrupt-based send and receive functions are declared to support **non-blocking USART communication**, where data transfer occurs asynchronously in the background while the CPU performs other tasks.

In a fully implemented version:

- Transmission would be triggered by TXE or TC interrupts
- Reception would be driven by RXNE interrupts
- Data buffers and transfer states would be managed through interrupts

Currently, these functions return default values to indicate that interrupt-driven transfer is not active.

Interrupt Configuration and Priority Management

The interrupt configuration and priority functions are placeholders for enabling and configuring USART-related interrupts at the NVIC level.

In a complete implementation, these functions would:

- Enable or disable USART interrupts
- Assign interrupt priority levels
- Control system responsiveness and interrupt nesting behavior

Including these stubs ensures that the driver interface already supports advanced interrupt management features.

USART Interrupt Handling Routine

The USART interrupt handling function is defined as a placeholder for processing USART interrupt events. In an interrupt-driven design, this function would:

- Handle transmit-complete events
- Handle receive-ready events
- Update internal driver state
- Invoke application callbacks

At this stage, the function is intentionally left empty, as the driver operates in blocking mode.

Application Event Callback Mechanism

The application event callback function defines a standardized way for the driver to notify the application about USART events, such as:

- Transmission completion
- Reception completion
- Error conditions

Although unused in the current implementation, this callback mechanism provides a clean separation between driver logic and application behavior, supporting event-driven system design.

Peripheral Control and Status Functions

Additional stubs are provided for:

- Peripheral enable/disable control
- Status flag querying
- Status flag clearing

These functions form part of a complete USART driver API but are not required for the current blocking-only implementation. Including them ensures interface completeness and future extensibility.

```
// =====
// LEGACY SUPPORT IMPLEMENTATION
// =====

// Legacy Function: Initialize UART2 with fixed settings (115200 8N1)
void UART2_Init(void) {
    ENABLE_GPIOA(); // Enable GPIOA Clock

    // GPIO Config: PA2(TX), PA3(RX) -> AF7 (USART2)
    // MODER: 10 (AF) for pin 2, 3 -> (2U << 4) | (2U << 6)
    GPIOA->MODER |= (2U << 4) | (2U << 6);
    // AFRL: Set AF7 (0111) for Pins 2 and 3
    GPIOA->AFRL &= ~((0xFU << 8) | (0xFU << 12));
    GPIOA->AFRL |= (7U << 8) | (7U << 12);

    // Create a temporary handle to configure USART2
    USART_Handle_t usart2_handle;

    usart2_handle.pUSARTx = USART2;
    usart2_handle.USART_Config.USART_Mode = USART_MODE_TXRX;
    usart2_handle.USART_Config.USART_Baud = USART_STD_BAUD_115200;
    usart2_handle.USART_Config.USART_NoOfStopBits = USART_STOPBITS_1;
    usart2_handle.USART_Config.USART_WordLength = USART_WORDLEN_8BITS;
    usart2_handle.USART_Config.USART_ParityControl = USART_PARITY_DISABLE;
    usart2_handle.USART_Config.USART_HwFlowControl = USART_HW_FLOW_CTRL_NONE;

    // Call generic init
    USART_Init(&usart2_handle);
}
```

Legacy UART2 Initialization with Fixed Configuration

This step implements a **legacy support function** that initializes **UART2** using a predefined, fixed configuration. The function is designed to provide a simple and quick way to enable UART2 communication without relying on the generic, configurable USART driver framework.

This approach is commonly used for **debug output, console logging, or backward compatibility** in embedded systems.

Enabling GPIO Clock for UART2 Pins

The clock for **GPIO Port A** is enabled to allow configuration of the UART2 transmit and receive pins. UART2 signals are mapped to **PA2 (TX)** and **PA3 (RX)**, and enabling the GPIO clock is mandatory before modifying their configuration registers.

This step ensures that the GPIO hardware controlling the UART pins is powered and accessible.

Configuring GPIO Pins for UART Alternate Function

Pins PA2 and PA3 are configured in **alternate function mode**, transferring control of these pins from general-purpose GPIO logic to the UART2 peripheral.

The alternate function mapping is set to **AF7**, which internally connects PA2 and PA3 to the UART2 transmit and receive signals. This mapping ensures that serial data transmitted by UART2 appears on PA2 and incoming data on PA3 is correctly routed into the UART2 receiver.

This configuration establishes the physical interface required for UART communication.

Creation of a Temporary USART Handle

A temporary **USART handle structure** is created to configure UART2 using the existing generic USART initialization logic. This handle acts as a container for all configuration parameters related to the UART peripheral.

Using a handle-based approach maintains consistency with the overall driver design while allowing this legacy function to remain simple and self-contained.

Fixed UART2 Configuration Parameters

UART2 is configured with a fixed and widely used serial communication format:

- **Full-duplex mode (TX and RX enabled)** allows simultaneous transmission and reception.
- **Baud rate of 115200** provides a good balance between speed and reliability, commonly used for debugging and console interfaces.
- **1 stop bit** ensures standard frame termination.
- **8-bit word length** defines the size of each data frame.
- **Parity disabled** maximizes data throughput when error detection is not required.
- **No hardware flow control** simplifies wiring and is sufficient for low to moderate data rates.

These settings correspond to the commonly known **115200 8N1** configuration, which is widely supported by serial terminals and debugging tools.

```
// Legacy Function: Send a string via UART2
void UART2_SendString(char *string) {
    while(*string) {
        USART_Handle_t handle;
        handle.pUSARTx = USART2;
        handle.USART_Config.USART_WordLength = USART_WORDLEN_8BITS;
        handle.USART_Config.USART_ParityControl = USART_PARITY_DISABLE;
        // Send single byte created from the string pointer
        USART_SendData(&handle, (uint8_t*)string, 1);
        string++;
    }
}
```

Legacy UART2 String Transmission Function

This step implements a **legacy support function** for transmitting a **null-terminated character string** over **UART2**. The function sends the string sequentially, one character at a time, using the existing blocking USART transmission routine.

This design is commonly used for **debug messages, logging output, and console communication** in embedded systems.

Character-by-Character Transmission Approach

The function processes the input string in a loop that continues until the null terminator is reached. Each iteration handles the transmission of a single character. This approach ensures compatibility with standard C-style strings and allows simple, predictable serial output.

By transmitting characters individually, the function avoids the need for additional buffering or string length calculation.

Temporary USART Handle Creation

For each character transmission, a temporary **USART handle** is created and configured specifically for UART2 operation. The handle defines the essential communication parameters required for transmission, such as:

- UART2 peripheral selection
- 8-bit word length
- Parity disabled

These settings are consistent with the fixed UART2 configuration established during legacy initialization and ensure that each character is transmitted using the correct frame format.

Reuse of Generic Send Function

The actual data transmission is performed by calling the generic **blocking USART send routine**. By reusing this lower-level function, the legacy string transmission routine benefits from:

- Proper synchronization with USART status flags
- Correct handling of data register readiness
- Guaranteed completion of each transmitted character

This reuse improves code reliability and avoids duplication of low-level transmission logic.

Pointer Advancement and String Progression

After each character is transmitted, the string pointer is incremented to advance to the next character. This continues until the end of the string is reached, ensuring that the entire message is transmitted in the correct order.

```
// Legacy Function: Send a number via UART2 (Integer to ASCII)
void UART2_SendNumber(int number) {
    char buf[12]; // Buffer for conversion
    int i = 0;

    // Handle specific cases
    if(number == 0) { UART2_SendString("0"); return; }
    if(number < 0) { UART2_SendString("-"); number = -number; }

    // Convert to reverse string
    while(number > 0) { buf[i++] = (number % 10) + '0'; number /= 10; }

    USART_Handle_t handle;
    handle.pUSARTx = USART2;
    handle.USART_Config.USART_WordLength = USART_WORDLEN_8BITS;
    handle.USART_Config.USART_ParityControl = USART_PARITY_DISABLE;

    // Send in correct order
    while(--i >= 0) {
        USART_SendData(&handle, (uint8_t*)&buf[i], 1);
    }
}
```

Legacy UART2 Integer Transmission Function (Integer-to-ASCII Conversion)

This step implements a **legacy support function** that transmits an **integer value** over **UART2** by converting it into its ASCII string representation. The function enables numerical data to be sent through a serial terminal, which is especially useful for **debugging, monitoring, and diagnostic output** in embedded systems.

Buffer Allocation for Conversion

A fixed-size character buffer is allocated to store the ASCII representation of the integer. The buffer size is chosen to safely accommodate typical integer values, including the sign and all decimal digits.

An index variable is used to track the number of digits generated during the conversion process.

Handling Special Numeric Cases

Before performing the general conversion, special cases are handled explicitly:

- If the number is **zero**, the function immediately transmits the character '0' and exits. This avoids unnecessary processing and ensures correct output.
- If the number is **negative**, a minus sign is transmitted first, and the number is converted to its positive equivalent. This ensures that the sign is preserved while simplifying digit extraction.

Handling these cases upfront improves clarity and correctness of the conversion logic.

Integer to ASCII Conversion (Reverse Order)

The integer is converted into ASCII characters by repeatedly extracting the least significant digit. Each digit is converted to its ASCII equivalent and stored sequentially in the buffer.

This process naturally generates the digits in **reverse order**, because the least significant digit is processed first. The conversion continues until all digits of the number have been extracted.

Temporary USART Handle Configuration

A temporary USART handle is created and configured for **UART2 operation**. The handle specifies:

- UART2 as the target peripheral
- 8-bit word length
- Parity disabled

These settings match the fixed UART2 configuration used in other legacy functions, ensuring consistency across all legacy UART operations.

Transmission in Correct Order

After the conversion is complete, the buffer is traversed in reverse order so that the most significant digit is transmitted first. Each digit character is sent individually using the generic blocking USART send routine.

This step ensures that the integer value appears correctly formatted when displayed on the receiving terminal

```
// Legacy Function: Read a single char from UART2
char UART2_GetChar(void) {
    uint8_t c;
    USART_Handle_t handle;
    handle.pUSARTx = USART2;
    handle.USART_Config.USART_WordLength = USART_WORDLEN_8BITS;
    handle.USART_Config.USART_ParityControl = USART_PARITY_DISABLE;

    USART_ReceiveData(&handle, &c, 1);
    return (char)c;
}
```

Legacy UART2 Single-Character Reception Function

This step implements a **legacy support function** that receives a **single character** from **UART2** using a blocking reception mechanism. The function provides a simple and direct interface for reading character input from a serial terminal or external device.

Purpose of Single-Character Reception

Single-character reception is commonly used in embedded systems for:

- Command-line interfaces
- Menu-driven user input
- Debugging interactions
- Simple control commands

By isolating character reception into a dedicated function, the design simplifies application-level code and improves readability.

Temporary USART Handle Configuration

A temporary USART handle is created and configured for UART2 operation. The handle specifies:

- UART2 as the active peripheral
- 8-bit word length
- Parity disabled

These settings ensure compatibility with the fixed UART2 configuration used throughout the legacy support layer.

Blocking Data Reception

The function invokes the generic blocking USART receive routine to read exactly one data unit. The blocking behavior ensures that the function waits until a character is fully received before returning.

This approach guarantees data correctness and simplifies control flow, which is particularly useful in low-speed or user-interactive communication scenarios.

Data Return to Caller

Once the character is received, it is returned to the caller as a standard character type. This allows the received data to be easily processed, compared, or echoed by higher-level application logic.

```
// Legacy Function: Send a single char via UART2
void UART2_SendChar(char c) {
    USART_Handle_t handle;
    handle.pUSARTx = USART2;
    handle.USART_Config.USART_WordLength = USART_WORDLEN_8BITS;
    handle.USART_Config.USART_ParityControl = USART_PARITY_DISABLE;

    USART_SendData(&handle, (uint8_t*)&c, 1);
}
```

Legacy UART2 Single-Character Transmission Function

This step implements a **legacy support function** that transmits a **single character** over **UART2** using a blocking transmission mechanism. The function provides a minimal and efficient interface for sending individual characters to a serial terminal or external device.

Purpose of Single-Character Transmission

Single-character transmission is commonly required for:

- Echoing received input
- Sending prompt characters

- Debug indicators
- Character-based protocols

Providing a dedicated function for this purpose simplifies application code and improves clarity, especially in interactive or low-level debugging scenarios.

Temporary USART Handle Configuration

A temporary USART handle is created and configured for UART2 operation. The handle specifies:

- UART2 as the active peripheral
- 8-bit word length
- Parity disabled

These parameters are consistent with the fixed UART2 configuration used throughout the legacy support layer, ensuring uniform communication behavior.

Blocking Data Transmission

The function invokes the generic blocking USART send routine to transmit exactly one data unit. The blocking behavior ensures that the character is fully transferred to the USART hardware before the function returns.

This guarantees that the character is reliably transmitted on the serial line and avoids partial or corrupted output.

Design Simplicity and Reusability

By reusing the generic USART transmission function, this legacy routine avoids duplicating low-level transmission logic. Any improvements or fixes made to the core send function automatically apply to this legacy interface as well.

Timer driver.c source file Explanations

```
void TIM2_Init(void) {  
    // 1. Enable Clock for TIM2 Peripheral (APB1 Bus)  
    ENABLE_TIM2();  
  
    // 2. Configure Time Base  
    // Timer Clock = PCLK1 = 16MHz (assuming default HSI/APB1 prescalers)  
    // Target Frequency = 8000 Hz  
  
    // PSC (Prescaler): Divide clock by PSC+1  
    // 0 means no division, count at full 16MHz  
    TIM2->PSC = TIM2_PSC_VALUE;  
  
    // ARR (Auto-Reload Register): Counter counts up to ARR then resets.  
    // This defines the period of the timer.  
    // 8000Hz = 16MHz / (1999 + 1)  
    TIM2->ARR = TIM2_ARR_VALUE;  
  
    // 3. Configure Trigger Output (TRGO)  
    // CR2 (Control Register 2) MMS Bits (Master Mode Selection)  
    // We want TRGO signal to pulse on Update Event (UEV - Counter Overflow)  
    // This pulse is internally connected to ADC Trigger.  
  
    // Clear MMS bits (4-6) first then set  
    TIM2->CR2 &= ~(7U << 4);  
    // Set MMS = 010 (Update)  
    TIM2->CR2 |= TIM_CR2_MMS_UPDATE;  
  
    // 4. Enable Timer  
    // Set CEN (Counter Enable) bit in CR1  
    TIM2->CR1 |= TIM_CR1_CEN;  
}
```

TIM2 Initialization for Periodic ADC Trigger Generation

This step configures **Timer 2 (TIM2)** to operate as a **precise periodic time base** and to generate a **trigger output (TRGO)** for synchronized ADC sampling. The timer is configured to produce an update event at a fixed frequency, which is internally routed to the ADC as a hardware trigger source.

Enabling the TIM2 Peripheral Clock

The clock for the TIM2 peripheral is enabled through the APB1 bus. Enabling the peripheral clock is mandatory before accessing or modifying any timer registers. This step powers the internal timer logic and allows subsequent configuration to take effect.

Time Base Configuration

The timer is configured to use the **APB1 peripheral clock**, which operates at **16 MHz** under the default clock configuration. The timer time base is defined using two parameters: the prescaler and the auto-reload value.

The prescaler is configured such that no clock division is applied. This allows the timer counter to increment at the full peripheral clock frequency.

The auto-reload register is configured to define the timer period. When the counter reaches this value, it resets and generates an update event. The selected value results in a timer update frequency of **8 kHz**, meaning the timer generates one update event every **125 microseconds**.

This configuration establishes a stable and deterministic timing source.

Trigger Output (TRGO) Configuration

The timer is configured to generate a **trigger output signal (TRGO)** on each update event. The master mode selection is set so that the update event (counter overflow) produces a trigger pulse.

This trigger output is internally connected to the ADC trigger input. As a result, each timer overflow causes the ADC to start a conversion sequence. This mechanism ensures that ADC sampling occurs at precisely controlled and evenly spaced intervals.

Using a hardware timer trigger eliminates software timing jitter and provides consistent sampling frequency, which is essential for accurate signal analysis and digital signal processing.

Enabling the Timer

After completing all configuration steps, the timer is enabled by activating its counter. Once enabled, TIM2 begins counting continuously and generating update events at the configured frequency.

At this stage, TIM2 operates autonomously and serves as a reliable timing backbone for the ADC data acquisition system.

Energy meter.c source file explanations

```
#include "energy_meter.h"    // Include application header
#include "adc_dma_driver.h"  // Include ADC/DMA driver for data acquisition
#include "uart_driver.h"     // Include UART driver for data logging
#include "i2c_driver.h"      // Include I2C driver for display communication
#include "timer_driver.h"    // Include Timer driver for periodic sampling
#include "ssd1306.h"         // Include OLED driver for display output
#include <math.h>             // Include math library for sqrtf, fabs
#include <stdlib.h>           // Include standard library
#include <string.h>           // Include string manipulation library
```

Inclusion of Application and Driver Header Files

This step includes all required **application-level and peripheral driver header files** necessary for building the complete embedded system. These header files define interfaces, data structures, macros, and function prototypes used throughout the project, ensuring modularity, clarity, and maintainability.

Application-Level Header Inclusion

The main application header file is included to provide access to **system-wide definitions, global variables, and application logic** specific to the energy meter project. This header typically coordinates interactions between different subsystems such as data acquisition, communication, and display.

Including the application header establishes the overall project context and enables integration of all functional modules.

ADC and DMA Driver Inclusion

The ADC/DMA driver header is included to support **high-speed analog data acquisition** using hardware-assisted data transfer. This driver provides interfaces for configuring the ADC, initializing DMA streams, and managing continuous sampling operations.

Including this header allows the application to acquire voltage and current signals efficiently with minimal CPU overhead, which is critical for real-time energy measurement and signal processing.

UART Driver Inclusion

The UART driver header enables **serial communication** for data logging, debugging, and external interfacing. This driver supports both generic USART functionality and legacy UART2 routines used for console output and user interaction.

Including this header allows the application to transmit measured data, diagnostic messages, and system status information over a serial interface.

I2C Driver Inclusion

The I2C driver header provides support for **inter-integrated circuit (I2C) communication**, which is used to interface with external peripherals such as display modules and sensors.

In this project, I2C communication is primarily used for controlling the OLED display, enabling real-time visualization of measured electrical parameters.

Timer Driver Inclusion

The timer driver header enables access to **hardware timer configuration and control functions**. Timers are used to generate precise periodic events, such as triggering ADC conversions at fixed sampling frequencies.

Including this header ensures accurate time-based synchronization between data acquisition and processing modules.

OLED Display Driver Inclusion

The OLED driver header provides functions for initializing and controlling the **SSD1306 OLED display**. This allows the application to render numerical data, system status, and measurement results directly on the display.

Including this header integrates the user interface component of the system.

Standard Library Inclusions

Several standard C library headers are included to support common operations:

- The mathematics library provides access to mathematical functions required for signal processing and calculations.
- The standard library header supports general-purpose utilities such as memory management and type conversions.
- The string manipulation library enables handling of character arrays, formatting output, and managing textual data.

These libraries complement the embedded drivers by enabling higher-level data processing and presentation logic.

```
// --- CONSTANTS ---
#define BUF_LEN          128U          // ADC Buffer Size (Power of 2 ideal for halves)
#define V_OFFSET         2065         // Voltage Sensor DC Offset (calibrated value)
#define I_OFFSET         2045         // Current Sensor DC Offset (calibrated value)
#define SAMPLES_PER_SEC  8000         // Expected Sampling Rate in Hz
#define NOISE_THRES_V    20.0f        // Voltage Noise Threshold below which V=0
#define NOISE_THRES_I    0.05f        // Current Noise Threshold below which I=0
#define ZERO_CROSS_THRES 100          // Zero Crossing Hysteresis threshold in ADC counts
```

Definition of System Constants and Calibration Parameters

This step defines a set of **global constants** that control buffer sizing, sampling behavior, sensor calibration, noise filtering, and signal processing thresholds. These constants play a critical role in ensuring accurate measurement, stable operation, and reliable digital signal processing within the energy metering system.

ADC Buffer Length Definition

The buffer length constant defines the size of the memory buffer used to store ADC samples collected via DMA. The chosen value is a **power of two**, which is ideal for efficient signal processing operations such as buffer halving, averaging, and FFT-based analysis.

A properly sized buffer ensures:

- Continuous data acquisition without overflow
 - Efficient block-based processing
 - Predictable memory usage
-

Voltage Sensor DC Offset Calibration

The voltage offset constant represents the **DC bias value** of the voltage sensing circuit as observed by the ADC. In AC measurement systems, sensor outputs are typically biased around mid-scale to allow measurement of both positive and negative signal swings.

This calibrated offset is subtracted from raw ADC readings to recover the true AC waveform centered around zero volts.

Current Sensor DC Offset Calibration

Similarly, the current offset constant represents the DC bias introduced by the current sensing hardware. Accurate current measurement requires removal of this offset to prevent artificial DC components from affecting RMS calculations and power computation.

Separating voltage and current offsets allows independent calibration of each sensing channel, improving overall measurement accuracy.

Sampling Rate Definition

The sampling rate constant defines the **expected ADC sampling frequency**, which is determined by the timer-triggered ADC configuration. This value is used throughout the application for timing calculations, signal analysis, and frequency-domain processing.

A fixed and known sampling rate is essential for:

- Accurate RMS and power calculations
 - Reliable zero-cross detection
 - Consistent digital filtering behavior
-

Voltage Noise Threshold

The voltage noise threshold defines a minimum magnitude below which voltage samples are treated as zero. This helps suppress low-level noise when no meaningful signal is present.

Applying this threshold:

- Prevents false RMS readings due to noise
- Improves stability of zero-cross detection
- Reduces numerical drift in power calculations

Current Noise Threshold

The current noise threshold performs a similar function for the current measurement channel. Any current sample below this threshold is considered noise and ignored.

This is especially important when measuring light loads, where sensor noise could otherwise appear as false current flow.

Zero-Crossing Hysteresis Threshold

The zero-crossing threshold defines a hysteresis window around zero to ensure robust detection of signal crossings. Instead of reacting to every small fluctuation near zero, the algorithm waits until the signal crosses a defined boundary.

This hysteresis:

- Prevents false multiple zero-cross events
- Improves frequency and phase measurement accuracy
- Stabilizes waveform segmentation for DSP operations

```
// --- CALIBRATION FACTORS ---  
static const float CAL_V = 0.727f;    // Voltage calibration multiplier to get Volts  
static const float CAL_I = 0.0136f;   // Current calibration multiplier to get Amps
```

Definition of Voltage and Current Calibration Factors

This step defines the **calibration multipliers** used to convert raw ADC sample values into **real-world electrical units**, namely volts and amperes. These factors bridge the gap between digital measurements and physical quantities, enabling meaningful interpretation of sensed data.

Voltage Calibration Factor

The voltage calibration factor represents a **scaling multiplier** applied to processed voltage samples to obtain values in **volts**. This factor encapsulates the combined effect of:

- Voltage sensor gain
- Signal conditioning circuitry (resistors, amplifiers, dividers)
- ADC reference voltage
- ADC resolution

After offset removal and noise filtering, the raw voltage samples are multiplied by this factor to map ADC counts to actual voltage levels. Proper calibration ensures that displayed and computed voltage values closely match real input voltages.

Current Calibration Factor

The current calibration factor serves a similar role for the current measurement channel. It converts processed ADC samples into **amperes** by accounting for:

- Current sensor sensitivity
- Signal conditioning gain
- ADC scaling characteristics

Accurate current calibration is critical for correct RMS current calculation and for downstream computations such as power, energy, and load analysis.

Importance of Using Floating-Point Constants

The calibration factors are defined as floating-point constants to preserve precision during scaling operations. Electrical measurements often involve fractional values, and using floating-point arithmetic minimizes rounding errors, especially during RMS and power calculations.

Defining these factors as constants ensures consistency across the entire application and prevents accidental modification during runtime.

Calibration Methodology

These calibration factors are typically determined through an **experimental calibration process**, where known reference voltages and currents are applied to the system. The resulting ADC values are measured, and the calibration multipliers are adjusted until the computed output matches the reference values.

Once determined, these factors remain fixed unless the sensing hardware or reference conditions change.

```
// --- BUFFERS ---  
static uint32_t adc_buffer[BUF_LEN]; // DMA destination buffer for raw ADC values (interleaved)
```

ADC DMA Buffer Declaration for Raw Sample Storage

This step declares a **static memory buffer** used as the **destination for ADC conversion results** transferred via Direct Memory Access (DMA). The buffer serves as the primary storage area for raw ADC samples collected during continuous data acquisition.

Purpose of the ADC DMA Buffer

The buffer is designed to receive ADC data automatically, without CPU intervention. Each time the ADC completes a conversion, the DMA controller transfers the resulting digital value directly into

this buffer. This mechanism enables high-speed, low-latency data acquisition while minimizing processor load.

Static Allocation and Memory Stability

The buffer is declared as a static data structure, ensuring that it is allocated in a fixed memory location for the entire lifetime of the application. This is a critical requirement for DMA operation, as the DMA controller relies on stable memory addresses during active transfers.

Static allocation prevents issues related to stack overflow, memory relocation, or buffer invalidation during runtime.

Buffer Length and System Integration

The buffer size is defined using a compile-time constant, allowing easy adjustment of the sampling window without modifying the core logic. The chosen length aligns with the system's sampling rate and processing requirements, enabling efficient block-based signal processing.

Using a power-of-two buffer length further simplifies operations such as buffer splitting, averaging, and frequency-domain analysis.

Interleaved Sample Storage

The buffer stores ADC samples in an **interleaved format**, where samples from multiple ADC channels are placed sequentially in memory. This layout reflects the ADC scan mode configuration and preserves the temporal relationship between voltage and current samples.

Interleaved storage is especially useful for paired signal analysis, as it allows synchronized processing of multiple channels sampled at the same instant.

```
// --- STATIC Prototypes ---
static void Hardware_Init(void);           // Internal function to initialize hardware
static void Accumulate_Data(int32_t start_index); // Internal function to process a batch of data
```

Declaration of Internal Static Function Prototypes

This step declares **static function prototypes** used internally within the module to organize hardware initialization and data processing logic. These functions are intended for **internal use only** and are not exposed outside the source file, enforcing proper modular design.

Purpose of Static Function Prototypes

Declaring functions as static restricts their visibility to the current source file. This encapsulation:

- Prevents unintended access from other modules
- Avoids name collisions across the project
- Improves code readability and maintainability
- Clearly separates internal logic from public APIs

Static prototypes also allow the compiler to validate function usage before their actual definitions appear later in the file.

Hardware Initialization Function

The hardware initialization function prototype represents a centralized routine responsible for initializing all low-level hardware components required by the application. This typically includes:

- Peripheral initialization sequences
- Clock configuration coordination
- GPIO, ADC, DMA, Timer, and communication setup

By grouping hardware setup into a single internal function, the design improves clarity and ensures that system initialization follows a well-defined and repeatable sequence.

Data Accumulation and Processing Function

The data accumulation function prototype represents an internal processing routine that operates on batches of acquired ADC samples. It is responsible for:

- Parsing raw ADC data from the DMA buffer
- Accumulating samples over a defined window
- Preparing data for higher-level calculations such as RMS, power, or energy computation

The use of a start index parameter allows flexible processing of different buffer segments, which is particularly useful when working with circular DMA buffers.

Design Rationale

Separating hardware initialization and data processing into dedicated static functions provides:

- Clear separation of concerns
- Easier debugging and testing
- Improved scalability for future feature additions
- Cleaner and more readable main application logic

This approach aligns with good embedded software engineering practices, especially in systems that combine real-time data acquisition with signal processing.

```
// Internal function to update display and send UART logs
static void Update_Display_And_Log(float v_rms, float i_rms, float active_power, float energy_kwh, float pf, float frequency);
```

Declaration of Internal Display and Logging Update Function

This step declares a **static internal function prototype** responsible for updating the user interface and transmitting measurement data through serial communication. The function is designed to operate at the application layer, using already processed electrical parameters rather than raw sensor data.

Purpose of the Update and Logging Function

The function represents a centralized routine that handles all **output-related tasks** of the system, including:

- Updating the OLED display with real-time measurement values
- Sending formatted measurement logs over UART for monitoring or debugging

By consolidating display and logging functionality into a single internal function, the system ensures consistent presentation and synchronized reporting of measured data.

Input Parameters and Their Meaning

The function accepts multiple floating-point parameters, each representing a **fully processed electrical quantity**:

- RMS voltage: reflects the effective value of the measured AC voltage
- RMS current: represents the effective current flowing through the load
- Active power: indicates real power consumed by the load
- Energy in kilowatt-hours: accumulated energy consumption over time
- Power factor: indicates the phase relationship between voltage and current
- Frequency: represents the measured AC signal frequency

Passing these values as parameters enforces a clear separation between **signal processing logic** and **output handling logic**.

Use of Floating-Point Parameters

Floating-point parameters are used to preserve numerical precision for display and logging. Electrical measurements often require decimal accuracy, especially for quantities such as power factor, frequency, and energy consumption.

This design ensures that output formatting does not degrade the accuracy achieved during computation.

Static Scope and Encapsulation

The function is declared as static, restricting its visibility to the current source file. This encapsulation:

- Prevents unintended external access
- Avoids symbol conflicts across modules
- Clearly marks the function as an internal utility

Only the application logic within the same file can invoke this function, reinforcing a clean modular architecture.

Role in System Execution Flow

This function typically executes after a complete data acquisition and processing cycle. Once voltage, current, power, and energy values are computed, the function is called to present the results to the user and transmit them externally.

This separation ensures that real-time data processing remains independent from slower operations such as display updates and serial communication.

```
// Function to Initialize the Energy Meter Application
void EnergyMeter_Init(void) {
    Hardware_Init(); // Identify and initialize all required peripherals

    // OLED Startup Sequence
    SSD1306_Init();   // Initialize the OLED display controller
    SSD1306_Clear();  // Clear any random garbage from display memory
    SSD1306_PrintCentered(2, "ENERGY METER"); // Print Title centered on line 2
    SSD1306_PrintCentered(4, "STARTING...");  // Print Status centered on line 4
    SSD1306_Update(); // Send buffer to physical display to show text

    UART2_SendString("System Online.\r\n"); // Send boot message via UART
}
```

Energy Meter Application Initialization Routine

This step defines the **top-level initialization function** for the energy meter application. It represents the formal startup sequence executed once after system reset, responsible for preparing hardware resources, initializing the user interface, and confirming system readiness.

Purpose of the Application Initialization Function

The energy meter initialization function acts as the **entry point for application-level setup**. Its role is to:

- Initialize all required hardware peripherals
- Prepare the display interface for user interaction
- Provide startup feedback through serial communication

This function ensures that the system enters a known, stable state before beginning real-time data acquisition and processing.

Hardware Initialization Phase

The first operation performed is a call to the internal hardware initialization routine. This step configures all low-level peripherals required by the application, such as:

- GPIOs
- ADC with DMA
- Timers for periodic triggering
- Communication interfaces (UART and I2C)

By isolating hardware setup into a dedicated routine, the design ensures clarity, reuse, and maintainability.

OLED Display Startup Sequence

After hardware initialization, the OLED display is initialized and prepared for use. This sequence includes:

- Initializing the display controller
- Clearing display memory to remove any undefined or residual data
- Rendering a centered application title
- Displaying a startup status message
- Transferring the prepared display buffer to the physical screen

This provides immediate visual feedback that the system has powered on and entered a valid operational state.

User Feedback and Status Indication

Displaying startup messages serves both functional and diagnostic purposes. It confirms that:

- The display interface is operational
- The application firmware has started successfully

The use of centered text improves readability and gives the system a professional user interface appearance.

UART Boot Message Transmission

After initializing the display, a startup message is transmitted over UART. This message:

- Confirms successful system initialization
- Allows developers or monitoring tools to detect system readiness
- Provides a communication heartbeat during debugging or logging

UART output is especially useful when the display is unavailable or when logging system events externally.

Role in System Execution Flow

This initialization function is typically called once from the main program before entering the main application loop. After its execution:

- All peripherals are configured
- The user interface is active

- The system is ready to begin continuous measurement, processing, and reporting

This structured startup sequence ensures deterministic and reliable system behavior.

```
// Main Application Loop
void EnergyMeter_Run(void) {
    // Check Half Transfer Flag (HTIF4) in DMA2 Stream 0 Interrupt Status Register (LISR)
    // Bit 4 corresponds to Half Transfer for Stream 0
    if ((DMA2->LISR & (1U << 4)) != 0U) {
        DMA2->LIFCR |= (1U << 4); // Clear the Half Transfer Interrupt Flag
        Accumulate_Data(0); // Process the first half of the buffer (indices 0 to BUF_LEN/2 - 1)
    }

    // Check Transfer Complete Flag (TCIF5) in DMA2 Stream 0 Interrupt Status Register (LISR)
    // Bit 5 corresponds to Transfer Complete for Stream 0
    if ((DMA2->LISR & (1U << 5)) != 0U) {
        DMA2->LIFCR |= (1U << 5); // Clear the Transfer Complete Interrupt Flag
        Accumulate_Data((int32_t)(BUF_LEN / 2U)); // Process second half (indices BUF_LEN/2 to BUF_LEN - 1)
    }
}
```

Main Energy Meter Application Execution Loop

This step defines the **main runtime function** of the energy meter application. It implements the continuous execution logic responsible for processing ADC data as it becomes available through DMA, ensuring real-time signal handling without blocking the processor.

Purpose of the Main Application Loop

The main application loop continuously monitors the DMA controller to determine when new ADC data has been transferred into memory. Instead of polling the ADC directly or waiting for blocking conversions, the system relies on **DMA transfer status flags** to trigger data processing at precise intervals.

This design enables:

- Deterministic real-time behavior
- Efficient CPU utilization
- Continuous high-speed sampling

Half-Transfer Interrupt Handling

The first condition checks the **Half Transfer flag** for DMA Stream 0. This flag indicates that the DMA controller has completed transferring the **first half of the ADC buffer**.

When this flag is detected:

- The flag is explicitly cleared to acknowledge the event
- The first half of the buffer is passed to the data accumulation function
- Processing occurs while the DMA continues filling the second half of the buffer

This technique allows overlapping data acquisition and data processing, which is critical for real-time systems.

Transfer Complete Interrupt Handling

The second condition checks the **Transfer Complete flag**, which indicates that the DMA controller has finished transferring the **entire buffer**.

When this flag is detected:

- The flag is cleared to prepare for the next DMA cycle
- The second half of the buffer is processed
- The DMA immediately begins refilling the buffer from the beginning

This creates a continuous circular data flow without data loss.

Double-Buffer Processing Strategy

By dividing the DMA buffer into two halves and processing each half independently, the system achieves a **ping-pong buffering mechanism**. While one half is being processed, the other half is being filled by the DMA controller.

This approach:

- Eliminates the need for large processing delays
 - Prevents overwriting unprocessed samples
 - Maintains precise timing alignment between voltage and current samples
-

Role of the Data Accumulation Function

The data accumulation function is responsible for parsing the raw ADC samples from the specified buffer segment. It performs tasks such as:

- Offset removal
- Noise filtering
- Sample accumulation for RMS and power calculations

Passing a start index allows the same processing logic to be reused for both halves of the buffer.

Non-Blocking System Behavior

The main loop does not contain any delays or blocking calls. It reacts only when DMA status flags indicate new data availability. This ensures that:

- The CPU remains responsive
- Peripheral interrupts are serviced promptly
- Real-time constraints are met

Such a design is well-suited for embedded measurement systems that require continuous sampling and processing.

```
// Internal Hardware Initialization
static void Hardware_Init(void) {
    FPU_CPCR |= (0xFU << 20); // Enable FPU (Floating Point Unit) by setting CP10 and CP11 to Full Access

    I2C1_Init(); // Initialize I2C peripheral for OLED
    UART2_Init(); // Initialize UART peripheral for Logging
    TIM2_Init(); // Initialize Timer for ADC triggering
    ADC_DMA_Init(adc_buffer, BUF_LEN); // Initialize ADC and DMA with the buffer
}
```

Internal Hardware Initialization Routine

This step defines the **internal hardware initialization function**, which performs all low-level system and peripheral setup required before the energy meter application begins normal operation. This function is invoked once during application startup and establishes the hardware foundation for reliable real-time measurement and processing.

Purpose of the Hardware Initialization Function

The hardware initialization routine centralizes all **core peripheral setup** in a single internal function. Its objectives are to:

- Enable processor features required for numerical computation
- Initialize communication interfaces
- Configure timing sources for synchronized sampling
- Prepare the ADC and DMA subsystem for continuous data acquisition

Keeping this logic isolated improves readability, maintainability, and reuse.

Floating Point Unit (FPU) Enablement

The first operation enables the **Floating Point Unit (FPU)** by granting full access to the coprocessor registers responsible for floating-point execution. This step is essential because:

- RMS, power, energy, and frequency calculations rely heavily on floating-point arithmetic
- Hardware FPU execution is significantly faster and more energy-efficient than software emulation
- It ensures deterministic execution time for mathematical operations

Enabling the FPU early guarantees that all subsequent computations use hardware acceleration.

I2C Peripheral Initialization

The I2C interface is initialized to support communication with external peripherals, primarily the OLED display. This configuration prepares:

- GPIO pins for I2C alternate function
- Clock timing and bus parameters
- Internal I2C state machines

This step ensures that the display interface is ready before any user-facing output is attempted.

UART Peripheral Initialization

The UART interface is initialized to enable **serial logging and debugging output**. This allows:

- Transmission of startup messages
- Logging of measurement data
- External monitoring during development and testing

UART initialization provides a reliable communication channel independent of the display.

Timer Initialization for ADC Triggering

A hardware timer is initialized to generate **periodic trigger events** at a precise frequency. These trigger events are internally routed to the ADC, ensuring:

- Fixed and accurate sampling intervals
- Stable sampling frequency for signal processing
- Synchronization between voltage and current samples

Using a timer-based trigger is critical for correct RMS and frequency calculations.

ADC and DMA Initialization

Finally, the ADC and DMA subsystem is initialized with a user-provided buffer and buffer length. This setup enables:

- Continuous ADC conversions in scan mode
- Automatic data transfer from ADC to memory via DMA
- Circular buffering for uninterrupted sampling

The ADC/DMA combination allows high-speed data acquisition with minimal CPU involvement, which is essential for real-time energy monitoring.

Encapsulation and Static Scope

The function is declared as static, restricting its visibility to the current source file. This design choice:

- Prevents unintended external invocation
- Clearly marks the function as an internal system routine
- Reinforces modular architecture principles


```
// Data Processing Function
static void Accumulate_Data(int32_t start_index) {
    // Static variables to persist state between function calls
    static uint64_t acc_v_sq = 0;        // Accumulator for Voltage Squared (for RMS)
    static uint64_t acc_i_sq = 0;        // Accumulator for Current Squared (for RMS)
    static float acc_p_inst = 0.0f;      // Accumulator for Instantaneous Power
    static int32_t sample_count = 0;     // Counter for number of samples processed
    static int32_t last_v_sign = 0;      // Sign of voltage in previous sample (for zero-crossing)
    static int32_t zero_crossings = 0;   // Counter for zero crossings detected
    static float energy_ws = 0.0f;       // Accumulated energy in Watt-Seconds
}
```

Data Accumulation and Signal Processing State Initialization

This step defines the **internal data processing function** and declares a set of **static state variables** used to accumulate, track, and compute electrical parameters from streamed ADC samples. These variables preserve computation state across multiple invocations of the function, enabling continuous real-time processing.

Purpose of the Data Processing Function

The data processing function operates on blocks of ADC samples delivered by the DMA subsystem. Its role is to:

- Accumulate voltage and current samples over time
- Prepare data for RMS, power, energy, and frequency calculations
- Track signal characteristics such as zero crossings
- Maintain continuity across buffer halves in a circular DMA setup

This function is called repeatedly as new data becomes available.

Use of Static Variables for Persistent State

All variables inside this function are declared as static so their values **persist between function calls**. This is essential because:

- DMA provides data in chunks (half-buffer / full-buffer)
- Electrical quantities require accumulation over many samples
- Resetting variables on each call would break continuity

Static storage ensures correct long-term accumulation without global exposure.

Voltage Square Accumulator

The voltage square accumulator stores the sum of squared voltage samples. This is a prerequisite for RMS voltage computation, where the mean of squared samples is calculated before applying the square root.

Using a wide integer type prevents overflow when accumulating a large number of squared samples.

Current Square Accumulator

Similarly, the current square accumulator holds the sum of squared current samples. This enables accurate RMS current computation using the same statistical approach as voltage RMS.

Separating voltage and current accumulators ensures independent and precise measurement of both signals.

Instantaneous Power Accumulator

The instantaneous power accumulator collects the product of voltage and current samples over time. This forms the basis for computing **average (active) power**, which reflects real energy consumption.

Accumulating instantaneous power over a window allows noise averaging and stable power estimation.

Sample Counter

The sample counter tracks the total number of samples processed during the current accumulation window. This value is used to:

- Normalize RMS and power calculations
- Determine when a full measurement cycle is complete
- Synchronize processing intervals

Accurate sample counting is critical for correct scaling of computed values.

Voltage Sign Tracking for Zero Crossing Detection

The previous voltage sign variable stores the polarity of the voltage signal from the last sample. Comparing successive signs enables detection of **zero crossings**, which are used to:

- Measure signal frequency
- Synchronize measurement windows to AC cycles
- Improve RMS and power accuracy

This approach is robust and computationally efficient for sinusoidal signals.

Zero Crossing Counter

The zero crossing counter increments whenever a valid zero crossing is detected. By counting crossings over time and knowing the sampling rate, the system can accurately estimate the AC signal frequency.

This method avoids reliance on external frequency references and adapts naturally to signal variations.

Energy Accumulator

The energy accumulator stores accumulated energy in **watt-seconds**, which is the fundamental unit of energy. Over time, this value can be converted to kilowatt-hours for billing and reporting purposes.

Accumulating energy incrementally ensures long-term accuracy and avoids loss of precision during extended operation.

```
// Calculate the limit index for this processing batch
int32_t limit = start_index + (int32_t)(BUF_LEN / 2U);

// Iterate through the buffer chunk
// Step by 2 because data is interleaved: [V0, I0, V1, I1, ...]
for(int32_t n = start_index; n < limit; n += 2) {
    // Read Raw Voltage and subtract offset to get AC component
    int32_t v = (int32_t)adc_buffer[n] - V_OFFSET;
    // Read Raw Current and subtract offset to get AC component
    int32_t i = (int32_t)adc_buffer[n+1] - I_OFFSET;

    // Accumulate squares for RMS calculation
    acc_v_sq += (uint64_t)(v * v);
    acc_i_sq += (uint64_t)(i * i);

    // Calculate Instantaneous Power: P = V * I.
    // Note: -V * I corrects for sensor polarity in hardware installation
    acc_p_inst += (float)(-(v * i));

    // Frequency Detection Logic (Zero-Crossing)
    // Check if voltage magnitude exceeds hysteresis threshold to avoid noise
    if (abs(v) > ZERO_CROSS_THRES) {
        // Determine current sign: positive (1) or negative (-1)
        int32_t current_sign = (v > 0) ? 1 : -1;
        // Detect sign change (transition from + to - or - to +)
        if((current_sign != last_v_sign) && (last_v_sign != 0)) {
            zero_crossings++; // Increment zero crossing counter
        }
        last_v_sign = current_sign; // Update last sign state
    }
    sample_count++; // Increment total sample counter
}
```

Batch-Wise ADC Sample Processing and Signal Feature Extraction

This step explains how a **specific chunk of the DMA buffer** is processed to extract meaningful electrical information such as RMS values, power, energy contribution, and frequency indicators.

Determination of Processing Window

A limit index is calculated to define the **end boundary of the current processing batch**. Since the DMA buffer is split into two halves (half-transfer and full-transfer processing), only half of the buffer is processed at a time.

This ensures:

- No overlap between DMA writing and CPU reading

- Safe concurrent operation of DMA and data processing
 - Deterministic batch size for calculations
-

Interleaved Data Interpretation

The ADC buffer stores samples in an **interleaved format**, where voltage and current samples alternate sequentially:

- One voltage sample
- Followed immediately by one current sample

The loop advances in steps of two to maintain correct pairing of voltage and current samples from the same sampling instant.

This preserves the **phase relationship** between voltage and current, which is essential for accurate power calculation.

Offset Removal and AC Signal Reconstruction

Each raw ADC sample includes a DC offset introduced by sensor biasing and ADC mid-scale referencing.

To recover the true AC waveform:

- The calibrated voltage offset is subtracted from the voltage sample
- The calibrated current offset is subtracted from the current sample

This centers both signals around zero, allowing correct RMS, power, and frequency analysis.

RMS Accumulation Logic

For RMS computation, each sample is squared and accumulated:

- Squared voltage samples contribute to the voltage RMS accumulator
- Squared current samples contribute to the current RMS accumulator

Accumulating squared values over many samples enables statistical averaging, which reduces noise and improves measurement stability.

Wide accumulators are used to prevent numerical overflow during long accumulation periods.

Instantaneous Power Accumulation

Instantaneous power is calculated conceptually as the product of voltage and current at each sample instant. These values are accumulated over time to estimate **active (real) power**.

A polarity correction is applied to account for sensor orientation or wiring direction, ensuring power sign correctness.

This method captures both magnitude and phase interaction between voltage and current.

Zero-Crossing Based Frequency Detection

Frequency estimation is performed using **zero-crossing detection** on the voltage waveform:

- Only samples whose magnitude exceeds a defined hysteresis threshold are considered
- This prevents false zero crossings caused by noise near zero

The sign of the voltage is tracked across samples:

- A change in sign from positive to negative or vice versa indicates a zero crossing
- Each valid crossing increments a counter

Tracking only sign transitions ensures robustness against amplitude variation.

Sample Count Tracking

A running sample counter is incremented for each processed voltage-current pair. This counter is later used to:

- Normalize RMS calculations
- Compute average power
- Convert accumulated energy into time-based units

Accurate sample counting is critical for all time-domain measurements.

```
// Check if we have accumulated 1 second worth of data (8000 samples)
if (sample_count >= SAMPLES_PER_SEC) {
    // Calculate RMS Voltage: sqrt(mean of squares) * Calibration Factor
    float v_rms = sqrtf(((float)acc_v_sq / (float)sample_count) * CAL_V);
    // Calculate RMS Current: sqrt(mean of squares) * Calibration Factor
    float i_rms = sqrtf(((float)acc_i_sq / (float)sample_count) * CAL_I);

    // Apply Noise Thresholds (Zero-out readings if below noise floor)
    if (v_rms < NOISE_THRES_V) {
        v_rms = 0.0f;
        i_rms = 0.0f; // If voltage is zero, current implies noise usually
        zero_crossings = 0; // No voltage means no frequency
    }
    if (i_rms < NOISE_THRES_I) {
        i_rms = 0.0f;
    }
}
```

One-Second Window Completion and RMS Computation

This step describes how the system detects the completion of a **fixed one-second measurement window** and converts accumulated raw signal data into meaningful RMS electrical quantities.

Detection of One-Second Measurement Interval

The system continuously counts the number of processed samples. When this count reaches the predefined sampling rate (samples per second), it indicates that **exactly one second of data** has been collected.

Using a one-second window provides:

- Stable RMS and power calculations
- Direct alignment with standard electrical measurement practices
- Simplified frequency estimation and energy integration

This time-based windowing ensures consistency and repeatability of results.

RMS Voltage Calculation Logic

Once one second of samples is accumulated, RMS voltage is computed using the statistical RMS definition:

- The mean of squared voltage samples is calculated
- The square root of this mean yields the RMS value
- A calibration factor is applied to convert from ADC units to volts

This approach accurately reflects the effective value of an AC voltage waveform, regardless of its instantaneous shape.

RMS Current Calculation Logic

RMS current is calculated using the same mathematical approach as voltage:

- Squared current samples are averaged
- The square root of the average is taken
- A current calibration factor converts the result into amperes

Using identical processing for voltage and current ensures symmetry and measurement consistency.

Noise Floor Suppression for Voltage

After RMS voltage calculation, a **noise threshold check** is applied. If the computed voltage RMS is below the defined noise floor:

- Voltage is forced to zero
- Current is also forced to zero
- Zero crossing count is cleared

This prevents:

- False readings when no load is connected
- Accidental frequency detection due to electrical noise

- Phantom power calculations

Voltage is treated as the primary reference signal; if voltage is absent, meaningful power measurement is not possible.

Noise Floor Suppression for Current

A separate noise threshold is applied to RMS current. If the current RMS falls below this threshold:

- Current is forced to zero

This eliminates small sensor noise and leakage effects that would otherwise appear as false load current.

Rationale for Noise Thresholds

Noise thresholds are essential in real-world measurement systems because:

- Sensors and ADCs always introduce small noise
- RMS calculations can amplify low-level noise
- Zero-load conditions must report clean zero values

Applying thresholds improves measurement reliability and user confidence.

RMS Voltage Equation

Let

- $v[n]$ = offset-corrected voltage sample
- N = total number of samples in 1 second
- K_V = voltage calibration factor

Mean Square Voltage

$$V_{\text{mean}} = \frac{1}{N} \sum_{n=0}^{N-1} v[n]^2$$

RMS Voltage

$$V_{\text{RMS}} = \sqrt{V_{\text{mean}}} \times K_V$$

RMS Current Equation

Let

- $i[n]$ = offset-corrected current sample
- K_I = current calibration factor

Mean Square Current

$$I_{\text{mean}} = \frac{1}{N} \sum_{n=0}^{N-1} i[n]^2$$

RMS Current

$$I_{\text{RMS}} = \sqrt{I_{\text{mean}}} \times K_I$$

Noise Threshold Conditions

Voltage Noise Suppression

$$\text{If } V_{\text{RMS}} < V_{\text{noise_threshold}} \Rightarrow \begin{cases} V_{\text{RMS}} = 0 \\ I_{\text{RMS}} = 0 \\ f = 0 \end{cases}$$

Current Noise Suppression

$$\text{If } I_{\text{RMS}} < I_{\text{noise_threshold}} \Rightarrow I_{\text{RMS}} = 0$$


```

// Calculate Active Power: Mean of instantaneous power * Calibration Factors
float active_power = (acc_p_inst / (float)sample_count) * CAL_V * CAL_I;

// Final sanity checks on power
if (i_rms == 0.0f) { active_power = 0.0f; } // No current flow means no power
if (active_power < 0.0f) { active_power = -active_power; } // Absolute value for display

// Calculate Apparent Power: V_rms * I_rms
float apparent_power = v_rms * i_rms;

// Calculate Power Factor: Active Power / Apparent Power
float pf = 0.0f;
if (apparent_power > 0.5f) { // Avoid division by near-zero
    pf = (active_power / apparent_power) * 100.0f; // In percentage
    if (pf > 100.0f) { pf = 100.0f; } // Cap at 100%
}

// Calculate Frequency: Zero Crossings / 2 (since 2 crossings per cycle)
float frequency = (float)zero_crossings / 2.0f;

// Accumulate Energy
energy_ws += active_power; // Since window is 1 sec, Power * 1s = Energy in Joules (Ws)
float energy_kwh = energy_ws / 3600000.0f; // Convert Ws to kWh (1000 * 3600)

// Update the User Interface and Logs
Update_Display_And_Log(v_rms, i_rms, active_power, energy_kwh, pf, frequency);

// Reset accumulators for the next 1-second window
acc_v_sq = 0;
acc_i_sq = 0;
acc_p_inst = 0.0f;
sample_count = 0;
zero_crossings = 0;

```

Active (Real) Power Equation

Let

- $v[n]$ = offset-corrected voltage sample
- $i[n]$ = offset-corrected current sample
- N = number of samples in 1 second
- K_V = voltage calibration factor
- K_I = current calibration factor

Mean Instantaneous Power

$$P_{\text{mean}} = \frac{1}{N} \sum_{n=0}^{N-1} v[n] \cdot i[n]$$

Active Power

$$P_{\text{active}} = P_{\text{mean}} \times K_V \times K_I$$

Sanity Conditions

$$\text{If } I_{\text{RMS}} = 0 \Rightarrow P_{\text{active}} = 0$$

$$P_{\text{active}} = |P_{\text{active}}|$$

(Absolute value ensures positive display regardless of sensor polarity.)

Apparent Power Equation

$$S = V_{\text{RMS}} \times I_{\text{RMS}}$$

Where

- S = apparent power (VA)
-

Power Factor Equation

$$\text{PF} = \frac{P_{\text{active}}}{S}$$

In percentage form:

$$\text{PF}(\%) = \left(\frac{P_{\text{active}}}{S} \right) \times 100$$

Safety Constraints

$$\text{If } S \leq 0 \Rightarrow \text{PF} = 0$$

$$\text{PF} \leq 100\%$$

Frequency Calculation (Zero-Crossing Method)

Let

- Z = number of zero crossings detected in 1 second

Since one AC cycle produces **two zero crossings**:

$$f = \frac{Z}{2}$$

Where

- f = frequency in Hz
-

Energy Accumulation Equation

Energy in Watt-Seconds (Joules)

Since the window is 1 second:

$$E_{\text{Ws}} = E_{\text{Ws}(\text{prev})} + P_{\text{active}} \times 1$$

Energy in Kilowatt-Hours (kWh)

$$E_{\text{kWh}} = \frac{E_{\text{Ws}}}{3,600,000}$$

(1 kWh = 1000 × 3600 Ws)

Measurement Window Reset Logic

After completing one 1-second computation cycle:

$$\begin{aligned}\sum v^2 &\leftarrow 0 \\ \sum i^2 &\leftarrow 0 \\ \sum (v \cdot i) &\leftarrow 0 \\ N &\leftarrow 0 \\ Z &\leftarrow 0\end{aligned}$$

This prepares the system for the next independent measurement window.

```

// Function to update OLED and UART
static void Update_Display_And_Log(float v_rms, float i_rms, float active_power, float energy_kwh, float pf, float frequency) {
    SSD1306_Clear(); // Clear display buffer
    SSD1306_PrintCentered(0, "ENERGY METER"); // Print Header

    // Format Current for display (Integers and Decimals manually for embedded usage)
    int i_int = (int)i_rms;
    int i_dec = (int)((i_rms - (float)i_int) * 100.0f); // 2 decimal places

    // Display Voltage
    SSD1306_SetCursor(8, 2);
    SSD1306_Print("V:"); SSD1306_PrintNumber((int)v_rms);

    // Display Current
    SSD1306_SetCursor(70, 2);
    SSD1306_Print("A:"); SSD1306_PrintNumber(i_int);
    SSD1306_Print("."); if(i_dec<10) { SSD1306_Print("0"); } SSD1306_PrintNumber(i_dec);

    // Format Energy
    int e_int = (int)energy_kwh;
    int e_dec = (int)((energy_kwh - (float)e_int) * 1000.0f); // 3 decimal places

    // Display Power (Watts)
    SSD1306_SetCursor(8, 4);
    SSD1306_Print("W:"); SSD1306_PrintNumber((int)active_power);

    // Display Accumulated Energy (kWh)
    SSD1306_SetCursor(70, 4);
    SSD1306_Print("E:"); SSD1306_PrintNumber(e_int);
    SSD1306_Print(".");
    if(e_dec<100) { SSD1306_Print("0"); } // Zero padding for decimal
    if(e_dec<10) { SSD1306_Print("0"); }
    SSD1306_PrintNumber(e_dec);

    // Display Power Factor
    SSD1306_SetCursor(8, 6);
    SSD1306_Print("PF:");
    if (pf >= 99.9f) { SSD1306_Print("1.00"); }
    else {
        SSD1306_Print("0.");
        if(pf < 10.0f) { SSD1306_Print("0"); }
        SSD1306_PrintNumber((int)pf);
    }
}

// Display Frequency
SSD1306_SetCursor(70, 6);
SSD1306_Print("F:"); SSD1306_PrintNumber((int)frequency);

SSD1306_Update(); // Send buffer to OLED

```

Display Update and UART Logging Function

This step explains the internal function responsible for **presenting computed electrical parameters on the OLED display and updating the user interface in real time**. The function operates on **fully processed measurement values**, not raw ADC data, ensuring a clean separation between signal processing and output presentation.

Purpose of the Display and Logging Function

The primary objectives of this function are to:

- Convert computed floating-point values into display-friendly formats
- Present voltage, current, power, energy, power factor, and frequency
- Maintain a clear and readable layout on the OLED screen
- Update the display periodically after each completed measurement window

This function represents the **final stage of the measurement pipeline**, where numerical results are made visible to the user.

Display Buffer Management

Before writing new data, the display buffer is cleared to remove previous content. This ensures that:

- Old values do not overlap with new readings
- Display artifacts are avoided
- Each update reflects the latest measurement state

The display header is then printed at the top of the screen to identify the application context.

Manual Formatting of Current Measurement

Current is displayed with **two decimal places**, which requires manual separation of integer and fractional parts. This approach is used because:

- Embedded systems often avoid floating-point printing libraries
- Manual formatting reduces memory footprint
- Display precision is fully controlled

The integer part represents the whole-number current value, while the fractional part provides fine resolution suitable for load monitoring.

Voltage Display Logic

Voltage is displayed as an integer RMS value. This reflects common energy meter practice, where voltage is typically shown as a rounded RMS quantity. The value is placed at a fixed screen position to maintain a stable and readable layout.

Active Power Display

Active (real) power is displayed in watts as an integer value. Since active power already represents an averaged quantity over one second, integer display provides sufficient resolution while keeping the interface uncluttered.

Energy Display in Kilowatt-Hours

Energy is displayed in kilowatt-hours with **three decimal places**, reflecting accumulated consumption over time. Similar to current formatting:

- Integer and fractional parts are extracted manually
- Zero padding is applied to ensure consistent digit width
- This format closely resembles commercial energy meters

Displaying energy in kWh makes the system suitable for billing and long-term monitoring applications.

Power Factor Display Logic

Power factor is displayed as a normalized value between 0 and 1. Special handling ensures:

- Values very close to unity are shown as exactly 1.00
- Leading zeros are displayed correctly for values less than 1
- The display remains intuitive and user-friendly

Power factor visualization helps users understand load characteristics and efficiency.

Frequency Display

Frequency is displayed as an integer value in hertz. Since frequency is derived from zero-crossing counts over a one-second window, integer resolution is both accurate and stable.

OLED Buffer Update

After all values are written into the display buffer, the buffer is transmitted to the OLED controller. This single update operation:

- Minimizes I2C traffic
 - Ensures flicker-free display updates
 - Keeps display timing predictable
-

Role in System Architecture

This function is called once per completed measurement window, after all DSP and energy calculations are finalized. It acts as the **final output stage**, bridging numerical computation and human-readable presentation.

```
// UART LOGGING (Send plain text via Serial)
UART2_SendString("\r\n--- UPDATE ---\r\n");
UART2_SendString("V: "); UART2_SendNumber((int)v_rms);
UART2_SendString(" I: "); UART2_SendNumber((int)i_int); UART2_SendString("."); if(i_dec<10) {UART2_SendString("0");} UART2_SendNumber(i_dec);
UART2_SendString(" W: "); UART2_SendNumber((int)active_power);
UART2_SendString(" E: "); UART2_SendNumber((int)e_int); UART2_SendString("."); if(e_dec<100) {UART2_SendString("0");} if(e_dec<10) {UART2_SendString("0");} UART2_SendNumber(e_dec);
UART2_SendString(" PF: "); UART2_SendNumber((int)pf);
UART2_SendString(" F: "); UART2_SendNumber((int)frequency);
UART2_SendString("\r\n");
```

UART-Based Text Logging of Measurement Parameters

This step describes the **UART logging mechanism** used to transmit real-time measurement data as plain text over a serial interface. The logging output mirrors the values shown on the OLED display and serves as an external monitoring, debugging, and data-recording interface.

Purpose of UART Logging

UART logging provides a **human-readable serial output** that allows:

- Monitoring of system behavior without relying on the display

- Capturing measurement data on a PC or external logger
- Debugging and validation of signal processing results
- Verification during calibration and testing phases

Serial logging is especially useful during development and long-term observation.

Structured Log Frame Generation

Each logging cycle begins with a clear **update delimiter**, marking the start of a new measurement frame. This delimiter helps:

- Separate consecutive measurement updates
- Improve readability in serial terminals
- Simplify log parsing for external tools or scripts

The logging format is consistent across updates, making it suitable for automated data collection.

Voltage Logging

The RMS voltage value is transmitted first. Voltage is logged as an integer value, reflecting common electrical measurement conventions and matching the OLED display representation.

This value confirms the measured line voltage and serves as a reference for interpreting other parameters.

Current Logging with Decimal Precision

Current is transmitted with **two decimal places**. To achieve this:

- The integer part is sent first
- The decimal point is transmitted explicitly
- The fractional part is zero-padded when required

This approach avoids floating-point string formatting while still providing sufficient resolution for load analysis.

Active Power Logging

Active (real) power is transmitted in watts as an integer value. Since active power represents an averaged quantity over a one-second window, integer precision is adequate and stable.

This value reflects actual energy consumption by the connected load.

Energy Logging in Kilowatt-Hours

Energy consumption is logged in **kilowatt-hours**, with three decimal places of resolution. The output formatting:

- Separates integer and fractional parts
- Applies leading zero padding to maintain fixed width
- Matches standard energy meter presentation formats

This makes the serial output suitable for billing verification and long-term monitoring.

Power Factor Logging

Power factor is transmitted as a numeric value representing the ratio of active power to apparent power. Logging power factor helps:

- Identify inductive or capacitive load behavior
 - Assess power quality and efficiency
 - Validate phase alignment between voltage and current
-

Frequency Logging

The measured AC frequency is logged as an integer value in hertz. Since frequency is derived from zero-crossing counts over a one-second window, the logged value is both stable and accurate.

End-of-Frame Termination

Each logging update ends with a line termination sequence. This:

- Ensures clean formatting in serial terminals
 - Separates updates clearly
 - Simplifies post-processing of log data
-

Role in System Architecture

UART logging operates in parallel with the OLED display update. While the display provides local visualization, UART logging enables:

- Remote observation
- Data archival
- Integration with external monitoring systems

This dual-output approach enhances system usability and robustness.

THANK YOU :)